

ALGORITMI e STRUTTURE DATI

Indice

0. [Algoritmi di Fibonacci](#)
1. [Notazione Asintotica](#)
2. [Equazioni di Ricorrenza](#)
3. [Algoritmi di Ordinamento](#)
4. [Strutture Dati Elementari](#)
5. [Alberi e Alberi Binari](#)
6. [Alberi Binari di Ricerca](#)
7. [Alberi AVL](#)
8. [Tabelle Hash](#)
9. [Heap Binari](#)
 - 9.2 [Code con Priorità](#)
10. [Union Find](#)
11. [Tecniche Algoritmiche](#)
12. [Grafi](#)
13. [Algoritmi di Visite dei Grafi](#)
14. [Minimum Spanning Tree](#)
15. [Cammini Minimi](#)

0^ Lezione

Introduzione agli algoritmi – Fibonacci

Idea

Quando si ha un algoritmo si vuole che il programma su cui si esegue tale algoritmo sia performante. Un algoritmo semplice ed intuitivo, ma che richiede troppo tempo di esecuzione e troppe risorse per la sua esecuzione è inutile. Quindi prima di implementare un algoritmo bisogna tenere in considerazione degli aspetti, uno fra tutti è per l'appunto il tempo di esecuzione.

Fibonacci

I numeri di Fibonacci sono parecchio conosciuti per la loro natura ricorsiva; tuttavia, esistono diversi molti algoritmi che prevedono l'implementazione dei numeri di Fibonacci. Soffermiamoci a vedere qualche algoritmo (per la precisione 5) e definiamo quale tra essi è il più veloce o il più lento.

Fibonacci

1^ Algoritmo

$$F_n = \frac{1}{\sqrt{5}} \left(\phi^n - \hat{\phi}^n \right)$$

$$\phi = \frac{1+\sqrt{5}}{2} \approx 1.618 \text{ e } \hat{\phi} = \frac{1-\sqrt{5}}{2} \approx -0.618$$

Il primo algoritmo è costituito da una formula, formula che per il suo calcolo utilizza dei numeri reali, numeri che il calcolatore non è in grado di esprimere al massimo. Inoltre, tale formula prevede delle approssimazioni, che si propagano sempre di più all'aumentare dei numeri da calcolare.

Ad esempio, quando si va a calcolare $F(18)$ con questa formula si ottiene il numero 2583, tuttavia, il 18^ numero della sequenza di Fibonacci è pari a 2584. Tale algoritmo è quindi inefficiente, in quanto crea delle approssimazioni, che portano ad un errore, che all'inizio può sembrare irrilevante, ma che alla lunga può inficiare parecchio sulla correttezza della computazione.

2^ Algoritmo

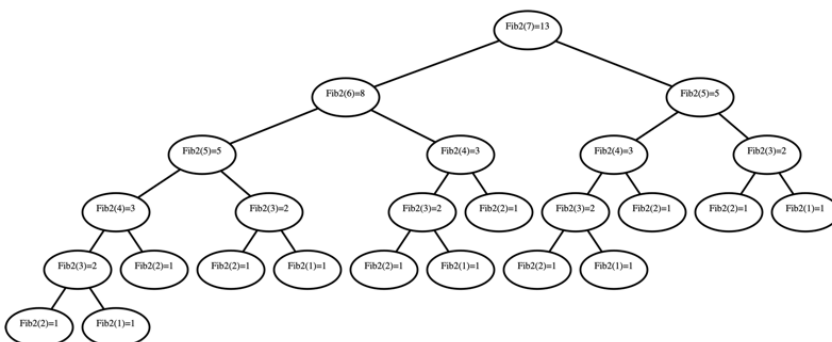
Il secondo algoritmo è ricorsivo, quindi è l'algoritmo per natura più corretto, completo ed intuitivo.

```
1: function FIB2(INT n) → INT
2:   if n ≤ 2 then
3:     return 1
4:   else
5:     return FIB2(n - 1) + FIB2(n - 2)
```

Tale algoritmo, seppure sia compatibile con la natura ricorsiva dei numeri di Fibonacci è il meno efficiente; ciò deriva proprio dalla ricorsione, che impedisce una corretta efficienza. L'algoritmo affianco durante la sua

computazione genera tanti activation record (allocazioni di memoria) che devono essere ricostruite (ricomposte/ricollegate tra di loro) una volta terminato il programma. Questo eccessivo uso di memoria può portare a sovraccaricare il calcolatore (andando a generare un errore del tipo *out-of-memory*). Inoltre, tale algoritmo duplica le chiamate ricorsive, ovvero calcola più volte la stessa cosa. Come si può vedere dall'albero binario degli activation record, lo stesso nodo viene computato più volte. Ciò è dovuto dal fatto che tutto il sottoalbero di sinistra è completamente contenuto e

calcolato già all'interno del sottoalbero di destra. Senza stare ad analizzare la totalità della composizione e della struttura, ma limitandoci ad osservare solo le foglie, siamo davanti a 13 diversi activation record differenti volti a dare lo stesso identico valore in output.



Calcolare il tempo di esecuzione

Contare i secondi o il numero di istruzioni in linguaggio macchina sarebbe irrilevante, in quanto dipenderebbe dal calcolatore su cui il programma è in esecuzione.

Per calcolare il tempo di esecuzione si ricava quindi una funzione che considera il numero di **operazioni elementari** all'interno dello pseudocodice, dove per operazione elementare si intende un'istruzione di immediata istruzione, che quindi non richiede chiamate di funzioni.

Nel caso del secondo algoritmo di Fibonacci estraiamo una **relazione di equivalenza** che ci permette di calcolare il numero di nodi che vengono a crearsi.

$$T(n) = \begin{cases} 1 & n \leq 2 \\ T(n-1) + T(n-2) + 1 & n > 2 \end{cases}$$

Vediamo quindi che se il numero in input è minore o uguale a 2 abbiamo direttamente un output (unico nodo) quindi direttamente un'operazione elementare. Se invece il numero in input è maggiore a 2 si hanno tutti i nodi del sottoalbero di sinistra, più tutti i nodi del sottoalbero di destra più un nodo che rappresenta il valore finale in output.

Data questa relazione di equivalenza è quindi possibile capire il numero di nodi che vengono a crearsi durante l'esecuzione di questo programma, nonché il limite superiore inferiore di nodi creati.

Per il calcolo del **limite inferiore** si sfrutta il fatto che $T(n-2) < T(n-1)$, e da una generalizzazione si ha che il tale limite corrisponde a $\sqrt{2}^n$. Per il **limite superiore** si sfrutta che $T(n-1) > T(n-2)$, che da una generalizzazione al k -esimo passo si ha che il limite è pari a $2^n + 1$ nodi.

Per vedere come si è arrivati alla generalizzazione è utile guardare le slides 18 – 19 del primo set di slides relative all'introduzione degli algoritmi. Non è riportato in questo file in quanto la spiegazione è già abbastanza chiara e sufficiente nelle slides illustrate a lezione.

3° Algoritmo

Il terzo algoritmo usa un array, di lunghezza n , dove l' n -esimo elemento dell'array viene calcolato facendo la somma tra (l' n -esimo - 1) e (l' n -esimo - 2). Il terzo algoritmo, quindi cerca di riprodurre quell'approccio ricorsivo, ma non utilizzando la ricorsione, ma un array di interi che va man mano a costruirsi. Quando si inserirà un numero alla posizione k -esima = numero in input, l'output sarà pari al numero salvato in tale posizione.

Esempio: $F(5) \rightarrow \text{Array: } [0, 1, 2, 3, 5, \dots] \rightarrow F(5) = 5$

$F(7) \rightarrow \text{Array: } [1, 1, 2, 3, 5, 8, 13, \dots] \rightarrow F(7) = 13$.

Dato lo pseudocodice di questo algoritmo si hanno 4 operazioni elementari (creazione array,

```
1: function FIB3(INT  $n$ )  $\rightarrow$  INT
2:   LET  $F[1, \dots, n]$  BE A NEW ARRAY OF INT
3:    $F[1] = 1$ 
4:    $F[2] = 1$ 
5:   for  $i = 3, \dots, n$  do
6:      $F[i] = F[i-1] + F[i-2]$ 
7:   return  $F[n]$ 
```

inserimento valore 1 alla posizione 1, inserimento valore 1 alla posizione 3 e ritorno del valore). Successivamente il ciclo viene eseguito per $n-2$ volte, la somma al suo interno viene a sua volta eseguita per $n-2$ volte e anche la guardia viene aggiornata per $n-2$ volte.

Quindi per $n \leq 2$ costo di $F3(n) = 4$, invece, per $n > 3$ costo di $F3(n) = 4 + 3(n-2) = 3n - 2$

4° Algoritmo

Il quarto algoritmo usa solo 5 variabili, non usando quindi l'array, in quanto è superfluo avere in memoria un certo numero di Fibonacci precedente al numero di Fibonacci richiesto. Questo

algoritmo, quindi, utilizza un numero costante di variabili e funziona sovrascrivendo il più vecchio valore di Fibonacci calcolato.

```

1: function FIB4(INT  $n$ )  $\rightarrow$  INT
2:    $a = 1$ 
3:    $b = 1$ 
4:   for  $i = 3, \dots, n$  do
5:      $c = a + b$ 
6:      $a = b$ 
7:      $b = c$ 
8:   return  $b$ 

```

Si hanno quindi 3 operazioni elementari (assegnamento di a , assegnamento di b , ritorno del valore) mentre si hanno 5 operazioni che vengono eseguite per $n - 2$ volte (ciclo, aggiornamento guardia e aggiornamento dei valori all'intero del ciclo)

Quindi per $n \leq 2$ costo di $F4(n) = 3$, invece, per $n > 3$ costo di $F4(n) = 3 + 5(n - 2) = 5n - 7$.

A primo impatto questo algoritmo risulta meno efficiente del terzo, tuttavia, vedremo nella tabella riassuntiva che il tempo di esecuzione di questo algoritmo è minore del tempo di esecuzione del terzo, banalmente perché le istruzioni sono più semplici, e gli operandi possono stare direttamente nei registri della CPU invece che essere portati in memoria, letti e scritti dalla memoria.

5^ Algoritmo

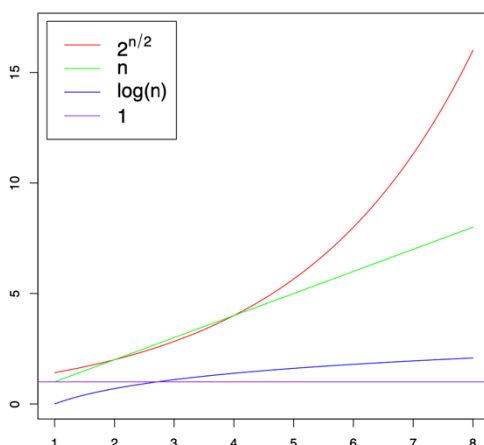
Questo algoritmo ha un costo logaritmico, in quanto usa una proprietà delle matrici che permettono di calcolare un numero di Fibonacci. Tuttavia, la sua applicazione è piuttosto complicata, e riportarla qui non sarebbe utile.

Per vedere il 5^ Algoritmo è utile andare a guardare le slides dalla 25 – 29 della presentazione del capitolo di riferimento.

Tabella Riassuntiva degli Algoritmi

Algoritmo	Limiti inf/sup	Uso di Memoria
Formula	0	0
Ricorsivo	$\sqrt{2^n} \mid 2^n + 1$	Esponenziale
Array	$4 \mid 3n - 2$	Proporzionale Input
Costanti (migliore)	$3 \mid 5n - 7$	Costante
Matriciale	X	Logaritmico

Grafico degli andamenti degli Algoritmi



In rosso è rappresentato il limite inferiore dell'algoritmo ricorsivo di Fibonacci, in verde è rappresentato il costo l'uso lineare di memoria dell'algoritmo con l'array. In Blu è rappresentato il costo logaritmico dell'algoritmo matriciale e in viola l'uso costante di memoria del quarto algoritmo.

1^a Lezione

Comportamento Asintotico Algoritmi

Abbiamo precedentemente detto che tutte le funzioni che calcolano il costo degli algoritmi sono relative al costo per **operazioni elementari**, dove per operazioni elementari si intendono (assegnamenti, confronti, matematica di base...).

Data quindi una funzione $F(n)$ abbiamo uno scopo, ovvero **analizzare il tempo di calcolo e l'occupazione di memoria** degli algoritmi in termini di **dimensione dell'input**. Tuttavia, sapere quanti secondi impiega un algoritmo per la sua esecuzione, o sapere quanti MB di memoria occupa non è molto utile, perché come detto precedentemente tutto ciò dipende dalla macchina su cui si esegue il programma implementativo dell'algoritmo.

Comportamento Asintotico

Per capire l'efficienza di un algoritmo ci si avvale del comportamento asintotico dell'algoritmo. Tale comportamento ignora tutti i termini di ordine inferiore (comprese costanti moltiplicative o additive) e si occupa solamente di descrivere **quanto velocemente** tempo e memoria crescono al **crescere** della **dimensione** dell'input. Questa relazione di crescita (confrontata con l'input) permette di confrontare algoritmi differenti che risolvono lo stesso problema, **indipendentemente** dall'hardware su cui sono eseguiti.

Ricapitolando, il comportamento asintotico permette di caratterizzare il rate di crescita delle funzioni di costo associate agli algoritmi, in termini di dimensione dell'input.

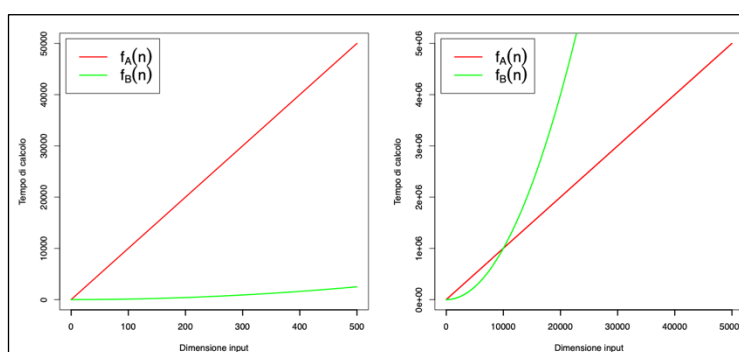
Funzioni di Costo

Dato un $n \geq 0$ (input) si indica con $F(n) \geq 0$ la **quantità di risorse** (tempo di calcolo oppure occupazione di memoria) **impiegate** da un **algoritmo** su un **input di dimensione n** .

Date quindi due funzioni siamo interessati a valutare il **rate di crescita** di queste due funzioni, ovvero quante operazioni in più vengono eseguite all'aumentare dell'input.

È possibile che a primo impatto un algoritmo sembri nettamente più veloce di un altro, tuttavia, questo comportamento deve essere analizzato con un input che tende all'infinito. Questa analisi, quindi, ricade nel comportamento asintotico degli algoritmi che prevedono il calcolo del limite che tende all'infinito del rapporto di funzioni.

Esempio: $F_A(n) = 10^2 n$ | $F_B(n) = 10^{-2} n^2$. $F_A(n)$ sembra sempre più lenta rispetto a $F_B(n)$, tuttavia al raggiungimento di un certo valore di n , la seconda funzione ha un costo più grande rispetto alla prima. Questo è possibile notarlo anche dal grafico sottostante.



Paragoni – Funzioni di Costo

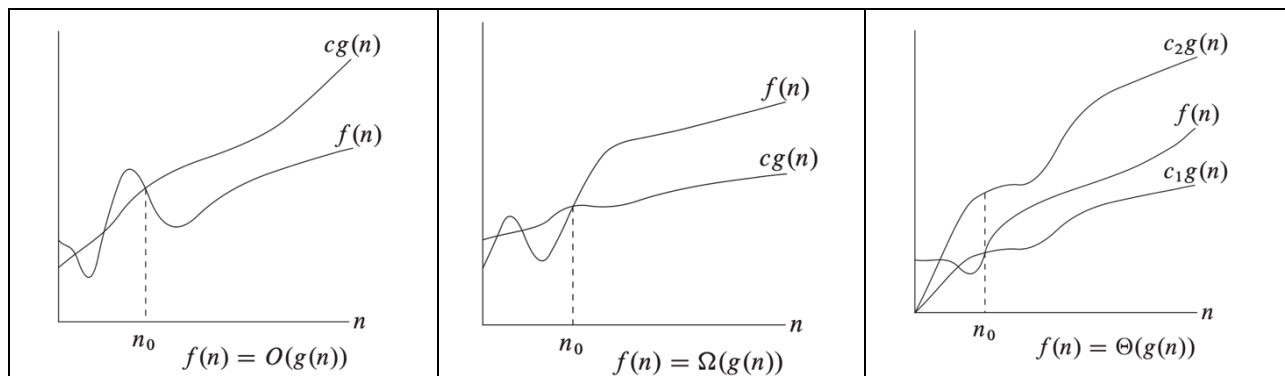
Per paragonare due funzioni, si usano diverse notazioni che permettono proprio di identificare quale delle due funzioni ha il costo minore, e quale ha il costo maggiore.

Le notazioni che vengono utilizzate sono tre: $O(g(n))$; $\Omega(g(n))$; $\theta(g(n))$.

Nota! Si è interessati a tutte quelle funzioni che prendono in input un valore positivo (Dominio) e restituiscono in output un valore positivo (Codominio)

O – Grande: $O(g(n))$

Date due funzioni, f e g , diciamo che $f(n) = O(g(n))$ quando dopo un certo input e per qualche costante c_0 , la funzione $f(n)$ ha un ordine di crescita minore rispetto a $g(n)$.

**Omega – Grande: $\Omega(g(n))$**

Date due funzioni, f e g , diciamo che $f(n) = \Omega(g(n))$ quando dopo un certo input e per qualche costante c_0 , la funzione $g(n)$ ha un ordine di crescita maggiore rispetto a $f(n)$.

Theta: $\theta(g(n))$

Date due funzioni, f e g , diciamo che $f(n) = \theta(g(n))$ quando dopo un certo input e per qualche costante c_0 e c_1 , la funzione $f(n)$ ha un ordine di crescita uguale rispetto a $g(n)$.

Graficamente i tre ordini di grandezza sono rappresentabili in questo modo.

Tabella Riassuntiva

	O $O(g(n))$	o $o(g(n))$	Omega $\Omega(g(n))$	omega $\omega(g(n))$	Theta $\theta(g(n))$
Andamento	Maggiore rispetto a f	Maggiore rispetto a f	Minore rispetto a f	Minore rispetto a f	Uguale rispetto a f
Costanti	Per qualche costante c_0	Per tutte le costanti c_0	Per qualche costante c_0	Per tutte le costanti c_0	Per qualche costante c_0, c_1
Implicazione		definizione: $o(g(n)) \rightarrow O(g(n))$		definizione: $\omega(g(n)) \rightarrow \Omega(g(n))$	definizione: $\theta \rightarrow O \wedge \Omega$
Calcolo limite (confrontabile)		$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$		$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$	$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = k > 0$
Limite / Bound	Upper Bound	Upper Bound	Lower Bound	Lower Bound	Costante

Nota! L'Implicazione tra o – piccolo e omega – piccolo non ha due versi, ma solo uno, inoltre, il limite è calcolabile e quindi gli ordini di grandezza sono confrontabili se le funzioni sono confrontabili tra di loro, per esempio, una funzione lineare non è confrontabile con una funzione che oscilla su un intervallo al variare dell'input. Quindi, $f(n) = n$ e $g(n) = n^{\sin(n)}$ non sono confrontabili.

Per esercizi vedere le slides 1 – 14 relative agli esercizi della notazione asintotica degli algoritmi.

Intuitivamente

Funzioni	Numeri reali
$f(n) = O(g(n))$	$f \leq g$
$f(n) = o(g(n))$	$f < g$
$f(n) = \Omega(g(n))$	$f \geq g$
$f(n) = \omega(g(n))$	$f > g$
$f(n) = \Theta(g(n))$	$f = g$

Incrociando la tabella precedente, con i grafici annessi, si possono trarre conclusioni riguardo gli ordini di grandezza delle funzioni di costo (sempre per operazioni elementari). Questa tabella è quindi un'interpretazione per il confronto tra ordini di crescita asintotici di funzioni in analogia con il confronto di numeri reali.

Siano $a, b \in \mathbb{R}$ codimini di $f(n) \mid g(n)$ si ha che solo una delle seguenti affermazioni è vera: $a < b$; $a > b$; $a = b$.

Queste relazioni tra a, b non possono verificarsi (o non

possono essere sempre verificate) se una delle due funzioni in analisi oscilla al variare dell'input.

Si parla quindi di **rate di crescita asintotici non confrontabili**.

Proprietà

La notazione asintotica gode di alcune proprietà, che nella maggior parte dei casi vengono utilizzate implicitamente; tuttavia, è utile riportarle per maggiore chiarezza.

Transitività: $f(n) = O(g(n)) \rightarrow g(n) = O(h(n)) \rightarrow f(n) = O(h(n))$ vale anche per le altre notazioni

Riflessività: $f(n) = O(f(n))$ una funzione è un O -grande di sé stessa

Simmetria: $g(n) = \Theta(f(n)) \Leftrightarrow f(n) = \Theta(g(n))$

Simmetria Trasposta: $f(n) = O(g(n)) \Leftrightarrow g(n) = \Omega(f(n))$ vale anche per o / omega - piccolo

Ordine di Crescita in equazioni

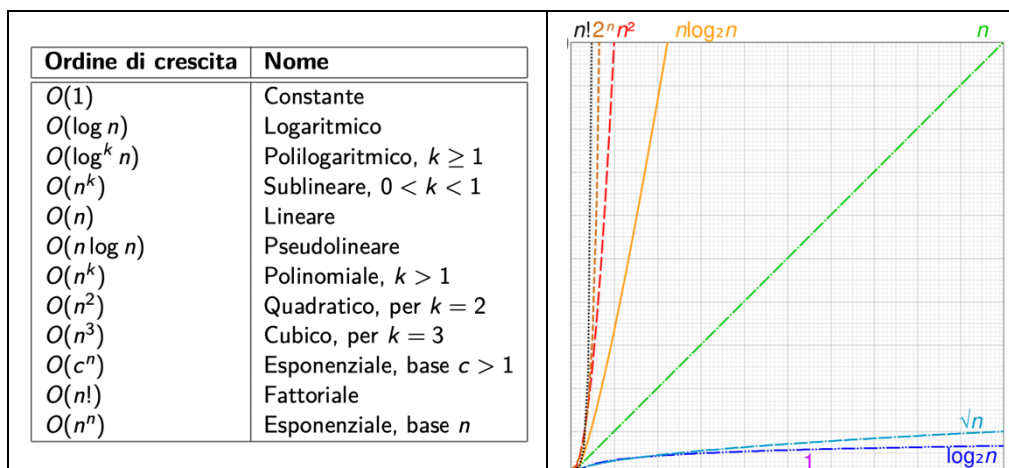
A volte capita che si fanno degli abusi matematici di questo tipo: $2n^2 + 3n + 1 = 2n^2 + \Theta(n)$.

Questo viene fatto per dire che l'equazione dipende innanzitutto dal quadrato del numero dato in input, a cui però viene aggiunta una crescita asintotica costante dello stesso numero dato in input.

In effetti pratici questa notazione non cambia nulla, ma fa capire subito che l'andamento oltre ad avere anche una parte quadratica ha anche un andamento lineare.

Ordini di Crescita Comuni

Nella tabella sottostante sono rappresentati gli ordini di crescita più comuni (dal più lento al più veloce), e per alcuni di essi è anche rappresentato il grafico dell'andamento di crescita. In generale, una volta che si conosce il grafico è possibile dedurre quale algoritmo è più lento / più veloce.



Complessità Computazionale

Per complessità computazionale si intendono tutte le risorse di calcolo (tempo ed energia) necessarie ad eseguire un algoritmo. Un algoritmo, tuttavia non è stabile, ma varia in base a tre casi, ovvero caso pessimo, caso ottimo e caso medio. Per intenderci, un algoritmo di ricerca di un elemento all'interno di un array non performa sempre e solo in condizioni uguali, infatti, se il numero da cercare è nella prima posizione, allora l'algoritmo impiegherà un tempo minimo, d'altro canto, se il numero non è presente

all'interno dell'array, l'algoritmo dovrà scansionare tutto l'array, portando quindi ad un'inefficienza generalizzata. Da ciò si deduce che i comportamenti degli algoritmi non sono sempre così puliti e lineari, è quindi importante studiare i casi che possono rallentare i processi.

I casi più interessanti di studio sono quindi due: il caso medio e il caso pessimo (anche il caso ottimo è interessante, ma in generale riguarda casi puramente fortunati).

Nota! Il caso medio è difficilissimo da individuare, perché bisogna trovare un comportamento medio generalizzabile su tutti i casi medi che non ricadono né all'intero del caso pessimo e né all'interno del caso ottimo. A volte capita che ci si riconduca solamente a trovare degli upper-bound e dei lower-bound. All'esame non verrà chiesto di trovare il caso medio, a meno che non sia semplice.

Caso Ottimo – Pessimo – Medio

Caso Ottimo	Caso Pessimo	Caso Medio
Descrive il comportamento in condizioni ottimali , per esempio, in una ricerca lineare, l'elemento cercato è nella prima posizione dell'array	Descrive il comportamento in condizioni sfavorevoli , per esempio, l'elemento cercato compare nella ultima posizione dell'array	Descrive il comportamento su tutti i possibili input , per esempio, in una ricerca lineare il numero cercato è nella posizione centrale dell'array

Caso di Studio – Ricerca Lineare

Sia data la funzione di ricerca lineare all'interno di un elemento all'interno di un array si può analizzare il costo computazionale di tutti i possibili casi.

```

1: function LINSEARCH(ARRAY A[1...n], INT x) → INT
2:   for i = 1, ..., n do
3:     if A[i] == x then
4:       return i
5:   return -1

```

	Caso Ottimo	Caso Pessimo	Caso Medio
<i>Situazione</i>	L'elemento è nella prima posizione, o nelle posizioni primarie dell'array	L'elemento è nell'ultima posizione, o nelle ultime posizioni dell'array	Elemento può trovarsi nelle posizioni centrali dell'array con una probabilità uniforme
Costo	Basta una singola, o poche iterazioni $\Theta(1)$	Serve iterare fino in fondo all'array $\Theta(n)$	$\Theta(n)$

Analisi Costo Medio – Ricerca Lineare

Si parte dal presupposto di avere una probabilità uniforme, altrimenti non si arriverà mai alla conclusione, o almeno non in modo abbastanza semplice. La probabilità è quindi di $\frac{1}{n}$, ma in questo caso non viene inclusa la ricerca fallita (elemento non presente), quindi bisogna aggiornare la probabilità aggiungendo questa casistica: $P_i = \frac{1}{n+1}$.

Ora per trovare il costo medio basta prendere la sommatoria tra il tempo impiegato per ogni iterazione e moltiplicare tale sommatoria per la probabilità generalizzata, quindi:

$$\text{Probabilità} \times \sum (\text{tempo impiegato per ogni iterazione})$$

$$\frac{1}{n+1} * \sum_{i=1}^{n+1} i = \frac{1}{n+1} * \frac{(n+1)(n+2)}{2} = (n+2) * \frac{1}{2} = \Theta(n)$$

Caso di Studio – Ricerca Binaria

Sia data la funzione di ricerca binaria, analizzarne il costo nel caso ottimo e nel caso pessimo.

```

1: function BINSEARCH(Array A[1...n], INT x) → INT
2:   i = 1, j = n
3:   while i ≤ j do
4:     m = (i + j) / 2
5:     if A[m] == x then
6:       return m
7:     else if A[m] < x then
8:       i = m + 1
9:     else
10:      j = m - 1
11:   return -1

```

Nota! Il caso medio è interessante, perché da come risultato lo stesso costo del caso pessimo. Tuttavia, il processo che porta all'individuazione del caso medio è molto complicato.

Se si vuole studiare il costo medio guardare le slides relative al capitolo di riferimento.

	Caso Ottimo	Caso Pessimo
Situazione	L'elemento cercato è in posizione centrale	L'elemento cercato è in una posizione centrale che è ricavata dopo n iterazioni. Oppure l'elemento non è presente
Costo	$\Theta(1)$	$\Theta(\log n)$

Analisi Costo Pessimo – Ricerca Binaria

Per aiutarci ad analizzare questo meccanismo possiamo avvalerci di una tabella e poi da quella tabella possiamo trovare il vero e proprio costo computazionale.

Iterazione	1	2	3	4	k-esima
Dimensione Array	n	$n/2$	$n/2^2$	$n/2^3$	$n/2^{k-1}$

Abbiamo quindi una situazione del genere:

$$\Theta(n) + \Theta\left(\frac{n}{2}\right) + \Theta\left(\frac{n}{2^2}\right) + \dots + \Theta\left(\frac{n}{2^{k-1}}\right)$$

Dobbiamo capire quanto vale $\Theta\left(\frac{n}{2^{k-1}}\right)$, nonché quando si ferma la ricerca binaria. Dal frammento di codice sappiamo che la ricerca binaria termina quando lo spazio di dimensione dell'array è minore o uguale a 1, quindi possiamo fare:

$$\begin{aligned} \frac{n}{2^{k-1}} = 1 &\rightarrow n = 2^{k-1} \rightarrow \log_2 n = \log_2 2^{k-1} \rightarrow \log_2 n = k - 1 * \log_2 2 \\ \log_2 n = k - 1 &\rightarrow k = \log_2 n + 1 \rightarrow k = \log_2 n \rightarrow k = \log n \\ &\quad \Theta(\log n) \end{aligned}$$

Esercizi Proposti:

1^o Esercizio: Calcolare i costi computazionali delle due funzioni.

☆☆☆

Esercizio senza soluzioni, ma reperibili dalle slides relative agli esercizi

```

1: function MYSTERY1(INT n) → INT
2:   tot = 0
3:   for i = 1, ..., 2100 do
4:     tot = tot + MYSTERY2(n)
5:   return tot

6: function MYSTERY2(INT n) → INT
7:   LET A[1...2100n][1...2100n] BE A NEW ARRAY
8:   for i = 1, ..., 2100n do
9:     for j = 1, ..., 2100n do
10:      A[i] = 0
11:   return A[n][n]

```

2^ Esercizio: Calcolare i costi computazionali delle due funzioni.

☆☆☆

Esercizio con soluzione, ma reperibile anche dalle slides relative agli esercizi

```

1: function MYSTERY1(INT n) → INT
2:   tot = 0
3:   x = 0
4:   while n ≥ 1 do
5:     n = n/2
6:     x = x + 1
7:     tot = tot + MYSTERY2(x)
8:   return tot

9: function MYSTERY2(INT n) → INT
10:  LET A[1...n] BE AN ARRAY
11:  for i = 1...n do
12:    A[i] = 0
13:  return A[n]
```

ANALISI MYSTERY2()

PER n volte il ciclo si ripete, NON ci sono
CONDIZIONI CHE LO FERMANO: $\Theta(n)$

ANALISI MYSTERY1()

FINCHÉ n ≥ 1 CONTINUA IL CICLO
FACCO TABELLA → ANALIZZO

PER	1	2	3	
n	n	n/2	n/4	$n/2^{k-1}$
x	1	2	3	x

Si ha: $\Theta(1) + \Theta(2) + \Theta(3) + \dots + (n)$ quindi:

$$\sum_{i=1}^k i \rightarrow \text{lo faccio per } n/2^{k-1} \rightarrow \text{Trovo quindi } k$$

$$\frac{n}{2^{k-1}} = 1 \rightarrow n = 2^{k-1} \rightarrow \log_2 n = \log_2 2^{k-1} \rightarrow$$

$$\rightarrow \log_2 n = k-1 \cdot \log_2 2 \rightarrow \log_2 n = k-1 \rightarrow k = \log_2 n + 1$$

$$\rightarrow k = \log_2 n + 1$$

aggiungo sommatoria: $\sum_{i=1}^{\log n} i \rightarrow \text{da formula generale: } \frac{\log n (\log n + 1)}{2} = \Theta(\log^2 n)$

Questo nuovo argomento viene trattato come terzo argomento, tuttavia è più un'estensione del capitolo riguardante la notazioni asintotica. Quindi le slides relative si riferiscono al terzo capitolo.

Equazioni di Ricorrenza

Le equazioni di ricorrenza vengono utilizzate per capire l'andamento asintotico degli algoritmi ricorsivi, quindi, questo sistema è da applicarsi **solamente** con tali algoritmi, altrimenti si perde gran parte del tempo a scrivere l'equazione di ricorrenza per poi mai usarla.

Un'equazione di ricorrenza che abbiamo già visto è quella relativa all'algoritmo di Fibonacci.

Principalmente ci sono due modi per trovare il costo computazionale di un algoritmo ricorsivo, il primo è l'algoritmo iterativo, che può risultare molto lungo e macchinoso, l'altro metodo è il Master Theorem, che però si può applicare solo quando l'equazione di ricorrenza assume una certa forma.

Metodo Ricorsivo

Il metodo ricorsivo abbastanza lungo e tedioso quando l'equazione di ricorrenza prevede più chiamate ricorsive, tuttavia, in situazioni semplici è abbastanza immediato.

L'idea dietro a questo metodo è quello di fare meccanicamente l'esecuzione dell'algoritmo, successivamente, l'obiettivo è quello trovare una generalizzazione utile a risolvere il costo computazionale.

Esempio

Dato questo algoritmo eseguire qualche passo con il metodo iterativo, generalizzare e trovare costo

$$T(n) = \begin{cases} 1 & n = 1 \\ T(n/2) + c & n > 1 \end{cases}$$

$$\begin{aligned}
 T(n) &= T(n/2) + c \rightarrow \\
 &T(n/4) + c + c \rightarrow \\
 &T(n/8) + c + c + c \rightarrow \\
 &T(n/2^i) + ic \quad \text{generalizzazione}
 \end{aligned}$$

Abbiamo adesso la formula generalizzata, ma dobbiamo capire quando la ricorsione termina; quindi, quando $n/2^i$ raggiunge il caso base, quindi 1.

Ora possiamo procedere:

$$n/2^i = 1 \rightarrow n = 2^i \rightarrow \log_2 n = \log_2 2^i \rightarrow \log_2 n = i \cdot \log_2 2 \rightarrow \log_2 n = i \rightarrow i = \log_2 n \rightarrow \Theta(\log 2)$$

Dopo questa serie di passaggi, quindi abbiamo capito che il costo computazionale di questo algoritmo è pari a $\Theta(\log 2)$.

Secondo Esempio

Dato questo algoritmo eseguire qualche passo con il metodo iterativo, generalizzare e trovare costo.
Nota! Il processo è più lungo e difficile e richiede l'uso di formule notevoli.

$$T(n) = \begin{cases} 1 & n = 1 \\ T(n/2) + n & n > 1 \end{cases}$$

Procediamo con l'iterazione e con la generalizzazione:

$$\begin{aligned} T(n) &= T(n/2) + n \\ T(n/4) + (n/2) + n \\ T(n/8) + (n/4) + (n/2) + n \\ T(n/16) + (n/2^3) + (n/2^2) + (n/2) + n \\ T(n/2^i) + \sum_{k=0}^{i-1} n/2^k &\rightarrow T(n/2^i) + n * \sum_{k=0}^{i-1} 1/2^k \end{aligned}$$

Capiamo quando la ricorsione termina:

$$T(n/2^i) = 1 \rightarrow \text{come esempio precedente} \rightarrow i = \log n$$

$$\begin{aligned} T(n/2^i) + n * \sum_{k=0}^{i-1} 1/2^k &= \rightarrow \text{da formule notevoli} \\ T(n/2^i) + n * \frac{1/2^i - 1}{1/2 - 1} &= \rightarrow \text{semplificazione} \\ T(n/2^i) + 2n(1 - \frac{1}{2^i}) &\rightarrow \text{ora basta sostituire } i \text{ con } \log n \\ T\left(\frac{n}{2^{\log_2 n}}\right) + 2n \left(1 - \left(\frac{1}{2^{\log_2 n}}\right)\right) &= T(1) + 2n * \left(1 - \frac{1}{n}\right) = 1 + 2n + \frac{2n}{n} = \Theta(n) \end{aligned}$$

Come si può vedere, fare questo metodo può diventare in poco tempo molto complicato, soprattutto se non si riesce a ridurre la sommatoria. Inutile dire che è ancora più complicato se le chiamate ricorsive si raddoppiano. Ora dopo la spiegazione del Master Theorem proviamo a risolvere lo stesso esercizio, e vediamo che la risoluzione è immediata.

Master Theorem

Il Master Theorem è applicabile solo quando si ha una formula di questo genere:

$$T(n) = a T(n/b) + f(n)$$

dove a e b sono costanti e $f(n)$ è una funzione di costo asintoticamente positiva.

Vediamo come applicare il Master Theorem:

$$T(n) = \begin{cases} d & n = 1 \\ aT(n/b) + cn^\beta & n > 1 \end{cases}$$

Definiamo ora $\alpha = \frac{\log a}{\log b}$, da ciò si hanno tre possibili casistiche:

- Se $\alpha > \beta$ allora $T(n) = \Theta(n^\alpha)$
- Se $\alpha = \beta$ allora $T(n) = \Theta(n^\alpha * \log n)$
- Se $\alpha < \beta$ allora $T(n) = \Theta(n^\beta)$

Ora risolviamo l'esercizio di prima con il Master Theorem:

$$\begin{aligned} T(n) &= \begin{cases} 1 & n = 1 \\ T(n/2) + n & n > 1 \end{cases} \\ \alpha = \frac{\log_2 1}{\log_2 1} = 0; \beta = 1; a < \beta &\rightarrow T(n) = \Theta(n^\beta) = \Theta(n^1) = \Theta(n) \end{aligned}$$

Abbiamo visto che il Master Theorem permette di risolvere esercizi in modo rapido e veloce. Per fare altri esercizi è possibile guardare le slides relative al capitolo di riferimento, oppure guardare le vecchie prove d'esame e fare la prima tipologia di esercizi.

In [questo file](#) ci sono già degli esercizi da esame svolti con tanto di soluzione commentata

3^ Lezione

Algoritmi di Ordinamento

L'idea degli algoritmi di ordinamento è molto semplice, si prende un array contenente dei numeri e si ritorna lo stesso array riempito con gli stessi numeri ma scambiati di posizione, in modo che tutti i numeri siano ordinati in ordine crescente (dal più piccolo al più grande).

Creare un algoritmo di ordinamento non è una cosa facile, ma è sicuramente più difficile creare un algoritmo di ordinamento efficiente, quindi classificare il costo computazionale di un problema algoritmico.

Esistono diversi algoritmi di ordinamento, in questo capitolo studieremo due algoritmi per tipologia.

	1^ Algoritmo	2^ Algoritmo
Incrementativi	Selection Sort	Insertion Sort
Divide et Impera	Merge Sort	Quick Sort
Non Comparativi	Counting Sort	Radix Sort

Prima di procedere e analizzare ogni tipologia è meglio definire alcune caratteristiche che un algoritmo di ordinamento può avere, in particolare, si dice che un algoritmo è **in-place** se non si alloca ulteriore memoria durante l'ordinamento, inoltre, si dice che un algoritmo è **stabile** se a parità di valore tra numeri, il numero con lo stesso valore (ma che compare prima del suo omonimo) comparirà con lo stesso ordine anche nell'array ordinato.

Esempio

Array non ordinato				Dopo ordinamento		
2	4	2		2	2	4

Si può vedere che questo algoritmo non è stabile, il "2" compare dopo il "2", anche se il primo numero compariva prima del secondo all'interno dell'array preordinamento

Algoritmi di Ordinamento – Analisi

Algoritmi Incrementali

Sono algoritmi semplici (si vedrà che sono anche quelli più lenti) che costruiscono l'ordinamento un elemento alla volta. Il funzionamento di questi algoritmi è veramente semplice e facile da intuire; infatti, basti pensare a continui confronti tra numeri fino a trovare il numero più piccolo (o il numero

più grande) e mettere tale numero minore nella cella più piccola o più grande dell'array causando uno swap o uno shift di numeri all'intero dell'array.

```

1: function SELECTIONSORT(INT A[1, ..., n])
2:   for i = 1, ..., n - 1 do
3:     m = MIN(A, i, n) ▷ Search min in A[i, ..., n]
4:     ▷ Swap A[m] with A[i]
5:     if m ≠ i then
6:       SWAP(A, i, m)
7:   function MIN(INT A[1, ..., n], INT i)
8:     for j = i + 1, ..., n do
9:       if A[j] < A[i] then
10:        i = j
11:   return i
12: function SWAP(INT A[1, ..., n], INT i, INT j)
13:   tmp = A[i]
14:   A[i] = A[j]
15:   A[j] = tmp

```

Selection Sort

Funzionamento: si ricerca il minor numero (nelle posizioni non ancora ordinate) e si inserisce tale numero nella posizione più piccola dell'array che non è stata ancora ordinata.

Passiamo ora a studiare il comportamento asintotico di tale algoritmo.

Vediamo che alla riga 2 c'è un ciclo che itera per " $n - 1$ " volte, inoltre, c'è una funzione ausiliaria (all'interno del ciclo primario) che ricerca il minimo valore.

Una volta trovato tale valore si effettua uno swap.

Ignoriamo la funzione che gestisce lo swap, in quanto ha costo costante, e guardiamo la funzione ausiliaria **min**.

Tale funzione ricerca il minimo in uno spazio che si riduce ad ogni iterazione del ciclo primario, ciò significa che lo spazio di ricerca del minimo diminuisce di volta in volta. Detto ciò, possiamo definire l'equazione:

$$\theta(n-1) + \theta(n-2) + \dots + 1 = \sum_{i=1}^{n-1} n-i = n \sum_{i=0}^{n-1} i = \frac{n * (n+1)}{2} = \theta(n^2)$$

Questa formula, che non dovrebbe essere di difficile comprensione, dice che per " $n-1$ " volte si ricerca il minimo, e tale minimo costa ogni volta " $n-i$ " iterazioni, dopo una prima semplificazione e il passaggio a una forma generalizzata si riesce ad ottenere il comportamento asintotico della funzione. Tuttavia, ignorando momentaneamente come si è arrivati a tale forma, in quanto non molto importante, soffermiamoci a capire se ci sono dei casi ottimi, pessimi e medi.

In realtà è immediato capire che questo algoritmo ha sempre lo stesso andamento asintotico, questo perché non esistono casi, ma l'algoritmo esegue sempre tutte le iterazioni.

Nota! Algoritmo in-place, ma non stabile, per colpa degli swap.

Insertion Sort

Funzionamento: non si ricerca nessuno specifico valore, ma si parte da un presupposto molto semplice, ovvero si ipotizza che il numero più piccolo affoghi verso il basso, portando più in alto tutti i numeri che sono maggiori.

```

1: function INSERTIONSORT(INT A[1, ..., n])
2:   for i = 2, ..., n do
3:     j = i
4:     while j > 1 and A[j] < A[j-1] do
5:       SWAP(A, j, j-1)
6:       j = j - 1

```

genera casi ottimi/pessimi/medi

Come si può vedere il primo ciclo viene eseguito per " n " volte, mentre il sotto-ciclo while non viene sempre eseguito, generando dei casi ottimi, pessimi e medi.

Idealmente il caso ottimo si verifica in una sola condizione, ovvero quando l'array è già completamente ordinato, mentre il caso pessimo si verifica quando l'array non è ordinato. Il caso medio, invece, si verifica quando l'array è parzialmente ordinato. Per precisione, guardiamo nel dettaglio ogni caso

Insertion Sort – Caso Ottimo

Tutti i numeri sono già ordinati dal più piccolo al più grande, la condizione del ciclo while è sempre falsa; quindi, non si entra mai nel loop. Tuttavia, per terminare il programma è necessario finire l'iterazione primaria, ovvero quella associata al ciclo for.

Detto ciò, il caso ottimo ha un costo pari a: $\theta(n)$

Da notare, che tale costo si applica anche ad array già parzialmente ordinati, sviluppati così:

7	8	9	10	11	3	4	5	6	6
---	---	---	----	----	---	---	---	---	---

Insertion Sort – Caso Pessimo

Analogamente, il caso pessimo si verifica quando l'array non è ordinato, quindi:

$$\sum_{i=2}^n i - 1 = \sum_{i=2}^{n-1} i = \frac{n * (n-1)}{2} = \theta(n^2)$$

Insertion Sort – Caso Medio

Dopo qualche assunzione probabilistica si riesce ad arrivare a una forma generalizzata che porta il costo ad essere pari a: $\theta(n^2)$.

È possibile vedere tutte le assunzioni e i calcoli che portano al costo finale nelle slides di riferimento del capitolo. Tuttavia, non è importante ricordarsi tutti i passaggi, procedimenti ed assunzioni, ma è importante ricordarsi le proprietà e il costo finale, in questo agli esami può venire chiesto.

Nota! Algoritmo in-place e stabile

Algoritmi Divide et Impera

Gli algoritmi di questa categoria sono ricorsivi, ovvero dividono il problema in sottoproblemi più piccoli e più facili da risolvere. Tali algoritmi sono generalmente più veloci degli algoritmi incrementali, seppur essendo ricorsivi. Tale ricorsione, seppur essendo più inefficiente, applicata in modo corretto, ma soprattutto parsimonioso risulta in un incremento complessivo di performance e velocità di ordinamento. In ogni modo, questi algoritmi sono più efficienti degli algoritmi incrementali.

Merge Sort

Funzionamento: si prende l'array e lo si divide a metà, successivamente si continua ricorsivamente questa suddivisione fino a ottenere un unico array di lunghezza 1. Quando si è arrivati alla fine della ricorsione si iniziano a ricostruire tutti i vari frammenti fino a ricostruire un array completamente ordinato.

Nella prossima pagina viene riportato solo lo pseudocodice del **mergesort**, e non quello del **merge**, questo perché la funzione merge seppur sia complicata da comprendere è molto banale; infatti, se si hanno 2 array di lunghezza complessiva 10, la funzione farà 10 iterazioni, quindi: $\theta(n)$

```

1: function MERGESORT(INT A[1, ..., n], INT p, INT r)
2:   if p < r then
3:     q = [(p + r) / 2]
4:     MERGESORT(A, p, q)
5:     MERGESORT(A, q + 1, r)
6:     ▷ Here A[p, ..., q] and A[q + 1, ..., r] are sorted
7:     MERGE(A, p, q, r)

```

Come già fatto nello scorso capitolo, possiamo facilmente capire che la funzione merge ha un costo logaritmico, tale costo logaritmico, tuttavia esiste per due volte. Dopo queste osservazioni è possibile estrarre un'equazione di ricorrenza, che si può risolvere in modo immediato facendo

uso del Master Theorem.

$$T(n) = \begin{cases} 1 & n \leq 1 \\ 2T(n/2) + f(n) & n > 1 \end{cases}$$

dove $f(n) = \Theta(n)$ è il costo della procedura MERGE

Risolvendo l'equazione di ricorrenza si ottiene un costo pari a: $\theta(n \log n)$

Nota! Il Mergesort ha sempre lo stesso costo per tutti e tre i casi (ottimo, pessimo e medio), è un algoritmo stabile, ma non in-place.

Quick Sort

Funzionamento: si prende un pivot, ovvero l'ultimo elemento di un array, si creano due array, nell'array di sinistra vengono messi tutti i numeri minori del pivot, mentre nell'array di destra vengono messi tutti i numeri maggiori del pivot, mentre, il pivot stesso viene posizionato al centro. Successivamente, si riesegue la stessa cosa per ogni sotto array creato

```

1: function QUICKSORT(INT A[1, ..., n], INT p, INT r)
2:   if p < r then
3:     q = PARTITION(A, p, r) possono essere molto sbilanciate le due partizioni
4:     ▷ Here  $\forall i \leq q - 1, A[i] \leq A[q]$  and  $\forall i \geq q + 1, A[i] > A[q]$ 
5:     QUICKSORT(A, p, q - 1)
6:     QUICKSORT(A, q + 1, r)

```

La funzione **partition** si occupa di scegliere un elemento come pivot e creare le due effettive partizioni. Seppure la funzione partition abbia sempre costo costante in ogni caso: $\theta(r - p + 1)$, tale funzione può creare due sotto array completamente sbilanciati, ciò significa avere una ricorsione con

casi pessimi (un array lungo 0 e un array lungo n), casi ottimi (due array esattamente lunghi $n/2$) o casi medi (un array poco più lungo dell'altro).

Prima di passare ad analizzare ogni caso, bisogna dire che l'analisi di ogni caso è molto complessa e richiede numerosi passaggi, questo perché ogni caso prevede la propria equazione di ricorrenza. Visto che non avrebbe senso stare a ricopiare tutti i passaggi, qui viene scritta solo l'equazione di ricorrenza e viene dato il costo computazionale, tutti i vari passaggi possono essere visti nelle slides relative al capitolo di riferimento.

Caso Pessimo: $\theta(n^2) \rightarrow$ Array già ordinato

$$T(n) = \begin{cases} 1 & n \leq 1 \\ T(n-1) + T(0) + n & n > 1 \end{cases}$$

Caso Ottimo: $\theta(n \log n)$

$$T(n) = \begin{cases} 1 & n \leq 1 \\ 2T(n/2) & n > 1 \end{cases}$$

Da risoluzione con Master Theorem

Caso Medio: $O(n \log n) \rightarrow$ Array parzialmente ordinato

Come si può vedere il caso medio è molto più veloce del caso ottimo, questo dipende tutto dalla scelta del pivot. Se il pivot fosse scelto casualmente (con apposita funzione), si riuscirebbe a far ricadere il caso pessimo all'interno del caso ottimo.

Idea Funzionamento Algoritmi Ricorsivi

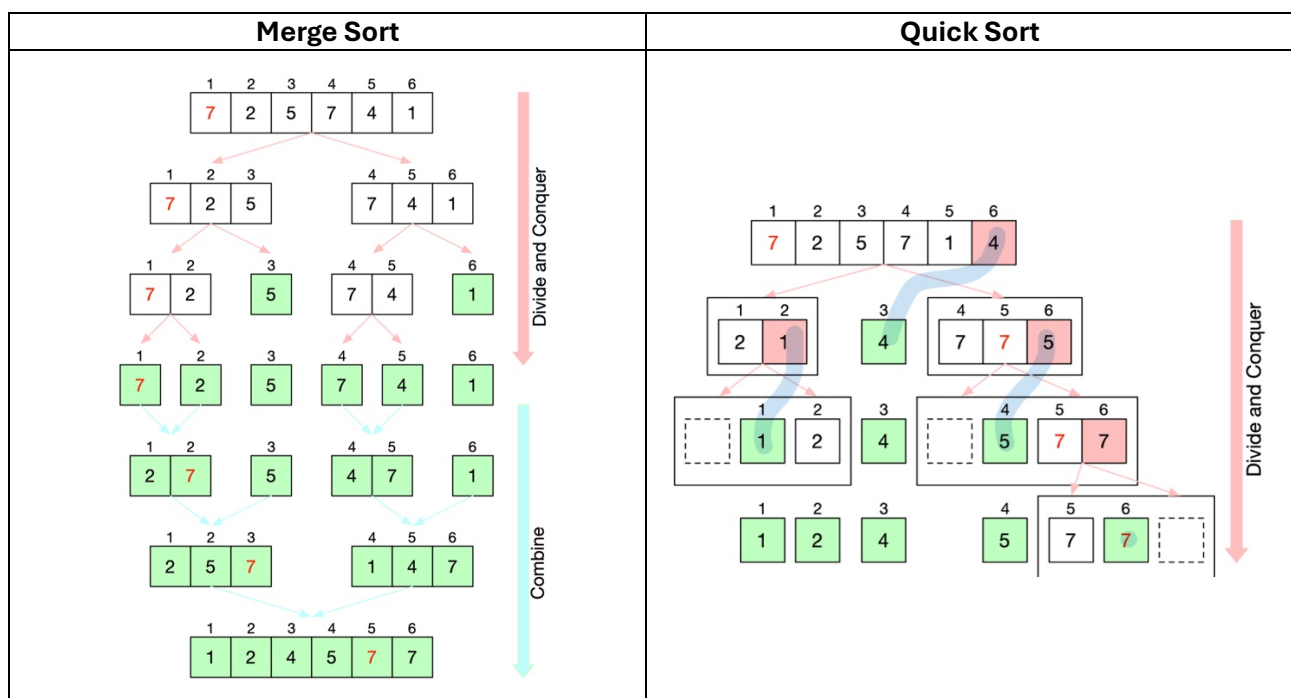


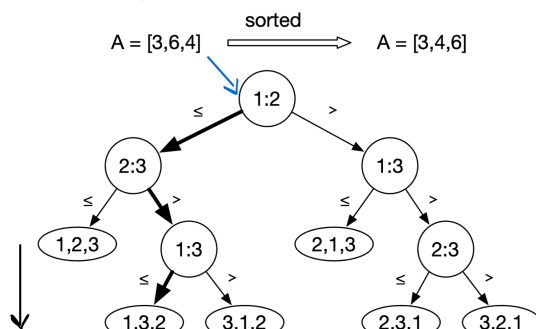
Tabella Riassuntiva

Tabella riassuntiva degli algoritmi di ordinamento comparativi, che quindi contengono procedure che confrontano i valori degli interi contenuti all'interno dell'array.

	Caso Ottimo	Caso Medio	Caso Pessimo	In-Place	Stabile
Selection Sort	$\theta(n^2)$	$\theta(n^2)$	$\theta(n^2)$	Si	No
Insertion Sort	$\theta(n)$	$\theta(n^2)$	$\theta(n^2)$	Si	Si
Merge Sort	$\theta(n \log n)$	$\theta(n \log n)$	$\theta(n \log n)$	No	Si
Quick Sort	$\theta(n \log n)$	$O(n \log n)$	$\theta(n^2)$	Si	No

È possibile fare meglio del Merge Sort per il caso pessimo?

Questa è una domanda interessante, che non è semplice da rispondere in modo immediato. L'unico modo per rispondere questa domanda è quello di costruire un albero binario dei decisioni.



Si può vedere che ogni nodo dell'albero di decisione ha esattamente due figli, e l'altezza complessiva dell'albero di decisione corrisponde al caso pessimo. Inoltre, si vede che il numero di foglie dell'albero è almeno uguale a $n!$, ciò significa che ci sono al massimo tante permutazioni possibili, quanto il fattoriale della lunghezza dell'array.

Ora per il fatto della suddivisione, e per il fatto del fattoriale possiamo scrivere ogni algoritmo comparativo richiede al minimo: $\Omega(\log n!)$ comparazioni, che equivale a $\Omega(n \log n)$, questo significa che è impossibile fare meglio del Merge Sort.

Esistono altri due algoritmi di sorting, tuttavia vengono raramente utilizzati, ma basta sapere che il loro metodo di confronto è diverso rispetto agli altri algoritmi analizzati precedentemente.

È sempre molto importante sapere il loro costo ottimo, medio e pessimo, ma visto il loro scarso utilizzo non viene è inutile riportare in sintesi il loro funzionamento e il loro pseudocodice.

Se si vuole vedere in preparazione all'esame (consigliato) consultare le slides relative al capitolo di riferimento, lì si trovano tutti gli pseudocodici, funzionamenti e sviluppi delle funzioni di costo.

Tabella Finale Riassuntiva

	Caso ottimo	Caso medio	Caso pessimo
SelectionSort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$
InsertionSort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$
MergeSort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$
QuickSort	$\Theta(n \log n)$	$O(n \log n)$	$\Theta(n^2)$
CountingSort	$\Theta(n + k)$	$\Theta(n + k)$	$\Theta(n + k)$
RadixSort	$\Theta(d(n + k))$	$\Theta(d(n + k))$	$\Theta(d(n + k))$

	In place	Stabile
SelectionSort	Si	No
InsertionSort	Si	Si
MergeSort	No	Si
QuickSort	Si	No
CountingSort	No	Si
RadixSort	No	Si

Tutti gli algoritmi sopra descritti approfonditamente sono stati implementati correttamente in Java.

Esercizi

Per esercizi vedere le slides con esercizi relative al capitolo di riferimento, come esercizio è consigliato in particolare l'esercizio 4 alla slides 10, sempre dello stesso pacco di Slides.

4^ Lezione

Strutture Dati Elementari

Struttura dati Dizionario

Contiene un insieme di chiavi univoche, dove ad ogni chiave è associato un dato, ovvero il dato che si vuole memorizzare / inserire all'interno della struttura.

Un dizionario è quindi un prototipo, ovvero descrive le possibili operazioni effettuabili con quella struttura dati, ma nasconde la vera e propria implementazione di quel prototipo, infatti, ad una struttura dati possono corrispondere diverse implementazioni. Ad esempio, il dizionario (che dispone di tre basilari metodi ricerca, inserimento e rimozione) può essere implementato con array.

Condizioni della struttura

La struttura deve essere costruita per contenere un insieme di **chiavi** univoche, dove ogni chiave è associata a dei **dati**. I dati possono essere duplicati, mentre le chiavi sono univoche, questo significa che un inserimento di un dato ad una chiave già presente porta errore

Implementazione

Array – non ordinato

Si usa un array per memorizzare una coppia di informazioni, ovvero, alla posizione 0 c'è un'informazione diversa, o uguale dell'informazione che è salvata alla posizione 1.

Procediamo ad analizzare i costi di ogni metodo definito dalla struttura dati.

Nota! Lo pseudocodice non è riportato, ma si può vedere nelle slides relative al capitolo di riferimento.

Le funzioni, nello pseudocodice prendono due parametri, per il **“d”** si intende il dato, mentre per **“k”** si intende la chiave associata al dato passato come parametro.

Array non ordinato	Search(d)	Insert(d, k)	Remove(d, k)
Caso Ottimo	$O(1)$	$O(1)$	$O(1)$
Caso Pessimo	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Caso Medio	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$

Nota! L'inserimento viene effettuato **solo se** non esiste già una chiave con lo stesso valore, e la rimozione viene effettuata **solo se** esiste una chiave associata a quel dato da rimuovere. È quindi di facile deduzione il motivo per il quale tutti i metodi hanno lo stesso costo computazionale

Array Ordinato

Anche se la struttura dati dizionario implementato come Array non ordinato è generalmente efficiente, questo per il fatto che i costi di ricerca sono tutti lineari, quindi abbastanza veloci si può pensare che un Array ordinato possa fornire prestazioni migliori.

Tuttavia, se si usa un array ordinato per salvare le coppie (Key, Data) la ricerca sarà veloce, in quanto verrà implementata come ricerca dicotomica, tuttavia il problema sorge nell'inserimento o rimozione di un elemento. Il concetto è molto semplice, infatti se ho una cosa ordinata, e la ricerca è molto veloce, devo essere sicuro che dopo l'inserimento o la rimozione di un elemento l'array continui a permettere una ricerca accurata ed efficiente come prima.

Detto ciò, salta subito all'occhio che l'array ordinato migliora il costo rispetto alla funzione **search()**, ma non cambia il costo rispetto alle altre due funzioni definite dalla struttura dati.

Funzione - Insert(*d*, *k*)

```

1: function INSERT(DICT D, KEY k, DATA d)
2:   i = BINSEARCHPOS(D.A, D.size, k)           ▷  $1 \leq i \leq m$ 
3:   if D.A[i].key ≠ k then
4:     RIGHTSHIFT(D.A, D.size, i)
5:     D.size = D.size + 1
6:     D.A[i].key = k
7:     D.A[i].data = d
8:
9: function RIGHTSHIFT(<KEY,DATA> A[1, ..., m], INT n, INT i)
10:  for j = n, ..., i do
11:    A[j + 1] = A[j]                               ▷ Overflow if n = m

```

Il concetto relativo al funzionamento è molto semplice, si ricerca tramite ricerca dicotomica (o binaria) la posizione dove la nuova coppia deve essere inserita (a meno che alla chiave da inserire non corrisponda già un altro dato), successivamente si spostano verso destra tutte le chiavi maggiori della chiave da inserire, creando quindi un

bucco. Tale buco creato viene riempito con la nuova coppia da inserire.

Come è possibile immaginare, questo meccanismo genera dei casi ottimi, pessimi e medi. Il caso ottimo è il caso più semplice, ovvero la nuova coppia è la prima da inserire all'interno dell'array, oppure è la coppia che deve essere inserita in fondo a destra nell'array, di conseguenza, non viene generato nessuno spostamento e la coppia può essere inserita senza nessuno shift.

Il costo pessimo, invece è l'opposto del caso ottimo, ovvero la coppia da inserire è la prima; quindi, necessita una shift di tutte le coppie verso destra. Per il caso medio si fa la stessa considerazione del caso pessimo, quindi il costo è uguale.

Array ordinato	Search(<i>d</i>)	Insert(<i>d</i> , <i>k</i>)	Remove(<i>d</i> , <i>k</i>)
Funzioni Eseguite	Ricerca	Ricerca + shift	Ricerca + shift
Costo	$\Theta(\log n)$	Ottimo: $\Theta(\log n)$	Ottimo: $\Theta(\log n)$
		Pessimo: $\Theta(n)$	Pessimo: $\Theta(n)$
		Medio: $\Theta(n)$	Medio: $\Theta(n)$

Tabella riassuntiva

	SEARCH		INSERT		DELETE	
	Medio	Pessimo	Medio	Pessimo	Medio	Pessimo
Array non ordinati	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Array ordinati	$O(\log n)$	$\Theta(\log n)$	$\Theta(n)$	$\Theta(n)$	$O(n)$	$\Theta(n)$

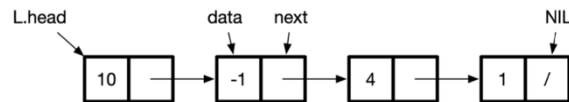
Struttura dati Lista

Un array occupa uno spazio di memoria contigua; quindi, se si vuole espandere l'array bisogna creare un nuovo array e copiare tutti gli elementi che erano precedentemente stati inseriti. Una lista invece ha il vantaggio di non usare memoria contigua, ma ha il potere di espandersi e accorciarsi dinamicamente, tramite una rete di puntatori che collega tutti i nodi della lista. La lista, quindi, permette di allocare più spazio di memoria, a discapito di un tempo di accesso agli elementi più lento; infatti, accedere alla posizione 4 su un array è immediato, accedere alla posizione 4 di una lista è più difficile; infatti, bisogna scorrere la lista dall'inizio fino ad arrivare alla quarta posizione.

Analisi costo

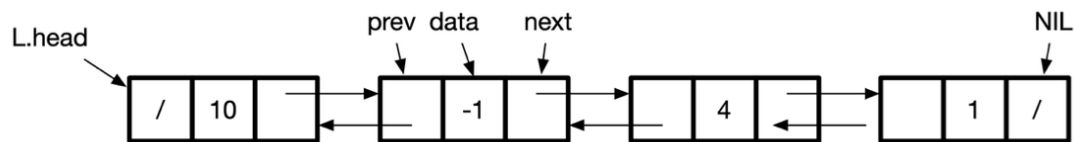
Ci sono diverse tipologie di liste che permettono di implementare i metodi *Search*, *Insert*, *Delete*, *Append*. Vediamone alcune e diamone il costo, in quanto non richiede nessun calcolo particolare, ma puramente una visualizzazione dell'immagine.

Lista Concatenata semplice



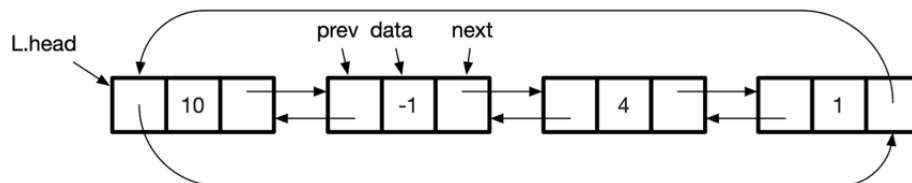
	Costo Ottimo	Costo Pessimo	Costo Medio
<i>Search</i>	$O(1)$	$\Theta(n)$	$\Theta(n)$
<i>Insert</i>	$O(1)$		
<i>Append</i>	$\Theta(n)$		
<i>Delete</i>	$O(1)$	$\Theta(n)$	$\Theta(n)$

Lista Doppia Concatenata



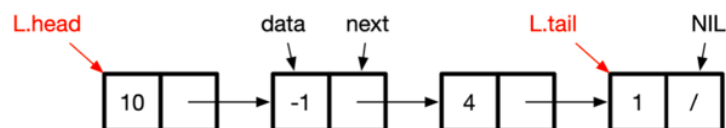
Stessi costi della lista concatenata semplice, rende solo più difficile la navigazione

Lista Concatenata Circolare



Stessi costi alle precedenti, con l'unica differenza è il costo dell'inserimento in coda: **$O(1)$**

Lista con Puntatori a Testa e Coda



Stessi costi della lista concatenata circolare

Tabella riassuntiva – Costo Pessimo

Type	SEARCH	INSERT	APPEND	DELETE
Concatenata semplice	$O(n)$	$O(1)$	$\Theta(n)$	$O(n)$
Doppiamente concatenata	$O(n)$	$O(1)$	$\Theta(n)$	$O(n)$
Circolare	$O(n)$	$O(1)$	$O(1)$	$O(n)$
Puntatori testa e coda	$O(n)$	$O(1)$	$O(1)$	$O(n)$

Nota! Sulle slides relative al capitolo di riferimento ci sono tutti gli pseudocodici relativi all'implementazione delle operazioni di search, insert, append e delete, per l'esame è importante sapere come sono strutturate le liste e come ogni lista viene usata per queste operazioni, in quanto alcuni esercizi comprendono le liste.

Dizionario con le Liste Concatenate

È possibile riadattare la struttura dati dizionario alle liste, in quanto il funzionamento è molto simile, tuttavia non si guadagna nessun miglioramento, in termini di prestazioni.

	SEARCH		INSERT		DELETE	
	Medio	Pessimo	Medio	Pessimo	Medio	Pessimo
Array non ordinati	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Array ordinati	$O(\log n)$	$\Theta(\log n)$	$\Theta(n)$	$\Theta(n)$	$O(n)$	$\Theta(n)$
Lista concatenata	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$

Struttura Dati Pila

La pila è una struttura dati che permette essenzialmente due operazioni: Push e Pop.

L'operazione di Push aggiunge un nuovo elemento alla prima posizione libera, mentre Pop rimuove dalla struttura l'elemento aggiunto più di recente; quindi, la pila funziona in modalità **LIFO**.

La Pila è una struttura dati molto versatile, infatti si può implementare sia tramite array che tramite liste. Per quanto riguarda gli array viene inserito il dato alla prima posizione libera, mentre viene rimosso l'ultimo dato inserito, ovvero quella in cima all'array. Stessa cosa per le liste, una pila può essere implementata tramite liste, dove si inserisce in testa alla lista e dove si rimuove sempre in testa alla lista.

In **entrambe** le implementazioni il costo è **costante**: $O(1)$.

La Pila è una grande struttura dati, il problema dell'implementazione dell'array è che la dimensione dell'array è fissa. Per risolvere questo problema si potrebbe creare un nuovo array con il doppio delle dimensioni di quello che è stato riempito, questa pratica permetterebbe di aumentare lo spazio disponibile, a discapito di un costo maggiore $\theta(n)$. Tuttavia, se si esegue un'analisi ammortizzata, si percepisce che in realtà tale costo non è reale, in quanto non si ricopiherebbe l'intero array ad ogni singolo inserimento. Questa implementazione, quindi, tramite un'analisi accurata verrebbe a costare come l'implementazione che adotta l'array statico.

Nota! Per vedere il calcolo del costo computazionale e lo pseudocodice guardare le slides relative al capitolo di riferimento.

Struttura Dati Coda

La coda può sempre realizzabile tramite array statici, o dinamici, oppure anche tramite liste. Il concetto è leggermente diverso rispetto alla pila. La coda permette l'inserimento del nuovo elemento in coda, e la rimozione dalla testa della coda. Le funzioni che permettono questo funzionamento si chiamano enqueue (inserimento in coda) e dequeue (rimozione dalla testa della coda).

In **entrambe** le implementazioni il costo è **costante**: $O(1)$.

Concettualmente

La pila mette da sopra e rimuove da sopra.

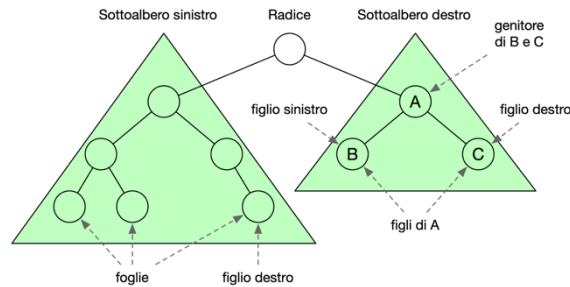
La coda mette da sotto e rimuove da sopra.

Esercizi

5^ Lezione

Alberi e Alberi Binari

Un albero è una struttura dati non lineare ad albero gerarchico. Per definizione, l'albero binario è composto da diversi nodi e da un insieme di archi (che connettono i nodi).
 Un albero viene detto radicato se uno dei suoi nodi è identificato come radice, inoltre, esiste un solo percorso per andare da un nodo all'altro. Infine, un albero viene detto ordinato se i figli di ogni nodo sono ordinati, quindi a livello implementativo si riesce a definire chi è il figlio sinistro e destro.



Profondità – Livello - Altezza

La profondità di un nodo u è la lunghezza del **percorso unico** che va dalla radice al nodo u . Per contare la profondità si contano i vari archi, ovvero il percorso che bisogna seguire dalla radice per arrivare al nodo in questione, dove ogni arco ha per estremità un nodo.

Un livello è l'insieme di nodi alla stessa profondità, mentre per altezza si intende la profondità massima dell'albero, ovvero il percorso radice-foglia più lungo.

Nota! Un albero binario vuoto ha altezza " - 1 "

Abero Binario Completo e Perfetto

Un albero binario è completo se ogni nodo intermedio ha due figli, mentre un albero binario perfetto è se tutte le foglie sono alla stessa profondità.

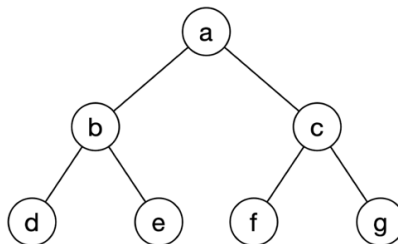
Teorema

Ogni albero non vuoto con n - nodi ha esattamente $n - 1$ archi

Algoritmi di visite costo $\Theta(n)$

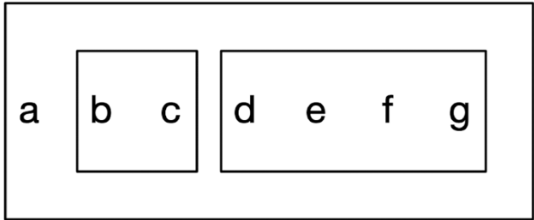
Ci sono tre tipi di visite in profondità (pre-ordine, post-ordine, in-ordine).

Dato questo albero binario, ci sono diversi modi per visualizzarlo



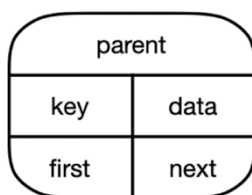
Visita in Profondità		
Pre-Ordine	Post-Ordine	In-Ordine
a b d e c f g	d e b f g c a	d b e a f c g
Viene prima stampata la radice, poi il sottoalbero di sinistra e sottoalbero di destra	Viene prima stampato il sottoalbero di sinistra, poi quello di destra e per ultima la radice	Viene stampato il sottoalbero di sinistra, la radice, e il sottoalbero di destra

È molto interessante invece la **visita in profondità** che riesce a stampare livello per livello il valore contenuto all'interno di ogni nodo. Se gli altri algoritmi erano per natura ricorsivi, questo è per natura iterativo e viene costruito tramite una Coda, concettualmente si inseriscono tutti i nodi in coda e poi vengono rimossi e stampati a schermo in ordine di livello.

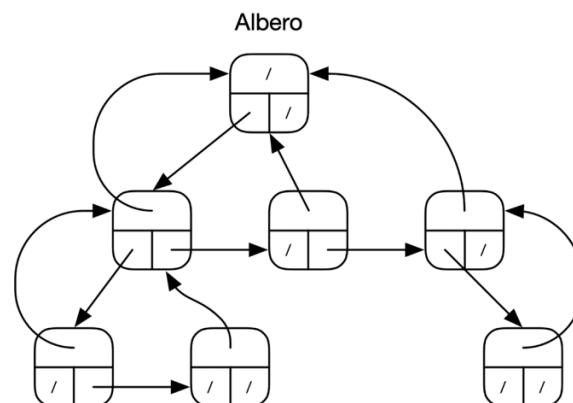
<pre> 1: function BFS(TREE T) 2: Q = QUEUE() 3: if T.root ≠ NIL then 4: ENQUEUE(Q, T.root) 5: while Q.size ≠ 0 do 6: x = DEQUEUE(Q) 7: VISIT(x) 8: if x.left ≠ NIL then 9: ENQUEUE(Q, x.left) 10: if x.right ≠ NIL then 11: ENQUEUE(Q, x.right) </pre>	
---	--

Esempio di Alberi non Binari

Ci possono essere anche degli alberi non binari, non binari perché un nodo può avere più di due figli. Il modo più semplice, ma soprattutto efficiente per rappresentare un albero non binario è avere dei nodi composti in questo modo:



il campo parent contiene un puntatore al padre del figlio, il campo first contiene un puntatore al figlio (primogenito) di quel nodo. Il campo next contiene eventualmente un puntatore al fratello più prossimo di quel nodo. In caso di più figli, la gestione dei fratelli viene implementata tramite una lista concatenata semplice. Questa gestione dei figli e dei fratelli richiede l'utilizzo di pochi puntatori, rendendola la rappresentazione di un albero non binario più utilizzata.



6^a Lezione

Alberi Binari di Ricerca

L'albero binario di ricerca è un albero binario con una struttura particolare, ovvero dato un nodo, in questo caso la radice, il sottoalbero di sinistra contiene tutti i numeri minori del valore della radice, mentre il sottoalbero di destra contiene tutti i numeri maggiori del valore contenuto all'interno della radice. Questo albero binario viene detto albero binario di ricerca ed è una rappresentazione ordinata di un albero binario.

Come già visto, le operazioni fondamentali di un albero binario di ricerca sono: inserimento di una nuova chiave, rimozione di una chiave presente e ricerca di una chiave.

Ricerca

La ricerca di una chiave all'interno di un albero binario di ricerca è piuttosto semplice, ma soprattutto richiede pochissimo tempo rispetto alla ricerca di un valore all'interno di un albero non binario. Questo perché si riesce ad indirizzare la ricerca in base al valore del nodo attualmente visitato e del valore da ricercare. Come è facile immaginare ci sono dei casi pessimi e ottimi.

Il caso ottimo con un costo pari a $O(1)$ si verifica quando il valore cercato è esattamente il valore associato alla chiave della radice dell'albero binario. Il caso pessimo, invece, si verifica quando il valore cercato è associato alla chiave presente nel nodo più distante dalla radice. A seguito di ciò l'algoritmo dovrà seguire il percorso radice-foglia più lungo, determinando un costo pari a $\Theta(h)$ dove " h " rappresenta l'altezza dell'albero binario di ricerca.

```
1: function SEARCH(BST T, KEY k) → NODE
2:   tmp = T.root
3:   while tmp ≠ NIL do
4:     if k == tmp.key then
5:       return tmp
6:     else if k < tmp.key then
7:       tmp = tmp.left
8:     else
9:       tmp = tmp.right
10:  return NIL
```

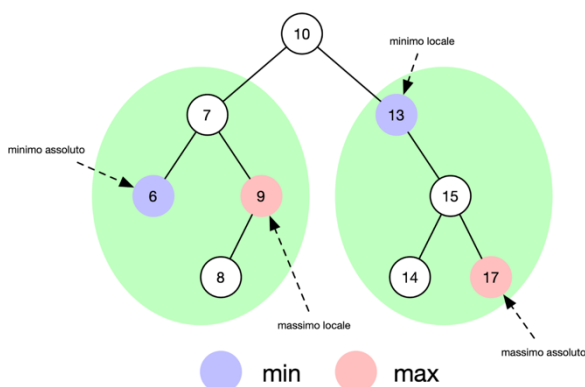
Inserimento

L'inserimento di un nodo all'interno dell'albero binario di ricerca è determinato dalla posizione in cui deve essere inserito questo nodo. Infatti, è di vitale importanza inserire correttamente (ovvero nella posizione adeguata) il nodo che si vuole inserire, questo per mantenere inalterato l'ordinamento dell'albero. Quindi, nel caso ottimo se l'albero è vuoto non bisognerà ricercare la posizione e il costo sarà $O(1)$, mentre nel caso pessimo se il nodo da inserire ricade nel percorso radice foglia più lungo il costo sarà di $\Theta(h)$, dove " h " rappresenta sempre l'altezza dell'albero binario. È facile quindi capire che il costo vero e proprio inserimento è pari a $O(1)$, mentre il costo globale dell'operazione è determinata principalmente dall'operazione di ricerca.

Delete

Prima di analizzare la rimozione, bisogna studiare alcune funzioni ausiliarie alla rimozione di un nodo, tali funzioni infatti permettono di mantenere inalterata la struttura dell'albero binario.

Massimo e Minimo



Il minimo valore all'interno di un albero binario di ricerca è il nodo più a sinistra del sottoalbero di sinistra, mentre la chiave con il valore maggiore (massimo) è il nodo che compare più a destra nel sottoalbero di destra. Anche qui, ci sono i soliti casi ottimi (radice + assenza di figli) e i casi pessimi (percorso più lungo radice – foglia)

Minimo	Massimo
Ottimo: $O(1)$	Ottimo: $O(1)$
Pessimo: $\Theta(h)$	Pessimo: $\Theta(h)$

Predecessore

Dato un nodo e dei sottoalberi, il predecessore è quella chiave che sostituita alla chiave del nodo non cambia la struttura dell'albero binario di ricerca, quindi, ne mantiene le sue proprietà.

Ci sono due casi

Caso 1: il predecessore di un nodo, che ha dei figli sinistri (sottoalbero di sinistra) è il massimo tra i minimi, ovvero è il nodo più a destra all'interno del sottoalbero di sinistra.

Caso 2: Il predecessore di un nodo, che non ha figli sinistri, ma ha solo figli destri (sottoalbero di destra) è il minimo tra i massimi, quindi il nodo che risiede più a sinistra nel sottoalbero di destra.

```

1: function PREDECESSOR(NODE T) → NODE
2:   if T == NIL then                                ▷ Empty tree
3:     return NIL
4:   else if T.left ≠ NIL then                          ▷ Case 1
5:     return MAX(T.left)
6:   else                                                ▷ Case 2
7:     P = T.parent
8:     while P ≠ NIL and T == P.left do
9:       T = P
10:      P = P.parent
11:    return P

```

- Caso pessimo: $\Theta(h)$
- Caso ottimo: $O(1)$
- Caso 1 (MAX): $\Theta(h)$
- Caso 1 (MAX): $O(1)$
- Caso 2: $\Theta(h)$
- Caso 2: $O(1)$

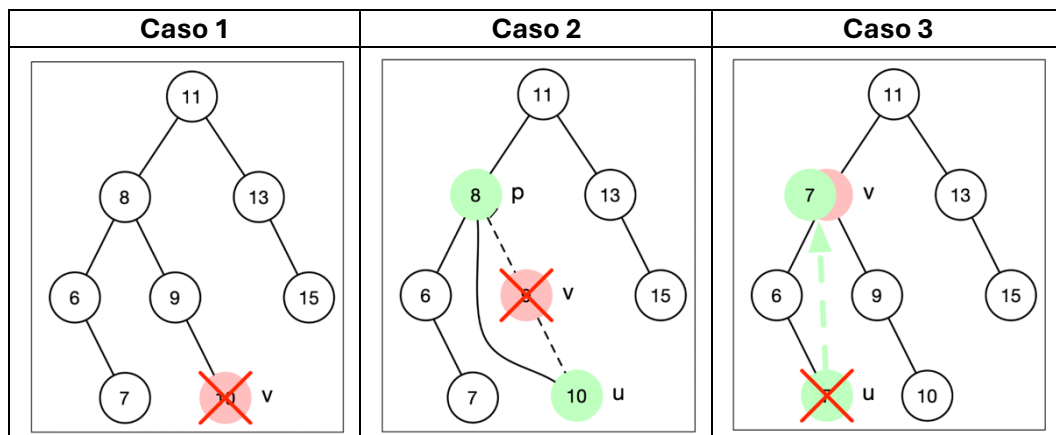
Rimozione

Anche per l'operazione di rimozione ci sono diversi casi.

Caso 1: il nodo da rimuovere è una foglia, questo non comporta la ricerca del predecessore, in quanto la rimozione di una foglia non altera la struttura dell'albero binario di ricerca.

Caso 2: Il nodo da rimuovere ha un solo figlio. In questo caso il figlio del nodo rimosso diventa la nova radice, o il figlio del padre del nodo rimosso.

Caso 3: Il nodo da rimuovere ha due figli, si ricerca quindi il predecessore.



Il codice dell'algoritmo è reperibile sulle slides relative al capitolo di riferimento, non è tanto lungo, nemmeno troppo complesso, in ogni modo all'esame non verrà chiesto di riscrivere l'algoritmo, ma di eseguire quello che fa l'algoritmo.

COSTO COMPUTAZIONALE	
Caso Pessimo: $\Theta(h)$	Caso Ottimo: $O(1)$
Search $\Theta(h)$	Search $O(1)$
Disconnect : $O(1)$	Disconnect $O(1)$
Predecessor $\Theta(h)$	Predecessor $O(1)$

Caso Medio

L'analisi del caso medio è piuttosto complicata; tuttavia, il costo medio viene calcolato sull'altezza media dell'albero binario di ricerca. In ogni modo, questa altezza può variare di tanto e può dipendere soprattutto dagli inserimenti casuali.

L'altezza massima abbiamo detto essere h , dove h rappresenta il numero di nodi del percorso radice-foglia più lungo. L'altezza minima di un albero binario di ricerca invece è logaritmica, e questa condizione la si trova quando si è di fronte ad un albero binario perfetto.

L'altezza media, invece è parecchio complicata, infatti dipende da inserimenti e rimozioni; quindi, l'altezza medio dell'albero binario di ricerca a seguito di inserimenti e rimozioni non è facile da estrapolare (nessuno ci è mai riuscito), tuttavia si può osservare che il generale le operazioni sono piuttosto veloci. Il caso "semplice" riguardante l'altezza media si verifica con **solo** inserimenti di numeri casuali, e tale caso riesce a dare un valore logaritmico all'altezza media.

Dizionario con Alberi Binari di Ricerca

È possibile implementare il prototipo della struttura dati dizionario con un albero binario di ricerca, il che non è molto complesso, in quanto le funzioni precedentemente studiate sono facilmente riadattabili (pur facendo mantenere il costo uguale a quello analizzato) per permettere l'inserimento di nodi che non sono già presenti.

Riassunto dei costi – Dizionario

Questa tabella sottostante è un riassunto dei costi delle varie implementazioni della struttura dati dizionario. Da notare che l'efficienza delle computazioni effettuate con l'albero binario di ricerca dipende completamente dall'altezza dell'albero, ciò significa che se si riuscisse a mantenere un'altezza logaritmica il costo delle computazioni sarebbe sempre uguale a $\Theta(n)$.

	SEARCH		INSERT		DELETE	
	Medio	Pessimo	Medio	Pessimo	Medio	Pessimo
Array non ordinati	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Array ordinati	$O(\log n)$	$\Theta(\log n)$	$\Theta(n)$	$\Theta(n)$	$O(n)$	$\Theta(n)$
Lista concatenata	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Albero Binario di Ricerca	$\Theta(h)$	$\Theta(h)$	$\Theta(h)$	$\Theta(h)$	$\Theta(h)$	$\Theta(h)$

7^a Lezione

Alberi AVL

Sappiamo che un albero binario con " n " nodi ha altezza pari a $\Omega(\log n)$ oppure $O(n)$. Quello che si vuole perseguire è quindi mantenere un albero binario di ricerca sempre bilanciato, in questo modo si è sicuri di riuscire ad effettuare inserimenti, rimozioni e ricerche all'interno dell'albero binario pagando sempre (al peggio) un costo logaritmico per ciascuna operazione.

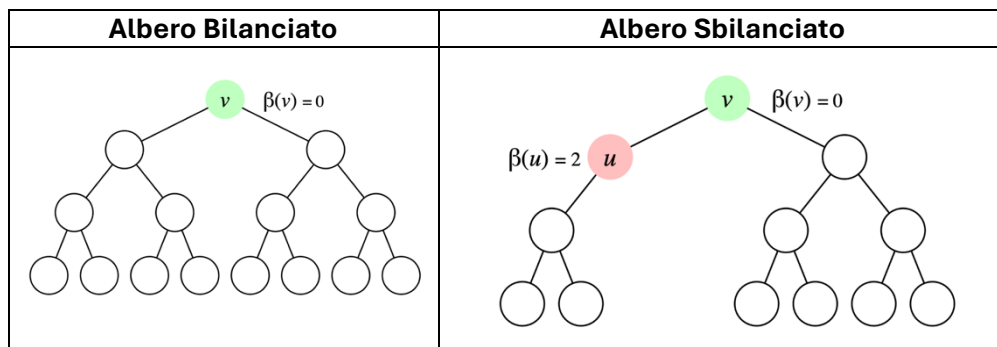
Gli alberi AVL, quindi, supportano le classiche operazioni viste precedentemente, ma aggiungono un fattore di auto bilanciamento, ciò significa che l'albero viene ribilanciato (dopo ogni modifica delle chiavi) affinché il costo al peggio rimanga sempre pari a $O(\log n)$.

Fattore di Bilanciamento

Il fattore di bilanciamento $\beta(v)$ di un nodo è dato dalla **differenza** tra l'altezza del sottoalbero di sinistra e il sottoalbero destro del nodo " v ".

$$\beta(v) = \text{altezza}(v.\text{left}) - \text{altezza}(v.\text{right})$$

Un albero binario AVL si dice **bilanciato in altezza** se per ogni nodo " v " le altezze dei suoi sottoalberi differiscono al più di 1. Per definizione, quindi, un albero AVL è un albero binario bilanciato in altezza.



Altezza massima

L'albero di Fibonacci è l'albero binario AVL con fattore di sbilanciamento massimo possibile (che pende verso sinistra), ciò significa che l'albero di Fibonacci con " n " nodi è l'albero binario con altezza massima tra tutti gli alberi bilanciati con " n " nodi. Da ciò possiamo concludere che l'altezza " h " di un albero AVL con " n " nodi è $\Theta(\log n)$.

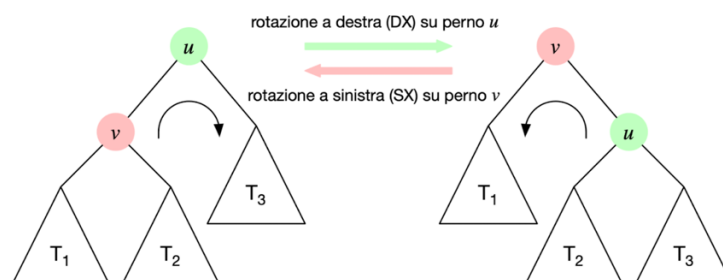
Mantenere il bilanciamento

La ricerca su un albero AVL viene effettuata come su un BST (albero binario di ricerca), l'unica differenza ricade su inserimenti e rimozioni, i quali richiedono un ribilanciamento dell'albero per mantenere inalterate le proprietà. Questo controllo e questo ribilanciamento richiedono funzioni ausiliarie, in quanto prima di procedere con delle rotazioni è necessario aggiornare l'altezza e il bilanciamento stesso dell'albero. Le due funzioni che vengono usate sono le seguenti e costano $O(1)$.

<pre> 1: function UPDATEHEIGHT(NODE u) 2: if u ≠ NIL then 3: lefth = -1, righth = -1 4: if u.left ≠ NIL then 5: lefth = u.left.height 6: if u.right ≠ NIL then 7: righth = u.right.height 8: u.height = MAX(lefth, righth) + 1 </pre>	<pre> 1: function BALANCEFACTOR(NODE u) → INT 2: lefth = -1, righth = -1 3: if u ≠ NIL and u.left ≠ NIL then 4: lefth = p.left.height 5: if u ≠ NIL and u.right ≠ NIL then 6: righth = p.right.height 7: return lefth - righth </pre>
---	---

Rotazioni

Le rotazioni sono le operazioni fondamentali per effettuare il ribilanciamento dell'albero AVL. Una rotazione può essere effettuata sia a destra che a sinistra, vediamo un esempio



Come si vede, quando si effettua una rotazione la si esegue su un perno, concettualmente, se si vuole fare una rotazione a destra (caso di sinistra), il perno (nodo " u ") diventa figlio del nodo " v ", e il sottoalbero di destra del nodo " v ", se presente, diventa figlio sinistro del perno " u ".

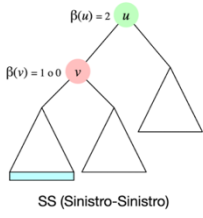
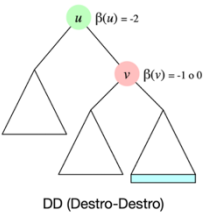
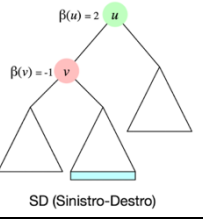
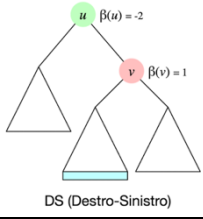
Analogamente, succede la stessa cosa per una rotazione a sinistra.

Sulle slides relative al capitolo di riferimento è presente lo pseudocodice di **rightrotate**, all'esame non verrà mai chiesto, ma verrà chiesto di sapere come lo pseudocodice effettua la rotazione.

In ogni modo, la cosa da sapere è che il costo per ogni tipo di rotazione è costante: $O(1)$.

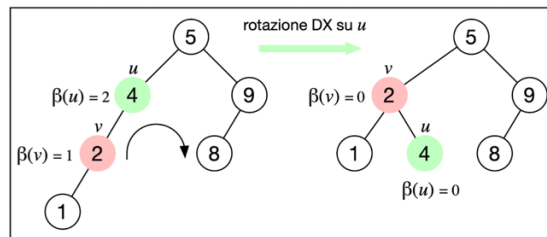
Sbilanciamenti

Ci sono diversi tipi di sbilanciamenti, che vengono scaturiti dopo diverse operazioni di inserimento o rimozione. Si possono verificare quattro casi (simmetrici a due a due) di sbilanciamenti, nella tabella vengono rappresentati e spiegati.

Sbilanciamento SS		Sbilanciamento DD	
$\beta(u) = 2$ e $\beta(u.left) \geq 0$		$\beta(u) = -2$ e $\beta(u.right) \leq 0$	
Sbilanciamento SD		Sbilanciamento DS	
$\beta(u) = 2$ e $\beta(u.left) = -1$		$\beta(u) = -2$ e $\beta(u.right) = 1$	

Sbilanciamento SS // DD (speculare)

Per riportare il bilanciamento su uno sbilanciamento sinistro-sinistro è necessario effettuare una semplice rotazione a destra dell'albero su perno "u".



Sbilanciamento SD // DS (speculare)

Per riportare il bilanciamento dobbiamo effettuare due rotazioni, prima verso sinistra e poi verso destra. Quindi prima si effettua una rotazione sinistra con perno il sottoalbero di sinistra rispetto al nodo "u", in questo modo si ottiene uno sbilanciamento verso sinistra, che è semplicemente risolvibile attraverso una rotazione a destra.

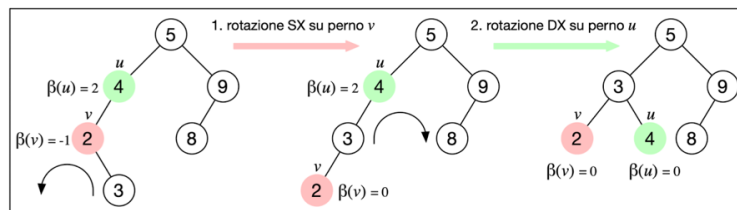


Tabella riassuntiva

Sbilanciamento SS		Sbilanciamento DD	
$\beta(u) = 2$ $\beta(u.left) \geq 0$	Rotazione	$\beta(u) = -2$ $\beta(u.right) \leq 0$	Rotazione
	DX su "u"		SX su "u"
Sbilanciamento SD		Sbilanciamento DS	
$\beta(u) = 2$ $\beta(u.left) = -1$	Rotazione	$\beta(u) = -2$ $\beta(u.right) = 1$	Rotazione
	SX su "u.left", e a DX su "u"		DX su "u.right", e a SX su "u"

Inserimento

Sappiamo che l'operazione di inserimento (nel caso pessimo) dipende dall'altezza dell'albero sul quale bisogna inserire la nuova chiave. Ora noi sappiamo che gli alberi AVL sono sempre degli alberi binari di ricerca, ma questa volta l'altezza è sempre logaritmica, di conseguenza, l'inserimento avrà sempre costo $\Theta(\log n)$.

```
1: function INSERT(AVLTree T, KEY k, DATA d) → NODE
2:   v = BSTINSERT(T, k, d)
3:   p = v.parent
4:   while p ≠ NIL and |BALANCEFACTOR(p)| ≠ 2 do
5:     UPDATEHEIGHT(p)
6:     p = p.parent
7:   if p ≠ NIL then
8:     BALANCE(T, p)
9:   return v
```

Il nuovo nodo viene inserito come per gli alberi binari di ricerca, dove tutti i percorsi di inserimento sono lunghi $\Theta(\log n)$, di conseguenza anche quello sarà il costo. Una volta completato l'inserimento vengono aggiornate tutte le altezze dei sottoalberi, (nel caso peggiore l'aggiornamento verrà eseguito un numero logaritmico di volte). Inoltre, se aggiornata l'altezza si verifica un fattore di sbilanciamento, l'albero viene ribilanciato, ma a tale albero non viene riaggiornata l'altezza, verrà fatto nuovamente nella prossima iterazione del ciclo.

Delete

Come prima, la procedura effettiva che rimuove il nodo è la stessa che avevamo già precedentemente analizzato sugli alberi binari di ricerca, tuttavia, adesso dobbiamo ribilanciare l'albero dopo una rimozione di un nodo/foglia. Questo significa che la procedura *balance* verrà richiamata molte volte; infatti, dopo una rimozione è necessario aggiornare bilanciamento e altezza dell'albero per tutti i nodi, altrimenti si rischia di non riportare l'albero al corretto bilanciamento, creando problemi a cascata.

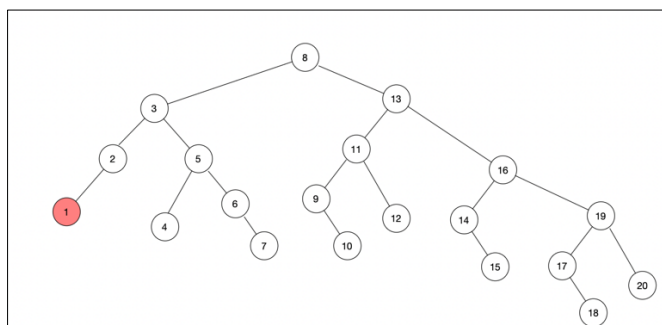
```
1: function DELETE(AVLTree T, KEY k, DATA d)
2:   v = BSTDELETE(T, k)
3:   if v ≠ NIL then
4:     p = v.parent
5:     while p ≠ NIL do
6:       if |BALANCEFACTOR(p)| == 2 then
7:         BALANCE(T, p)
8:       else
9:         UPDATEHEIGHT(p)
10:      p = p.parent
11:   return v
```

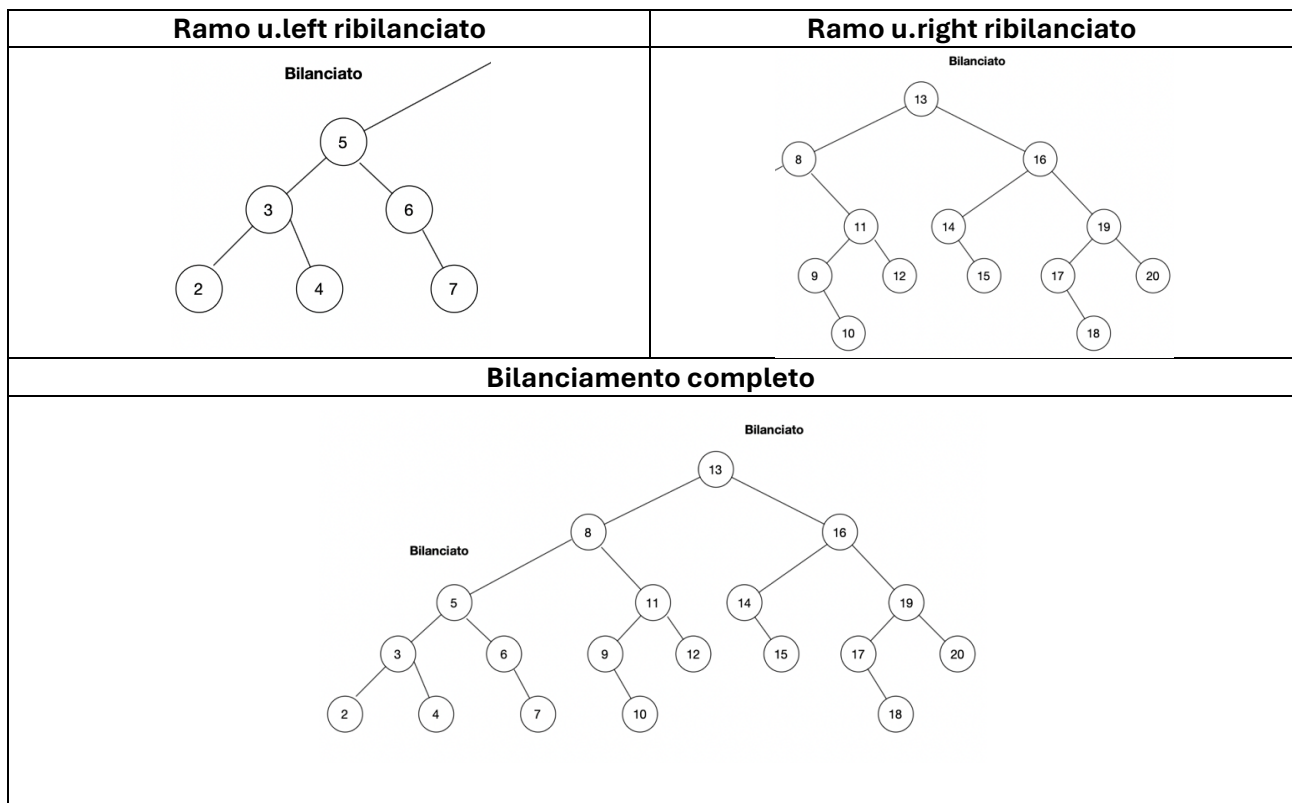
Arrivo sempre fino alla radice, questo perché spesso e volentieri la rimozione di un nodo sbilancia tutto l'albero, portando sbilanciamenti a cascata

Il nodo si rimuove come in un albero binario di ricerca, ma come detto, la rimozione richiede una ricerca, quindi, il costo (medio e pessimo) è pari a $\Theta(\log n)$. Successivamente, come detto è necessario riaggiornare tutte le altezze dell'albero, e nel caso peggiore bisogna farlo per tutto il cammino più lungo radice – foglia, anche qui il costo è sempre logaritmico ($O(\log n)/\Theta(\log n)$). Infine, bisogna effettuare delle rotazioni quando si incontrano nodi critici che devono essere ribilanciati con un costo costante, pari a $O(1)$. Dunque, il costo finale è pari a $\Theta(\log n)$.

Esempio – Bilanciamento a cascata

Sia 8 = "u"





Esercizi

8^a Lezione Tabelle Hash

Abbiamo visto e implementato la struttura dati dizionario in diversi modi, tuttavia, le operazioni di ricerca, inserimento e rimozione non rimangono particolarmente veloci, ad eccezione degli alberi AVL, che in ogni modo sono poco utilizzati nella realtà, questo perché creare alberi per salvare informazioni forse non è la cosa più immediata e rapida da fare.

Le tabelle Hash, largamente utilizzate permettono le tre operazioni basilari della struttura dati dizionario in un tempo di esecuzione lineare, quindi estremamente efficiente.

L'idea della struttura dati dizionario è quella di generalizzare l'indicizzazione di un array ordinario e di ottenere così un costo medio pari a $O(1)$. Si parla di costo medio perché sotto ragionevoli assunzioni probabilistiche è possibile concludere un costo medio lineare, in realtà nel caso pessimo si va ben oltre il costo lineare raccontato fino ad ora.

Tabelle Hash

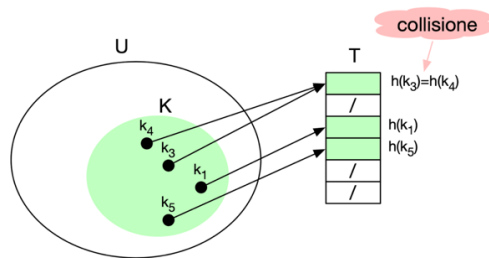
Sia " U " l'insieme di tutte le chiavi possibili, ad esempio l'insieme di tutti i numeri naturali e sia " K " l'insieme delle chiavi attualmente utilizzate abbiamo due opzioni.

La prima è creare un array di dimensione $\Theta(|U|)$ per creare un indirizzamento diretto tra la posizione dell'array " U " con la chiave " K " da inserire.

Tuttavia, se l'insieme universo è molto più grande dell'insieme delle chiavi utilizzate, allora bisogna utilizzare delle tabelle Hash.

Per utilizzare delle tabelle Hash è necessario creare un array iniziale di dimensione $m = \Theta(|K|)$, successivamente è necessario costruire una funzione Hash $h: U \rightarrow [0, \dots, m - 1]$.

Detto ciò, possiamo dire che $h(k)$ è il **valore Hash** della chiave " k " e che la funzione h trasforma una chiave " k " in un indice dell'array " T " (notazione per indicare la tabella Hash). Di conseguenza, è facile comprendere che la chiave " k " verrà mappata nello slot $T[h(k)]$, ma che ci saranno delle collisioni che dovranno essere risolte se due valori " k " mapperanno lo stesso indice.



Ricapitolando

Per implementare una tabella Hash efficiente abbiamo bisogno di:

- Una **funzione Hash**
 - Deve essere calcolata velocemente
 - Deve garantire una distribuzione omogenea delle chiavi su “T”
 - Minimizzare il rischio di collisioni
- Gestione delle **collisioni**
 - Sono inevitabili, ma devono essere gestite nel miglior modo possibile
- Un array $T[0, \dots, m - 1]$ di dimensione $m = \Theta(|k|)$

Funzioni Hash Uniformi

Una buona funzione Hash soddisfa la proprietà di **uniformità semplice**, ovvero le chiavi vengono distribuite in modo uniforme, in questo modo ogni indice dell’array ha probabilità $1/m$, in questo modo le collisioni vengono parzialmente evitate. Concettualmente, se ho una funzione che mappa sempre nello slot zero avrò infinite collisioni, se invece riesco a mappare tramite una funzione in modo omogeneo ogni posizione dell’array, allora dovrò solo gestire i casi particolari di collisione. Inoltre, una funzione Hash non può richiedere troppo tempo per essere calcolata, e deve sempre mappare valori interi positivi, quindi i caratteri, devono essere trasformati (tramite cast) in numeri interi.

Funzione Hash – Metodo della Divisione

Il metodo della divisione è molto semplice, infatti richiede una semplice divisione con il resto; infatti, l’indice dell’array dove dovrà essere memorizzata la chiave è pari al resto della divisione per un numero (modulo). Questa operazione è semplicissima, anche direttamente integrata da moltissimi processori, ma genera un numero eccessivo di conflitti. L’unico modo per attuare questa soluzione e limitare il problema dei conflitti è quello di scegliere un numero (su cui verrà effettuata la divisione) primo distante da potenze di 2 e di 10.

Funzione Hash – Metodo della Moltiplicazione

Analogamente a quanto detto prima, invece di dividere si moltiplica per una costante. La costante migliore individuata (migliore perché limita le collisioni) è pari al reciproco della sezione aurea; infatti, questa costante è abbastanza particolare e genera numeri pseudo-randomici.

Funzione Hash – Codifica Algebrica

Metodo simile all’algebra lineare, molto efficiente, ma costosissimo da calcolare, in quanto richiede numerose somme, addizioni e prodotti, portando il costo complessivo ad essere pari a $\Theta(\log^2 k)$

Funzione Hash – Regola di Horner

Valutazione del valore di una chiave in un punto tramite un polinomio di grado “n”. È sicuramente più efficiente del metodo algebrico in quanto il costo di calcolo si attesta su $\Theta(\log k)$. Se poi si assume che il numero di cifre di una chiave è tipicamente un numero relativamente piccolo (nessuno avrà mai una chiave di dimensioni milionesime/miliardesime) il costo effettivo è molto basso.

Questo tra tutti è indubbiamente il metodo più raffinato e utilizzato.

Per vedere come si calcolano le varie funzioni Hash guardare le slides relative al capitolo di riferimento

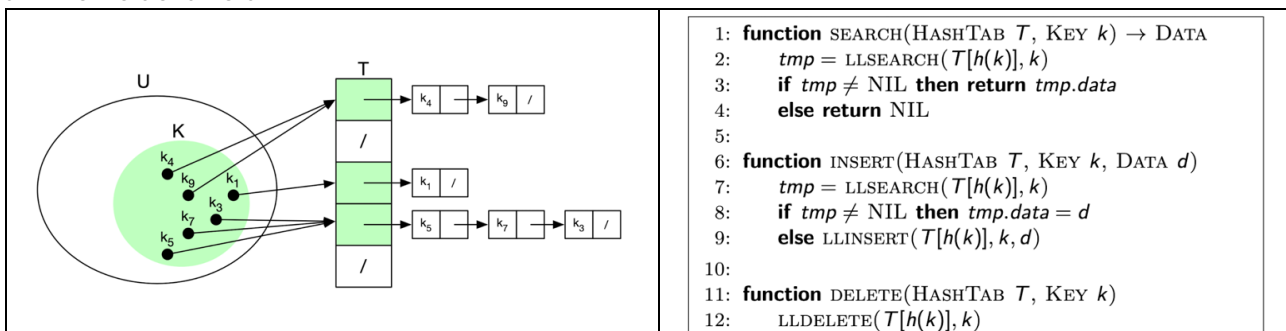
Collisioni

L'Hashing uniforme semplice non elimina le collisioni, ma le riduce, in generale, le collisioni non sono evitabili, ma solo riducibili. È quindi necessario trovare un sistema per gestire le collisioni, ovvero trovare una collocazione alternativa per le chiavi, e se una chiave non si trova nella sua posizione attesa, bisogna andare a cercarla nella posizione alternativa. In ogni modo, questa ricerca nella posizione alternativa risulta piuttosto costosa nel caso pessimo.

Analizzeremo due possibili tecniche di reindirizzamento, la prima (concatenamento) e la seconda (indirizzamento aperto).

Concatenamento

Le chiavi con lo stesso valore definito dalla funzione Hash sono memorizzate in una lista concatenata, detta lista di trabocco, per cercare il valore di "k" atteso in quella posizione bisognerà cercare all'interno della lista.



In generale, questa lista di trabocco aumenta il costo computazionale facendolo diventare lineare. Inoltre, la dimensione della tabella Hash deve essere ponderata, infatti, pur non essendoci un limite fisico, è necessario creare un array abbastanza grande (e non troppo per evitare di sprecare spazio), ma non troppo piccolo (per evitare di avere liste di trabocco lunghissime e lunghe da "scansionare"). Come detto prima:

Caso Pessimo	Caso Ottimo
$\Theta(L)$	$O(1)$

Dove "L" è la lunghezza della lista di trabocco più lunga tra tutte le liste di trabocco possibili.

Tuttavia, la storia cambia nell'analisi del costo medio. Infatti, il costo nel caso medio dipende dal **numero medio di accessi** per cercare (con successo o insuccesso) una chiave.

Ragionando, è facile capire che la ricerca inserimento e rimozione hanno uguale costo, e questo dipende dal fatto che tutte queste tre operazioni richiedono sempre una ricerca di una chiave, ma il vero e proprio costo della ricerca non è $\Theta(L)$, questo perché la funzione Hash distribuisce queste chiavi. Di conseguenza, il numero medio di accessi dipende da **come** vengono **distribuite** le chiavi dalla funzione Hash.

Chiamiamo quindi **fattore di carico** $\alpha = n/m$ il rapporto tra il **numero di elementi inseriti** (n) e la **dimensione della tabella Hash** (m). Di conseguenza, sotto l'assunzione di Hashing uniforme semplice ogni slot della tabella possiede mediamente α chiavi.

Da questa assunzione è possibile estrapolare due teoremi.

Teorema

Sotto l'assunzione di hashing uniforme semplice, una **ricerca senza successo** in una tabella hash con concatenamento ha costo medio $\Theta(1 + \alpha)$

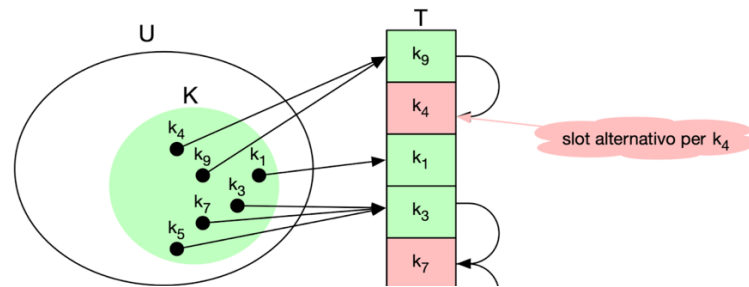
Teorema

Sotto l'assunzione di hashing uniforme semplice, una **ricerca con successo** in una tabella hash con concatenamento ha costo medio $\Theta(1 + \alpha)$

Con questi due teoremi abbiamo quindi dimostrato che il fattore di carico influenza il costo delle operazioni e se $n = O(m)$ allora $\alpha = O(1) \rightarrow$ costo medio ricerca $O(1)$.

Indirizzamento Aperto

Tutte le chiavi sono memorizzate nella stessa tabella, ogni slot contiene una chiave oppure un "NIL", se uno slot è occupato e c'è un conflitto, allora si cerca uno slot alternativo nella stessa tabella.



Ora la nostra funzione Hash non solo deve mappare l'indice della tabella Hash, ma deve anche avere un passo di ispezione, ovvero deve continuare a mappare un indice della tabella Hash (con un possibile offset) nel caso lo slot mappato precedentemente sia già stato riempito. In generale, se passo dopo passo la funzione non riesce più a mappare una posizione della tabella, allora si è in overflow.

<pre>1: function SEARCH(HASHTAB T, KEY k) → DATA 2: i = 0 3: repeat 4: j = h(k, i) ▷ hash value at step i 5: if A[j].key == k then 6: return A[j].data 7: i = i + 1 8: until A[j] == NIL or i == A.size 9: return NIL</pre>	<pre>1: function INSERT(HASHTAB T, KEY k, DATA d) 2: i = 0 3: repeat 4: j = h(k, i) ▷ hash value at step i 5: if A[j] == NIL or A[j] == DELETED then 6: A[j].key = k 7: A[j].data = d 8: return 9: i = i + 1 10: until i == A.size 11: error "overflow"</pre>
<pre>1: function DELETE(HASHTAB T, KEY k) 2: i = 0 3: repeat 4: j = h(k, i) ▷ hash value at step i 5: if A[j].key == k then 6: A[j] = DELETED 7: return 8: i = i + 1 9: until A[j] == NIL or i == A.size</pre>	

Ancora una volta il costo pessimo non è più costante, ma dipende dalla dimensione della tabella Hash. Per capire quanto costano le operazioni nel caso medio bisogna vedere quali tra le possibili strategie di ispezione è stata scelta.

Lineare

Nel caso dell'ispezione **lineare** si ha una funzione Hash che possiamo assumere sia implementata tramite il metodo della divisione. Con l'ispezione lineare, qualora ci fosse un conflitto si andrebbe semplicemente a verificare la cella successiva (applicando quindi un offset pari a uno) ogni volta che la cella della tabella Hash fosse già piena.

Esempio

Se $7 \bmod 10 = 7$, ma la cella risulta piena, si verifica la cella $(7+1 \bmod 10)$ e così via.

Questo metodo genera un problema noto come **clustering primario**, ovvero se esistono lunghe sottosequenze occupate che a mano a mano diventano sempre più lunghe, l'ispezione lineare dovrà computare sempre più volte la funzione Hash prima di trovare uno slot libero o prima di andare in overflow.

Quadratica

Nel caso dell'**ispezione quadratica**, invece di avanzare linearmente si avvanzerà con un passo di ispezione più ampio, che cresce di volta in volta, solitamente questo sbalzo dipende da due parametri. Tuttavia, possono sorgere due problemi, il primo è che i parametri non riescono ad eseguire una distribuzione omogenea, inoltre, se due chiavi distinte, con pari modulo vengono inserite, e sia la

prima che la seconda chiave hanno dovuto fare più di un passo di ispezione, la seconda chiave che dovrà essere inserita ripeterà gli stessi identici passi di ispezione della prima, portando quindi a perdere tempo di esecuzione. Questo fenomeno viene chiamato **clustering secondario**.

Doppio Hashing

Vengono usate due funzioni differenti, la prima determina il passo iniziale, mentre la seconda funzione Hash viene usata per determinare tutti i passi successivi che si verificano in caso di collisione. In generale, mappare con due funzioni diverse la stessa cella è parecchio improbabile, tuttavia devo essere sicuro che le due funzioni costruite mappino in modo omogeneo tutte le posizioni dell'array. Qualora queste due funzioni fossero così buone e prive di difetti di **eviterebbe** sia il **clustering primario** che **secondario**.

Lineare	Quadratica	Doppio Hashing
$h(k, i) = (h'(k) + i) \bmod m$	$h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod m$	$h(k, i) = (h_1(k) + i h_2(k)) \bmod m$

L'analisi del caso medio risulta parecchio complicata, questo perché i diversi metodi di ispezione incidono di gran lunga sul costo finale. Dunque, detto ciò, si distinguono due diversi costi, uno riguardante solo ed esclusivamente la ricerca lineare, e l'altro riguardante la ricerca quadratica e con doppio Hashing.

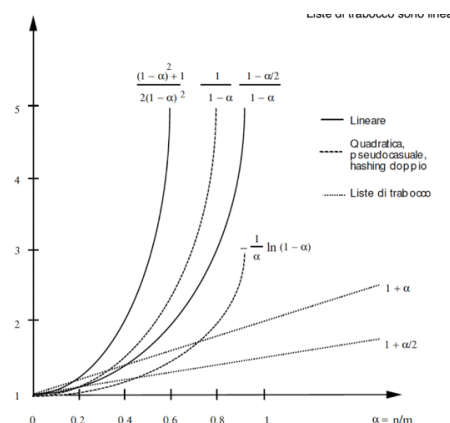
Concettualmente, per quanto riguarda la ricerca quadratica e con doppio Hashing, se la tabella è piena al 50%, la ricerca senza successo richiede al massimo due ispezioni, mentre la ricerca senza successo meno di due. Per quanto riguarda una tabella piena al 90% la ricerca senza successo richiede in media dieci ispezioni, mentre la ricerca con successo meno di tre.

Visto che l'analisi di questi costi è molto complessa, per approfondire questa parte, e trovare anche la spiegazione per il risultato finale guardare le slides 40-42 delle slides relative al capitolo di riferimento. Qui vengono riportati solo i costi e delle considerazioni finali

Tabelle Riassuntiva

In questa tabella riassuntiva si vedono i costi **nel caso medio** per i due diversi approcci che puntano a risolvere i conflitti, la strategia generale per mantenere un costo generalmente sempre basso è quello di avere un fattore di carico altrettanto basso ($< 0,75$), se questo è raggiunto, allora le operazioni saranno generalmente più veloci.

Tipologia Ricerca	Liste di Trabocco	Indirizzamento Aperto		
		Lineare	Quadratica	Doppio Hashing
Con successo	$\Theta(1 + \alpha)$	$\frac{(1 - \alpha/2)}{2(1 - \alpha)}$	$\frac{1}{\alpha} \log\left(\frac{1}{1 - \alpha}\right)$	
Senza successo	$\Theta(1 + \alpha/2)$	$\frac{(1 - \alpha)^2 + 1}{2(1 - \alpha)^2}$	$\frac{1}{(1 - \alpha)}$	

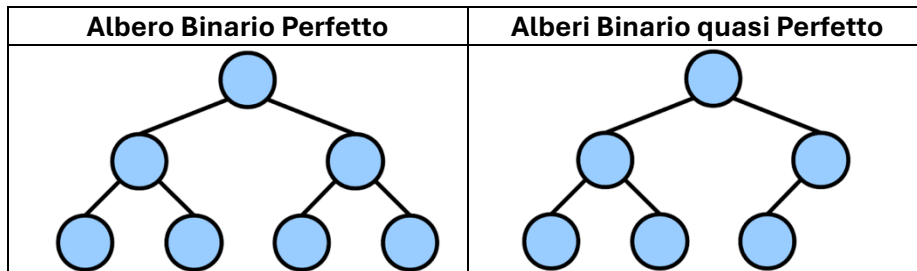


9^a Lezione

Heap Binari

Alberi Binari

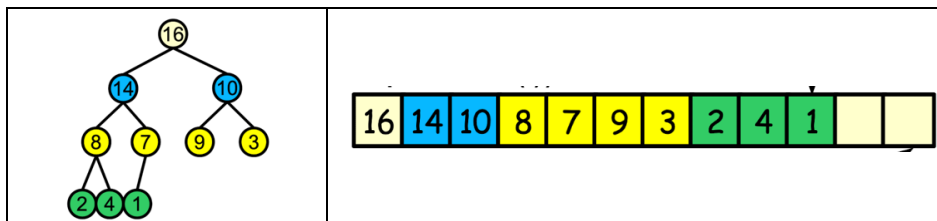
Gli alberi binari perfetti hanno una struttura simmetrica, ovvero ogni nodo ha esattamente due figli, di conseguenza, la sua altezza è pari al $\log_2 N$ dove per “ N ” si intende il numero di nodi dell’albero binario. Gli alberi binari quasi perfetti invece sono alberi binari dove tutti i livelli vengono saturati (ovvero ogni nodo interno ha esattamente due figli) tranne l’ultimo. Tale ultimo livello, seppure non saturato ha una specifica ben definita, infatti le foglie sono tutte compattate verso sinistra. Graficamente:



Alberi Binari Heap

Gli alberi Binari Heap sono alberi binari che contengono vincoli sui valori inseriti in ogni nodo, dove i figli di ogni nodo hanno il padre che contiene un valore maggiore (albero Max-Heap) o minore (albero Min-Heap) rispetto al valore contenuto da essi.

A questi alberi binari Heap è possibile associarci un array contiene la visita in profondità dell’albero heap di riferimento.



È interessante notare che un array Heap ha delle proprietà e queste proprietà permettono di risalire al padre o al figlio a parità di indice. Ad esempio, si può capire chi è il primo figlio della chiave 10 ($:=$ indice 3). Sapendo che ad $A[1]$ è presente la radice, e ad indice “ i ” è presente una chiave, il figlio sinistro di tale chiave sarà alla posizione $2*i$, mentre il figlio destro sarà alla posizione $(2*i) + 1$. Viceversa, il genitore di una chiave ad indice “ i ”, avrà il suo genitore posizionato alla cella corrispondente all’indice $\text{int}(i/2)$.

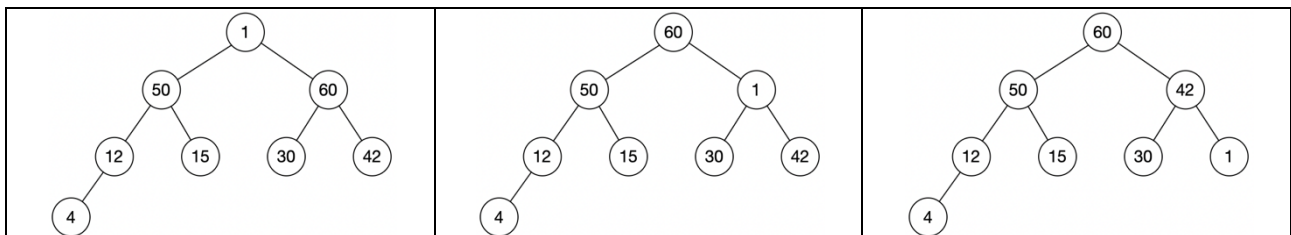
Operazioni Possibili

Dato questo array, definito in un modo particolare, e con tali proprietà è possibile individuare alcune operazioni plausibili. La prima operazione possibile è **findMax**, ovvero una funzione che individua il massimo valore contenuto in un array Heap. La seconda operazione è **fixHeap**, ovvero ripristina le proprietà dell’array heap nel caso nella radice non fosse più il valore massimo. La terza operazione è **deleteMax**, dove viene rimosso l’elemento massimo dall’array Heap. La quarta operazione è invece **heapify**, dove dato un array si converte tale array in un array heap.

FindMax Costo pari a $O(1)$

FixHeap

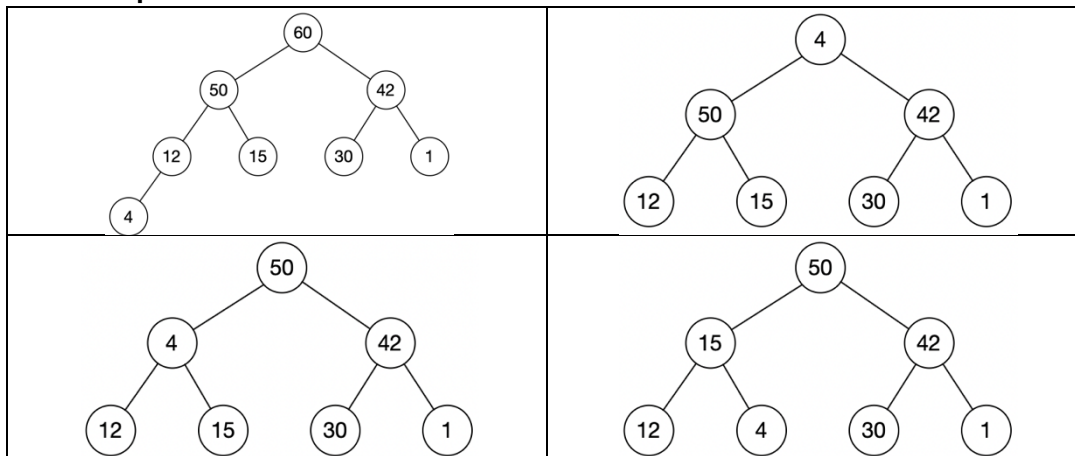
È una funzione ricorsiva, dove dato un albero con una radice, e dove tale radice non contiene il valore massimo, la funzione effettua variati scambi tra valori massimi fino a portare il numero che invalidava la struttura Heap in una posizione dove la struttura viene verificata.



La funzione, come si può vedere, è piuttosto semplice, inoltre, grazie a questo esempio patito è possibile capire che sono necessarie 2 iterazioni per portare a termine la funzione. Queste due iterazioni sono pari a $\log_2(\max(h)) = \log_2(h)$ dove per “ h ” si intende l’altezza massima dell’albero binario. L’albero binario ha altezza massima/profondità pari a 4, il logaritmo in base 2 di 4 infatti è proprio pari a 2, ovvero il numero di operazioni richieste per la completa esecuzione del programma. Costo finale: $O(\log_2 h)$

DeleteMax

L’operazione di rimozione del massimo richiede la rimozione di una foglia. Invece di stare a cancellare il massimo e successivamente eseguire rotazioni (a destra o a sinistra) per riportare il corretto bilanciamento si può fare qualcosa di più semplice. Infatti, l’idea migliore è quella di cancellare il valore in cima all’albero, copiare il valore dell’ultima foglia nella radice, rimuovere la foglia e riapplicare **FixHeap**.



Come si può capire, ancora una volta la funzione dipende interamente dalla funzione **FixHeap**; infatti, la rimozione ha un costo costante.

Ciò significa che il costo computazionale di **DeleteMax** sarà uguale a quello di **FixHeap**: $O(\log_2 h)$.

Heapify

```
function heapify(int S[], int n, int i) {
    if (i > n) return;
    heapify(S, n, 2 * i); // crea heap radicato in S[2*i]
    heapify(S, n, 2 * i + 1); // crea heap radicato in S[2*i+1]
    fixHeap(S, n, i);
}
// per trasformare un array S in uno heap:
// heapify(S, S.length, 1);
```

Come si può vedere dallo pseudocodice si ha una funzione ricorsiva, che crea un Heap a partire da un array qualsiasi. In pratica viene creato un heap sinistro e poi un heap destro. Alla fine di tutto ciò si esegue un **FixHeap** per portare la struttura heap finale al termine. Idealmente, prima creo tutto il sottoalbero di sinistra, poi tutto il sottoalbero di destra. Al termine di ciò sistemo i due sottoalberi portandoli ad una struttura di tipo heap. Se si vuole ragionare tramite un array, ogni chiamata ricorsiva ci sposta nella porzione dell’array da sistemare, fino al raggiungimento della sistemazione finale. Il costo è analizzabile tramite la seguente equazione di ricorrenza.

$$T(n) = \begin{cases} 1 & , \quad n = 0 \\ 2T(n/2) + \log_2 n & , \quad n > 0 \end{cases}$$

La soluzione di questa equazione non è direttamente applicabile tramite Master Theorem; tuttavia, è possibile semplificare il $\log_2 n$ per poter risolvere questa equazione tramite Master Theorem.

Nota! $2T(n/2)$ due chiamate ricorsive che dimezzano ogni volta l'ampiezza sul quale eseguire la chiamata.

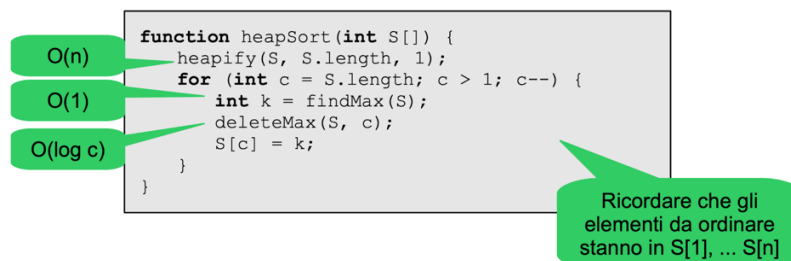
$$T(n) = \begin{cases} 1 & n = 0 \\ 2T(n/2) + \log_2 n & n > 0 \end{cases} \leq \begin{cases} 1 & n = 0 \\ 2T(n/2) + n & n > 0 \end{cases} \leq \begin{cases} 1 & n = 0 \\ 2T(n/2) + (n)^{\frac{1}{2}} & n > 0 \end{cases}$$

Dal Master Theorem è possibile approssimare l'equazione di ricorrenza a $O(n \log n)$ per $n \leq 16$, mentre sempre per $n > 16$ è possibile approssimare l'equazione a $O(n)$.

HeapSort

Questo algoritmo di ordinamento si svolge in quattro fasi e permette un costo pseudolineare.

1. Costruisco un Max-Heap a partire dall'array passato come parametro alla funzione
2. Estraggo il massimo, lo posiziono nella prima posizione non ordinata dell'array dato come valore di ritorno.
Estrazione del valore tramite FindMax e DeleteMax, facendo contrarre l'Heap di un elemento
3. Posiziono effettivamente nella prima posizione non ordinata dell'array il valore estratto
4. Ritorno al punto numero 2 fino a che l'Heap non raggiunge dimensione 0.



$$T(n) = O(n) + O\left(\sum_{i=1}^n \log i\right) = O(n \log n)$$

Tralasciando l'equazione si sa che all'inizio della funzione si paga un costo pari a $O(n)$ per la creazione dell'Heap, successivamente per n volte (dimensione dell'heap) si esegue una **DeleteMax**, che sappiamo avere costo pari a $O(\log_2 n)$. Pertanto, il costo finale risulta pari a $O(n \log n)$.

9.2^a Lezione

Code con Priorità

Una coda con priorità è una struttura dati che ha diversi elementi in memoria. Ogni elemento è rappresentato da una chiave (la priorità) e dall'informazione. Le chiavi, quindi le priorità sono disposte in ordine crescente, e più il valore di tale chiave è basso più la priorità dell'elemento è elevata.

Operazioni

Come per gli alberi binari Heap si possono distinguere diverse operazioni possibili.

FindMin è una funzione che ritorna il valore associato all'elemento con chiave minore, in altre parole ritorna l'elemento con priorità maggiore. **Insert**, operazione che inserisce nella struttura dati un elemento e associa a tale elemento la sua priorità. **Delete**, esegue l'operazione opposta della Insert.

IncreaseKey e DecreaseKey, sono operazioni che aumentano o diminuiscono la priorità dell'elemento all'interno della struttura dati, da notare che quando la priorità viene aumentata, il valore della chiave viene diminuito, questo perché le chiavi con valore più basso andranno "eseguite" per prime.

Implementazione

Si possono creare queste code con priorità tramite alberi d-Heap, per la precisione alberi d-Heap Minimi, ovvero i figli di ogni nodo assumeranno un valore maggiore rispetto al loro padre.

Si parla di alberi d-Heap e non binari, perché ogni nodo può avere più di due figli, ma può avere fino a

“d” figli. Gli alberi d-Heap in questione sono esattamente degli alberi binari Heap, ma che hanno più figli, di conseguenza, l’implementazione delle operazioni seguirà più o meno lo stesso schema delle operazioni studiate nel capitolo precedente.

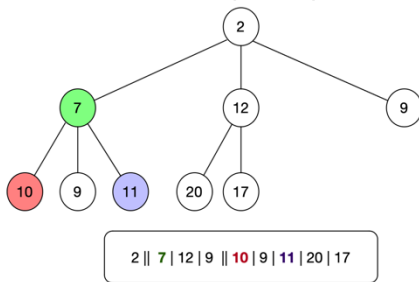
Alberi d-Heap

Estendono il concetto degli alberi heap, ma ogni nodo può avere al massimo “d” figli e ogni nodo “padre” ha un valore prioritario (chiave) minore rispetto ai propri figli.

Sono sempre alberi binari quasi perfetti e l’altezza massima dipende dal: $\log_d N$ dove “N” rappresenta il numero di nodi presenti nell’albero.

Dimostrazione dell’altezza dell’albero binario nelle slides relative al capitolo di riferimento – slide 27

Come detto nel capitolo precedente l’alberi d-Heap è rappresentabile anche sotto forma di un array



La posizione della chiave “11” all’interno dell’array è pari a:

$$(pos(padre) * d) + 1 \rightarrow pos(11) = 2 * 3 + 1 = 7$$

La posizione della chiave “10” è pari a:

$$((pos(padre) - 1) * d) + 2 \rightarrow pos(10) = (2 - 1) * 3 + 2 = 5$$

La posizione del padre di un nodo è pari a:

$$(pos(figlio) - 1) / d \rightarrow pos(7) = 5 - 1 / 2 = \frac{4}{2} = 2$$

(arrotondato per eccesso)

Operazioni Ausiliarie Fondamentali

In seguito, torneranno utilissime due funzione ausiliarie, la prima tramite dei confronti porta verso l’alto il nodo, la seconda, invece, fa esattamente l’opposto, porta verso il basso con chiave maggiore.

	MoveUp	MoveDown
Costo	$O(\log_d n)$	$O(d \log_d n)$

Il costo di MoveDown pseudolineare perché per tutta l’altezza dell’albero bisogna confrontare ogni nodo con i suoi fratelli, e questa cosa come detto viene eseguita per tutti i livelli dell’albero.

```

procedura muoviAlto(v)
  while ( v != root(T) and
    chiave(v) < chiave(padre(v)) ) do
    scambia di posto v e padre(v) in T;
    v := padre(v);
  endwhile

```

Costo: $O(h)$

```

procedura muoviBasso(v)
  repeat forever
    if ( v non ha figli ) then
      return;
    else
      sia u il figlio di v con la minima chiave(u)
      if ( chiave(u) < chiave(v) ) then
        scambia di posto u e v;
        v := u;
      else
        return;
      endif
    endif
  repeat

```

Costo: $O(d)$

Costo: $O(dh)$

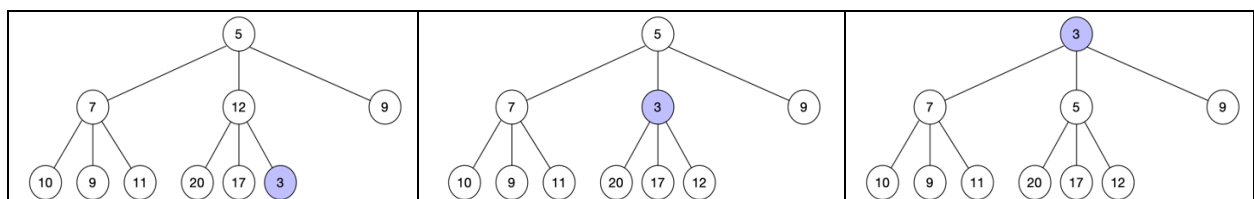
FindMin

Viene preso il valore in cima all’array, che ovviamente è anche quello più piccolo. Costo: $O(1)$

Insert

Viene inserito nell’albero d-heap (come foglia) il nuovo elemento, a cui viene associato un valore e una chiave. Successivamente si iniziano a fare degli spostamenti dei valori tramite la procedura ausiliaria precedentemente introdotta: **MoveUp / MuoviAlto**.

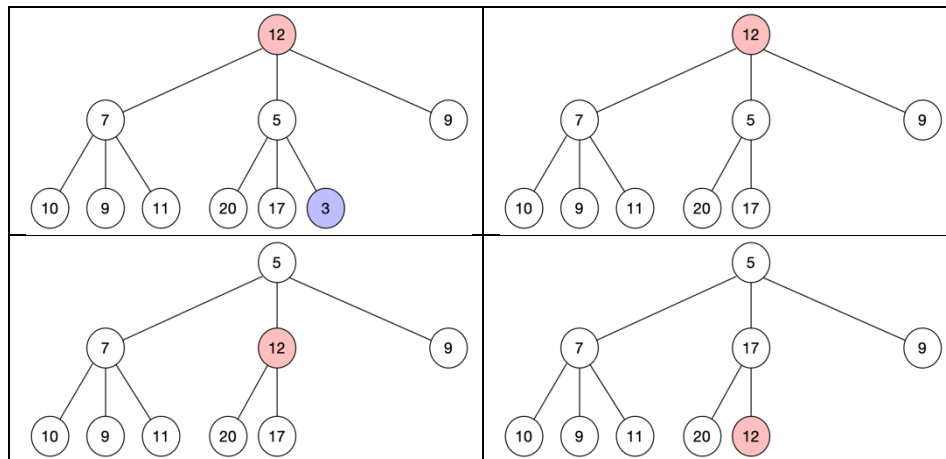
Il costo, come si può intuire dipende dal numero massimo di confronti possibili, nonché dalla procedura muovi alto, che come detto precedentemente ha un costo massimo pari a $O(\log_d n)$



Delete

Per creare una delete senza eseguire delle rotazioni (che non sono supportate dagli alberi d-Heap) si può procedere in modo analogo rispetto all'inserimento.

Rimuovere un elemento significa diminuire la dimensione dell'albero, e la cosa più facile da fare per diminuire la dimensione dell'albero è quella di rimuovere la foglia più a destra dell'albero, sostituire i valori (tra elemento da rimuovere ed elemento da mantenere) ed eseguire degli spostamenti sia in alto che in basso. Il costo, quindi, risulta essere pari a $O(d \log_d n)$



DecreaseKey – IncreaseKey

Come si può capire dagli esempi precedenti, per DecreaseKey vengono effettuati degli spostamenti verso l'alto, mentre per IncreaseKey vengono svolti degli spostamenti verso il basso.

Tabella riassuntiva

FindMin	Insert (esempio)	Delete
$O(1)$	$O(\log_d n)$	$O(d \log_d n)$
IncreaseKey	DecreaseKey	DeleteMin (esempio)
$O(d \log_d n)$	$O(\log_d n)$	$O(d \log_d n)$

Esercizi

10^a Lezione

Union Find

L'idea della struttura dati Union Find è quella di creare insiemi disgiunti per poi unire questi insiemi. L'insieme o gli insiemi uniti ora faranno riferimento a un rappresentante univoco; dunque, la ricerca di un elemento sarà di immediata velocità in quanto si saprà subito il rappresentante di riferimento di quell'elemento. Immaginiamo una cartina divisa per comuni, quando si ricercherà una via/località il risultato della ricerca ritornerà il comune sotto la quale la località ricade. In questo modo sarà possibile rintracciare in modo rapido l'appartenenza della via al comune di riferimento.

Operazioni Fondamentali e Struttura Dati

Le operazioni fondamentali sono principalmente tre. La prima operazione è la creazione di un insieme che contiene un solo nodo, quindi la creazione di un singoletto. La seconda operazione fondamentale è l'unione di insiemi (siano essi disgiunti o già precedentemente uniti). Il terzo metodo è l'identificazione dell'insieme di appartenenza di un elemento. La struttura dati è quindi una collezione di insiemi dinamici disgiunti, dove ogni insieme è identificato da un rappresentante univoco, ovvero il rappresentante di quell'insieme.

Operazioni Dettagliate

La prima operazione che vedremo è **makeSet**, ovvero l'operazione di creazione di un insieme il cui unico elemento (e rappresentante) è "x" e tale elemento non appartiene a nessun'altro insieme.

La seconda operazione è **Find**, ovvero l'operazione che restituisce il rappresentante dell'unico insieme contenente "x" (assumendo di avere accesso diretto all'elemento "x"). La terza operazione è infine

Union, che unisce i due insiemi rappresentati da "x" e da "y", sceglie un nuovo rappresentante e distrugge i vecchi insiemi per crearne uno unito.

makeSet(i) i=1..6	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>	<u>5</u>	<u>6</u>
union(1,2)	1, 2	<u>3</u>	<u>4</u>	<u>5</u>	<u>6</u>	

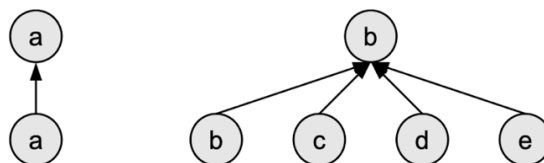
Implementazioni Possibili

Vedremo due approcci implementativi per implementare la struttura dati, e due miglioramenti su tali implementazioni. Da notare che le implementazioni di riferimento saranno le implementazioni migliorate sugli algoritmi elementari.

QuickFind e **QuickUnion** saranno gli algoritmi elementari, mentre **QuickFind – Euristica sul Peso** sarà il primo miglioramento e **QuickUnion – Euristica sul Rango** sarà il secondo miglioramento.

QuickFind

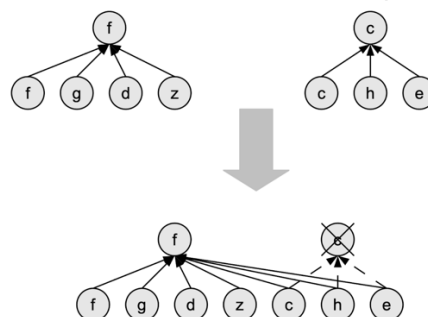
L'idea è quella di rappresentare gli insiemi (aggregati di elementi) tramite un albero di altezza uno, dove c'è una radice (rappresentante dell'insieme), con n -foglie come elementi dell'insieme.



In questo approccio l'operazione **Find** è veloce, infatti basta guardare il padre (la radice) della foglia. *Costo: $O(1)$* , infatti basta seguire un puntatore. Medesima cosa per l'operazione **Make**.

Più complessa è l'operazione di **Union**, in quanto devono essere presi due insiemi. Il secondo insieme verrà distrutto con tutti gli elementi che passeranno al primo insieme (quindi faranno riferimento al rappresentante del primo insieme).

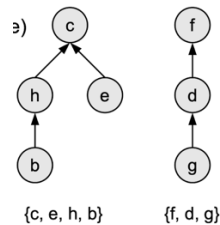
Nota! I puntatori vanno dalle foglie alla radice, e non viceversa, questo è importante per i costi.



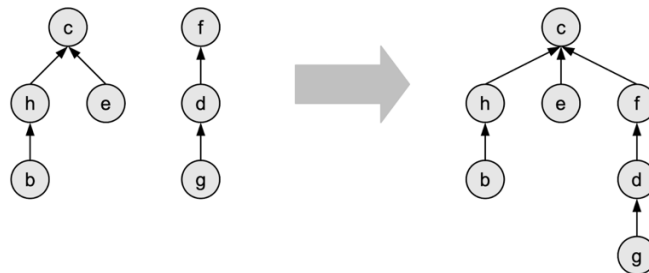
Il costo computazionale sarà pari a $O(n)$ dove " n " sono i nodi del secondo insieme. Questo approccio risulta abbastanza lento se il primo insieme è un singoletto, e se il secondo insieme è molto grande, in quanto si dovranno scambiare tutti i puntatori del secondo insieme, che abbiamo appena detto essere più grande. L'approccio euristico agirà sulla cardinalità degli elementi, ovvero verranno scambiati i puntatori dell'insieme più piccolo.

QuickUnion

L'implementazione è sempre creata tramite degli alberi, che questa volta sono alberi radicati generici. Ogni nodo dell'albero contiene l'oggetto e un puntatore al padre. Ancora una volta il rappresentante è la radice.



Come si può evincere dall'immagine, l'operazione **MakeSet** ha sempre costo costante $O(1)$, ma l'operazione **Find** ha costo lineare, pari a $O(n)$, in quanto si avranno degli alberi molto profondi, tali per cui la risalita da figlio a radice può essere molto dispendiosa. Questa struttura, tuttavia, premia l'operazione **Union** in quanto basta innestare l'albero con radice "y" all'albero con radice "x", ottenendo quindi un costo costante: $O(1)$.



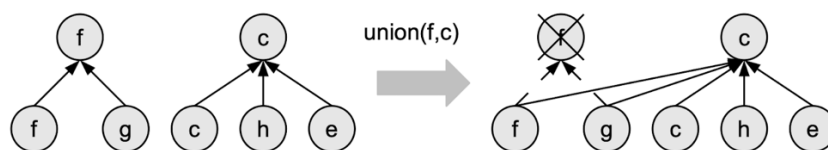
La teoria Euristica sul Ranking si baserà sul fatto di implementare l'union per creare alberi meno profondi, creando quindi delle unioni innestando l'alberi più piccolo all'albero più grande e non viceversa.

QuickFind – Euristica sul Peso

Eredita tutte le caratteristiche della struttura dati definita precedentemente, ora ragioniamo e miglioriamo l'unione, il punto critico di questa implementazione.

Esempio

Union migliorata, al posto di tre operazioni se ne fanno due:



Questa soluzione nel caso pessimo cambia veramente poco, perché il miglioramento è esiguo, e, nel calcolo generale del costo tutte le costanti vengono scartate, generando quindi un costo pari a $O(n)$. Se facciamo alcune osservazioni e calcoliamo il costo tramite analisi ammortizzata possiamo dedurre un costo pari a $O(\log n)$.

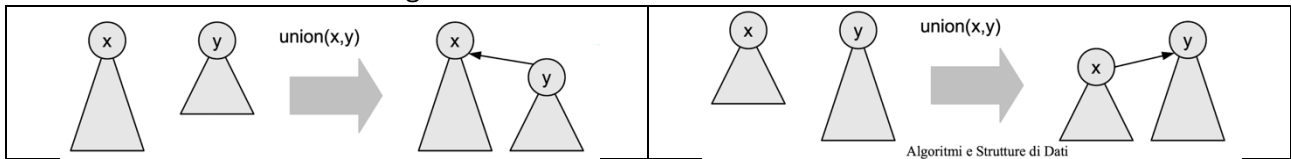
1^a Osservazione: se una foglia cambia padre, allora farà parte di un insieme che avrà almeno il doppio di elementi a quello cui apparteneva.

2^a Osservazione: avendo " n " insiemi separati, al massimo potrò fare " $n-1$ " unioni.

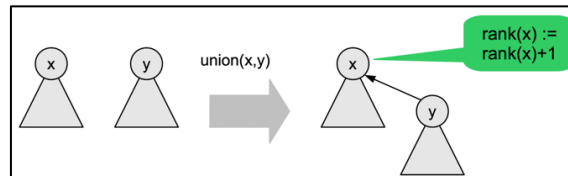
QuickUnion – Euristica sul Rango

Questa implementazione, come l'implementazione analizzata prima eredita tutte le caratteristiche della struttura dati associata a questo miglioramento. Il punto critico di questa implementazione era la ricerca, in quanto si venivano a creare alberi molto profondi, che impedivano una ricerca rapida.

L'idea, nonché il miglioramento è capire il rango di un albero (ovvero profondità) e innestare l'albero più piccolo (quindi con meno rango, ovvero meno profondo) come sottoalbero dell'albero di rango maggiore. Tramite questo stratagemma il rango globale di un albero non cambia se ad un albero viene innestato un altro alberi con rango minore.

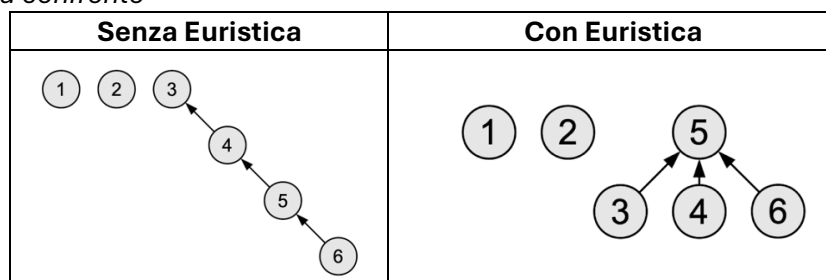


Come caso pessimo, invece, abbiamo due alberi di uguale rango, qui la questione è praticamente la medesima delle immagini soprastanti, con l'unica eccezione che il rango crescerà di uno.



Idealmente: Non vengono a crearsi lunghi alberi, con nodi che sono figli di figli..., ma rendiamo la radice dell'albero più basso figlia della radice dell'albero più alto, generando quindi alberi binari che sono il più bassi possibile.

Implementazioni a confronto



Dopo questi accorgimenti, il costo che si va a pagare per una Find() di QuickUnion con Euristica sul Rango è pari a $O(\log n)$.

Riassunto dei costi

	QuickFind	QuickUnion	QuickFind_Eur	QuickFind_Eur
MakeSet	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Union	$O(n)$	$O(1)$	$O(\log n) *$	$O(1)$
Find	$O(1)$	$O(n)$	$O(1)$	$O(\log n)$

* := Costo derivante da analisi con il costo ammortizzato

I costi logaritmici, per quanto riguarda le implementazioni che prevedono Euristica seguono dei teoremi, e fanno supposizioni, che portano il costo ad essere pari a " $O(\log n)$ ". Qui sono riportate solo alcune scelte di questi studi, tuttavia per uno studio completo guardare le slides relative al capitolo di riferimento, complete di spiegazioni, scelte, e analisi complete dei costi.

Esercizi

11^ Lezione

Tecniche Algoritmiche

1^ Tecnica – Divide – et – Impera

Questo capitolo è incentrato sulle tecniche algoritmiche, ovvero sulle tecniche che si possono adottare nella risoluzione di esercizi e di problemi.

La prima tecnica analizzata è la ricorsione, la seconda è la tecnica *greedy*, che contiene tante strade possibili, ma in base alla situazione verificata viene eseguito l'algoritmo più efficiente. Ad esempio, se bisogna trovare un elemento all'interno di un array e tale array è ordinato, la strada proposta sarà la ricerca binaria, se invece l'array non è ordinato verrà eseguita la ricerca binaria. L'ultima tecnica invece è la programmazione dinamica, simile alla ricorsione, ma nata per risolvere i problemi legati alla ricorsione.

Prima Tecnica – Ricorsione

Guardare le slides relative al capitolo di riferimento, già studiata precedentemente. Nelle slides ci sono degli esempi. La cosa difficile della ricorsione è il calcolo del costo computazionale tramite le equazioni di ricorrenza e la scrittura dello pseudocodice efficiente (richiesta negli esercizi). La cosa migliore per allenarsi in questa parte è lo svolgimento di [esercizi](#) di vario tipo.

Esercizio Proposto

Esercizio. Si consideri un array $A[1..n]$ contenente valori reali ordinati in senso non decrescente; l'array può contenere valori duplicati. Scrivere un algoritmo ricorsivo di tipo divide-et-impera che, dato A e due valori reali $low < up$, calcola quanti valori di A appartengono all'intervallo $[low, up]$. Determinare il costo computazionale dell'algoritmo proposto.

Soluzione 1:

Algorithm 2: $CONTAINTERVALLO(Reale\ A[1..n], Reale\ low, Reale\ up, Intero\ i, Intero\ j) \rightarrow Intero$

```
if  $i > j$  then
| return 0
else if  $A[i] \geq low$  and  $A[j] \leq up$  then
| return  $j - i + 1$ 
else if  $A[i] > up$  or  $A[j] < low$  then
| return 0
else
|  $Intero\ m = Floor((i + j)/2)$ 
| return  $CONTAINTERVALLO(A, low, up, i, m) + CONTAINTERVALLO(A, low, up, m+1, j)$ 
```

Soluzione 2:

```
bool Function In_Interval (int n, int low, int up){
    if (n > low && n < up){
        return true
    }
    return false
}

Int Function Interval_Numbers (int A[], int i, int j, int low, int up)
if (i > j){
    return 0; //array vuoto
}
else if (A[i] > up || A[j] < low){
    return 0; //nessun numero dell'array ricade all'interno dell'intervallo
}
else if (In_Interval(A[i], low, up) == true){
    return 1 + Interval_Numbers(A, i + 1, j, low, up); //sommo e continuo visita
}
return Interval_Numbers(A, i + 1, j, low, up); //continuo visita
```

2^a Tecnica – Greedy // Esercizi

La tecnica greedy permette di risolvere problemi attuando la scelta che momentaneamente sembra la migliore possibile, sperando che in un futuro questa scelta paghi i suoi risultati.

Un esempio di tecnica greedy è il problema, del resto, ovvero dato un importo, il resto lo si restituisce partendo dal taglio di monete più elevato; nessuno darebbe 75 euro di resto con monete da 1 centesimo, ma si partirebbe dalla banconota da 50, poi quella da 20 e poi quella da 5.

Algoritmo di Scheduling

Immaginiamo di avere diverse attività, dove ogni attività richiede diverso tempo; il nostro obiettivo è quello di far attendere ogni attività il meno possibile, quindi gestire le attività in modo tale che il tempo medio di attesa sia il più basso possibile. L'approccio greedy è quello di **eseguire le attività più semplici per prime**, per poi avere più tempo a disposizione per le attività difficili.

Attività	
4	
1	
6	
3	

Date diverse attività, ognuna che richiede istanti temporali organizziamo le attività e vediamo il tempo medio di attesa.

1° caso:

4	1	6	3
---	---	---	---

$$(0 + 4 + 5 + 11) / 4 = 20 / 4$$

Formula: istanti temporali che passano (esclusi gli ultimi per fare l'ultima operazione) / n°operazioni

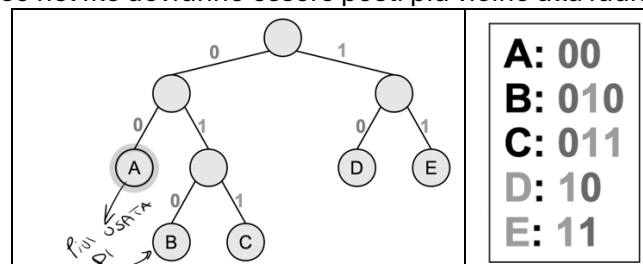
2° caso:

1	3	4	6
---	---	---	---

$$(0 + 1 + 4 + 8) / 4 = 9 / 4$$

Codici di Huffman

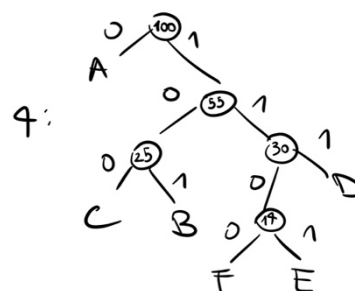
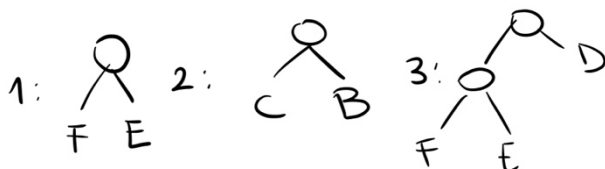
Dati dei caratteri e quante volte un carattere è all'interno di un file, si possono comprimere questi file usando meno bit. Ad esempio, lo standard di codifica dei caratteri è lo standard ASCII che utilizza 8 bit per carattere. Tuttavia, utilizzare sempre 8 bit per rappresentare dei caratteri può risultare dispendioso, se vengono usati solo un numero limitato di caratteri e non tutti i possibili rappresentabili. L'idea che si può avere è quella di utilizzare un albero binario, dove al posto delle foglie sono posti i caratteri e il percorso radice foglia indica la codifica del carattere da seguire. Logicamente, i caratteri che compaiono più spesso nel file dovranno essere posti più vicino alla radice.



Esempio

Data la seguente lista, l'idea è quella di collegare a due a due i nodi fino a farne rimanere 1 alla lista

F: 5 ; E: 9; C: 12; B: 13; D: 16; A: 45



3^ Tecnica – Programmazione Dinamica

La programmazione dinamica adotta la tecnica definita come memoization. In pratica dopo aver analizzato il problema e scritto la soluzione di quel problema in forma algebrica si passa a scrivere un algoritmo non ricorsivo. L'approccio tramite questo metodo permette di non eseguire più volte la stessa chiamata ricorsiva, ma garantisce il calcolo del problema tramite moltiplicazioni, sottrazioni... tra celle della tabella di memoization che si va via a via a costruire.

Esempio – Distanza di Levenshtein

Date due stringhe differenti, quante operazioni di editing devo fare per trasformare la stringa "S" nella stringa "T" sapendo che: operazioni di cancellazione, inserimento e sostituzione "costano" un'unità. Per **distanza di Levenshtein** si intende il numero minimo di operazioni che trasformano "S" in "T".

Soluzione e Approccio

Si identificano diversi casi base.

Caso 1: Entrambe le stringhe, quindi sia "S" che "T" sono stringhe vuote; quindi, non bisogna far nulla

Caso 2: Una stringa è vuota e una stringa è piena. Per cambiare la stringa "S" in "T" (indifferentemente di quale sia vuota o piena) sono necessarie tante operazioni quanti i caratteri all'interno della stringa piena.

Caso 3: Ho due stringhe esattamente uguali, dove l'unica differenza risiede nell'ultimo carattere, per trasformare "S" in "T" devo effettuare una cancellazione (di costo 1) sull'ultimo carattere di "S".

Caso 4: Ho due stringhe esattamente uguali, ma l'ultimo carattere di "T" non compare nella stringa "S", devo quindi inserire l'ultimo carattere "T" in "S".

Caso 5: Le due stringhe sono uguali, ma l'ultimo carattere di "S" differisce dall'ultimo carattere di "T", devo quindi effettuare una sostituzione.

Sia quindi "L" la tabella che andiamo a comporre per calcolare la distanza tra parole, se "S" o "T" saranno una vuota e l'altra piena la distanza sarà pari a: $\max\{i, j\}$, dove "i" sono i caratteri della stringa "S" e "j" i caratteri della stringa "T".

Se $S[i] = T[j]$, allora il costo sarà pari al minimo tra: $L[i - 1, j] + 1$ (cancellazione); $L[i, j - 1] + 1$ (aggiunta) e $L[i - 1, j - 1]$ (invariato);

Se $S[i] \neq T[j]$, allora il costo sarà pari al minimo tra $L[i - 1, j] + 1$; $L[i, j - 1] + 1$; $L[i - 1, j - 1] + 1$.

Esempio con la Tabella completa alla slide 57 degli appunti del capitolo di riferimento.

In generale, viene prima riempita la prima colonna e la prima riga della tabella, e successivamente riempire la tabella per righe. Una volta completato l'inserimento la distanza sarà pari al valore contenuto nell'ultima riga dell'ultima colonna della tabella.

Esercizi

Esercizio di Esempio

Esercizio. Si consideri il problema del resto nel caso generale, ovvero non assumendo di avere a disposizione monete di un sistema canonico. Più precisamente si deve stampare la combinazione con il numero minimo di monete, se esiste, per raggiungere il resto R considerando come possibili valori delle monete quelli indicati nell'array di naturali $T[1..n]$.

Soluzione – Passo Algebrico

Supponiamo di voler risolvere il problema della restituzione del resto con programmazione dinamica.

Consideriamo due indici:

$$i \in \{1, \dots, n\} \quad (\text{numero di tagli disponibili})$$

$$j \in \{0, \dots, R\} \quad (\text{resto da restituire})$$

Dove:

- $T[i]$ rappresenta il valore del taglio di moneta in posizione i
- R è il resto totale da restituire

Casi base:

$$P(1, j) = \begin{cases} j/T[1] & \text{se } j \bmod T[1] = 0 \\ -1 & \text{altrimenti} \end{cases}$$

Casi generali:

$$P(i, j) = \begin{cases} 0 & \text{se } j = 0 \\ P(i-1, j) & \text{se } T[i] > j \\ \min \{P(i, j - T[i]) + 1, P(i-1, j)\} & \text{altrimenti} \end{cases}$$

Dove $P(i, j)$ rappresenta il numero minimo di monete necessarie per restituire il resto j utilizzando i primi i tagli di moneta disponibili.

Soluzione – Pseudocodice

```
void function Conta_Resto (int R, int T[1..n]){
    int P [T[1..n], 0..R] //tabella, nelle righe i tagli e nelle colonne tutti i numeri da 0 al resto
    for (j = 0 .. R){ //ciclo che mi permette di muovermi tra le colonne, inserimento effettuato sempre in riga
        if ( j%T[1]==0 ) {
            P[1,j] = j/T[1]
        }
        else{
            P[1,j] = -1; //numero negativo, ovvero resto nullo
        }
    }
    for (i = 2 .. lenght(T[1..n])){
        for(j = 0 .. R){
            if (j == 0) {
                P[i, j] = 0
            }
            else if (T[i]>j) {
                P(i, j) = P(i - 1, j)
            }
            else {
                P(i, j) = min(P(i - 1, j), (i, j - T[i])+1)
            }
        }
    }
}

int taglio = n
int resto = R
if P[taglio, resto] == -1
    print ("Impossibile Erogare Resto") //parto dall'ultima cella
while resto > 0 {
    if (taglio == 1) { //ultima moneta dell'ultimo taglio disponibile da erogare
        print ("Una moneta da: " << T[taglio])
        resto = resto - T[taglio]
    }
    else if (P(taglio, resto) == P(taglio - 1, resto)){ //significa che c'è una combinazione migliore
        taglio = taglio - 1
    }
    else{ //bisogna erogare il resto
        print ("Una moneta da: " << T[taglio])
        resto = resto - T[taglio] //successivamente guarderò un'altra riga e un'altra colonna
    }
}
```

12^ Lezione

Grafi

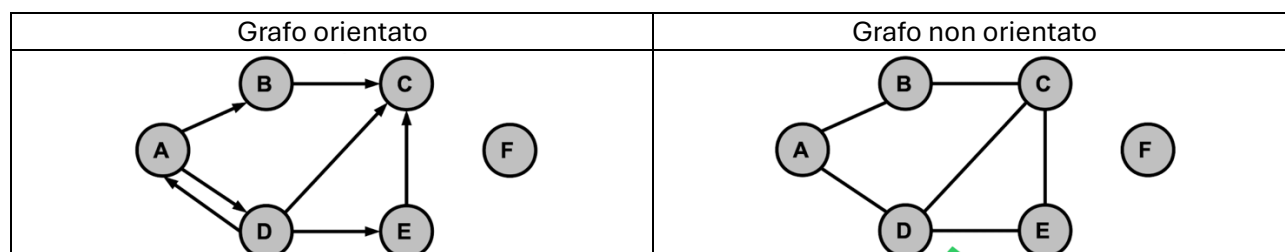
Un grafo lo si può immaginare come un albero con i nodi che possono puntare ad altri nodi, e dove non necessariamente un padre ha un figlio, ma tutti i nodi possono puntare a chiunque.

Innanzitutto, nei grafi non si parla di nodi ma di vertici, e i collegamenti vengono chiamati archi, inoltre, a seconda del tipo di collegamento tra vertici il grafo assume nomi diversi. Si parla di grafo orientato se l'arco ha una direzione, se quindi si può percorrere da sinistra verso destra, ma non viceversa. Si parla di grafo non orientato se un arco si può percorrere in entrambe le direzioni. Infine, si parla di grafi pesati se il percorso di ogni arco ha un peso, ovvero un "costo" da pagare per passare da un vertice ad un altro.

Algoritmi

I problemi sui grafi sono diversi, ma sono analogamente simili ai problemi degli alberi. Un problema comune è la visita del grafo, l'albero di copertura minimo e il cammino minimo. Tutti i vari problemi verranno analizzati in seguito. Ora definiamo in modo chiaro il grafo e come può presentarsi.

Grafi Orientati e Non Orientati



I nodi vengono detti vertici, mentre i percorsi vengono detti archi o edges. Nei grafi orientati esiste una relazione binaria tra i vertici; quindi, gli archi possono essere visti come un insieme che racchiude le coppie ordinate di vertici, mentre nei grafi non ordinati si possono vedere gli archi come un insieme che racchiude degli insiemi di copie non ordinate. Per esempio, nel grafo non orientato dire $\{A, B\}$ o $\{B, A\}$ è la stessa cosa.

Incidenza e Adiacenza

Un arco (v, w) si dice incidente da v a w , mentre si dice che w è adiacente a v se esiste un arco incidente tra i due vertici.

Rappresentazione / Costruzione di Grafi

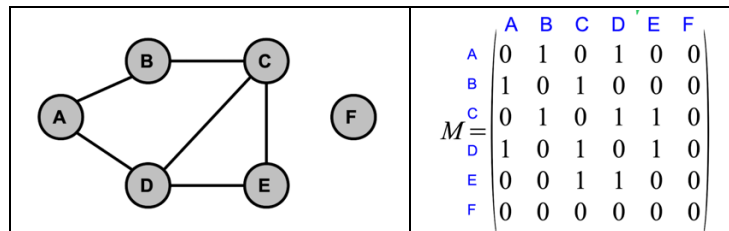
Operazioni	Funzione
Num_Vertici() $\rightarrow$ Int	Ritorna il numero di vertici di un grafo
Num_Archi() $\rightarrow$ Int	Ritorna il numero di archi di un grafo
Grado (Vertice v) $\rightarrow$ Int	Misura quanti archi sono collegati a quel particolare vertice
Archi_Incidenti(Vertice V) $\rightarrow$ [arco, arco, ..., arco]	Dato un vertice ci restituisce una lista di archi incidenti a quel vertice (archi che partono)
Estremi (Arco E) $\rightarrow$ [vertice, vertice]	Dato un arco viene restituita coppia di vertici che collega quell'arco
Sono_Adiacenti (Vertice x, Vertice y) $\rightarrow$ Bool	Dati due vertici, mi dice se tali vertici sono collegati direttamente da un arco
Aggiungi_Vertice(Vertice v)	Aggiunge un vertice al grafo
Aggiungi_Arco(Vertice v)	Aggiunge un arco al grafo

Rimuovi_Vertice(Vertex v)	Rimuove un vertice dal grafo
Rimuovi_Arco(Arco e)	Rimuove un arco da grafo

Matrici di Adiacenza – Costruzione (Grafo Non Orientato)

Dato un grafo non orientato viene costruita una matrice booleana che indica se esiste un vertice da (A,B). Se tale vertice esiste nella matrice booleana viene inserito 1, altrimenti uno zero. Da notare, che la matrice è simmetrica, e sulla diagonale principale sono posti tutti zero, in quanto nei grafi non orientati i cappi non sono ammessi

Il costo computazionale di tale costruzione: $\Theta(V^2)$

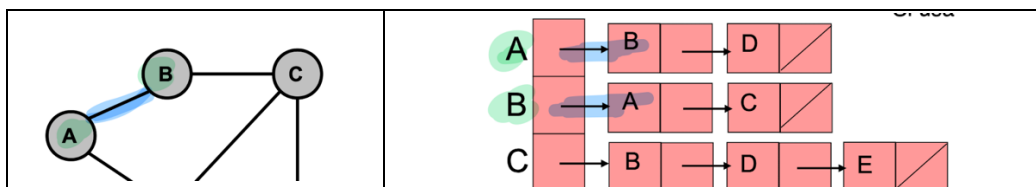


Matrici di Adiacenza – Costi (Grafo Non Orientato)

Operazioni	Costo
Grado (Vertex v) $\rightarrow$ Int	$\theta(n)$
Archi_Incidenti(Vertex V) $\rightarrow$ [arco, arco, ..., arco]	$\theta(n)$
Sono_Adacenti (Vertex x, Vertex y) $\rightarrow$ Bool	$O(1)$
Aggiungi_Vertice(Vertex v)	$\theta(n^2)$ – Ricostruzione matrice
Aggiungi_Arco(Vertex v)	$O(1)$
Rimuovi_Vertice(Vertex v)	$\theta(n^2)$ – Ricostruzione matrice
Rimuovi_Arco(Arco e)	$O(1)$

Liste di Adiacenza – Costruzione (Grafo Non Orientato)

In memoria vengono rappresentati solo gli archi che esistono, con il vantaggio di usare meno memoria. Tuttavia, per il grafo non orientato vengono generati due nodi, questo significa che ogni nodo compare due volte nelle liste di adiacenza (Esempio nell'immagine).



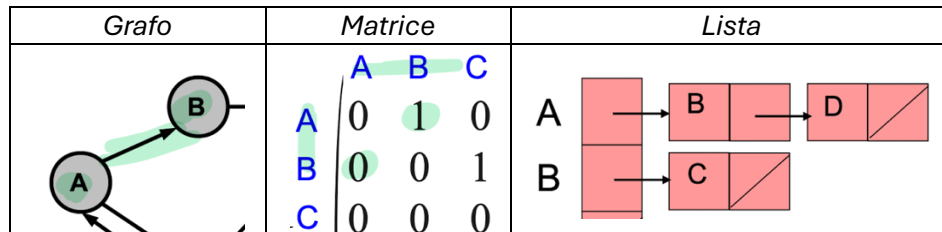
Liste di Adiacenza – Costi (Grafo Non Orientato)

Operazioni	Costo
Grado (Vertex v) $\rightarrow$ Int	$O(\delta(v))$
Archi_Incidenti(Vertex V) $\rightarrow$ [arco, arco, ..., arco]	$O(\delta(v))$
Sono_Adacenti (Vertex x, Vertex y) $\rightarrow$ Bool	$O(\min(\delta(x), \delta(y)))$
Aggiungi_Vertice(Vertex v)	$O(1)$
Aggiungi_Arco(Vertex v)	$O(1)$
Rimuovi_Vertice(Vertex v)	$O(m)$
Rimuovi_Arco(Arco e)	$O(\delta(x) + \delta(y))$

Nota: δ = grado del vertice; n = vertici, m = archi

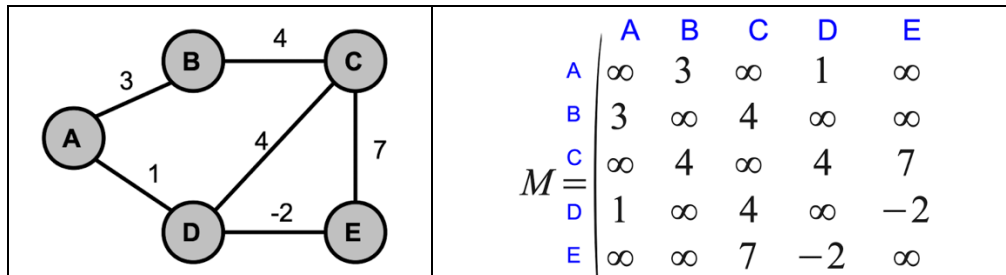
Liste e Matrici di Adiacenza (Grafici Orientati)

I costi sono i medesimi e le implementazioni sono le stesse, cambia solamente la composizione delle matrici e delle liste. Per quanto riguarda le matrici, ora non sono più simmetriche, mentre per quanto riguarda le liste ora viene usato ancora meno spazio in memoria, in quanto non bisogna rappresentare le doppie coppie. (Come si può vedere dalle immagini)



Grafici Pesati

Nei grafici pesati ogni arco ha un costo di attraversamento, nel caso della rappresentazione tramite matrici, la matrice non sarà più di tipo booleano, ma di tipo intero, e al posto degli "1" verrà inserito il peso di attraversamento di quell'arco. Inoltre, se l'arco non esiste viene inserito un infinito, ovvero l'ipotetico costo per costruire un arco da vertice a vertice. Anche qui i costi sono i medesimi, e la rappresentazione anche.



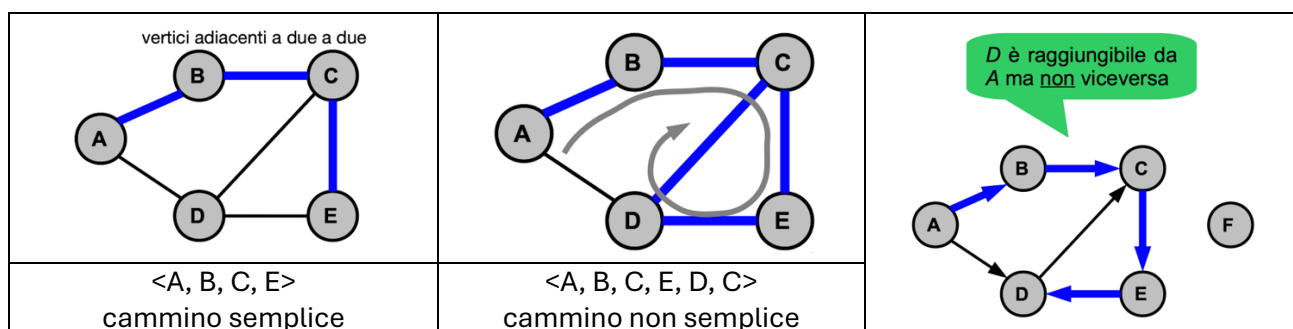
Grado

In un grafico non orientato il grado di un vertice è il numero di archi che partono da un vertice. Si può parlare sia di grado in entrata che di grado in uscita. Il grado in entrata è quanti archi arrivano a un vertice, mentre il grado in uscita è quanti archi partono da quel vertice. In generale, per grado si intende la somma di entrambi, o più comunemente il grado in uscita.

Cammini

Siano dei vertici, adiacenti a due a due, e sia un cammino definito come: {A, B, C, E}, la lunghezza minima del cammino è pari al numero di vertici meno 1.

Un cammino semplice è un cammino che passa per i vertici una ed una sola volta.



Nei grafici non orientati se esiste un cammino si può dire che A è raggiungibile da D e viceversa, nei grafici non orientati non è detto (esempio nell'immagine)

Grafi Connessi

Se un grafo è non orientato, si dice che il grafo è connesso se esiste un cammino da ogni vertice ad ogni altro vertice. Se G è un grafo orientato diciamo che G è fortemente connesso se esiste un cammino da ogni vertice ad ogni altro vertice

Non connesso	Connesso	Fortemente Connesso	Debolmente Connesso

Un grafo si dice debolmente connesso se la sua versione non orientata (esclusa di cappi) è connessa

Cicli

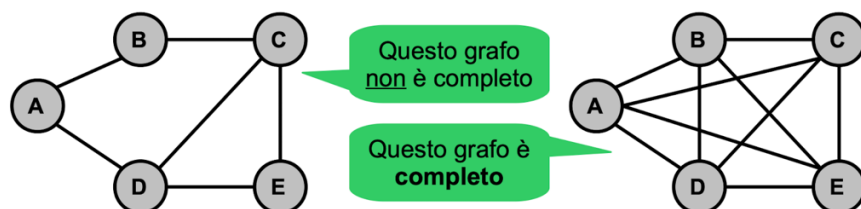
Un ciclo è un cammino con lo stesso punto di arrivo e di partenza. Un ciclo semplice è un cammino di lunghezza minore possibile, un ciclo non semplice è un cammino che ripercorre gli stessi archi già percorsi. Un grafo orientato è aciclico se è privo di cicli, mentre un grafo non orientato è aciclico se è privo di cicli semplici.

Nota! Un grafo orientato aciclico viene anche chiamato DAG

Grafo ciclico	Grafo Aciclico	DAG

Grafo Completo

Un grafo non orientato completo viene detto tale se ogni vertice ha un collegamento con ogni altro vertice, ovvero ogni vertice ha un arco tra ogni coppia di vertici.



13^ Lezione

Visite dei Grafi

Dato un grafo e un vertice sorgente (detto s) l'algoritmo di visita deve essere capace di visitare ogni vertice raggiungibile del grafo partendo dal nodo sorgente. Le visite sono sia in ampiezza che in profondità. La prima punta a scoprire i cammini minimi, l'altra punta a trovare i nodi andando il più lontano possibile dal nodo sorgente.

Bisogna prestare particolare attenzione alle visite dei grafi, in quanto quest'ultimi potrebbero essere ciclici, di conseguenza la visita potrebbe essere infinita.

Vista BFS (breadth-first search) – visita in ampiezza

L'algoritmo esplora il grafo a partire dal nodo sorgente, e costruisce passo dopo passo un albero "T" radicato in "s" che al termine contiene tutti i nodi raggiungibili a partire da "s".

Durante l'algoritmo ogni vertice del grafo subisce tre stati: Inesplorato, aperto, chiuso. Per tenere traccia dei vertici inesplorati, aperti o chiusi l'algoritmo si avvale di una coda. Inoltre, quando l'algoritmo apre un vertice, tale vertice viene contrassegnato, in questo modo non si riaprirà un nodo marcato/contrassegnato. Evitando in questo modo eventuali cicli.

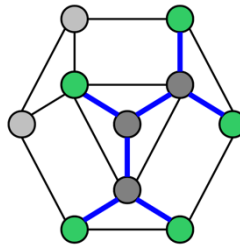
La visita in ampiezza quindi non solo visita i vertici, ma segue; quindi, seleziona quegli archi che hanno permesso di esplorare, quindi aprire tutti i vertici.

```

algoritmo BFS(Grafo G, vertice s) → albero
for each v in V do v.mark := false
T := s
F := new Queue()
F.enqueue(s) inserisco in coda
s.mark := true marco subito true quando inserisco il vertice in coda
s.dist := 0 sorgente a distanza 0 da sé stessa
while (F ≠ ∅) do finché nella coda non ci sono più vertici entro qui
  u := F.dequeue() tolgo e stampo il vertice
  "visita il vertice u"
  for each v adiacente a u do vado negli adiacenti
    if (not v.mark) then se non è marcato
      v.mark := true marco
      T := T ∪ v aggiungo all'albero di visita il nuovo vertice scoperto
      F.enqueue(v) inserisco il coda
      v.parent := u
      v.dist := u.dist + 1
    endif
  endfor
endwhile
return T

```

come si può vedere, viene creata una nuova coda, e si marca il vertice sorgente, e lo si inserisce in coda. Successivamente, fino a che la coda non è vuota, si toglie un vertice dalla coda, lo si visita (vedendo tutti gli adiacenti). Dato un adiacente non marcato, lo si apre (inserendolo in coda), lo si marca, lo si aggiunge all'albero di visita e si aumenta la distanza percorsa per scoprire quel nodo. Ad esempio, in questo istante

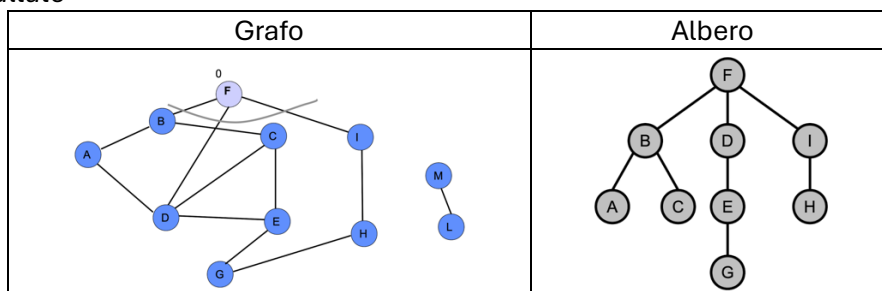


sono già stati aperti e visitati i nodi scuri, mentre in coda ci sono i nodi verdi, che, come si può vedere, sono gli ultimi. Quando si apriranno quei vertici si andranno a visitare gli stessi nodi che sono già in coda, quindi già marcati, di conseguenza, la

visita si interromperà e non continuerà all'infinito.

Costo: $\theta(n^2)$ con matrice di adiacenza || $\theta(n + m)$ con liste di adiacenza

Esempio di Risultato



La visita in ampiezza può avere diversi utilizzi, primo tra tutti capire se esiste un percorso da un vertice ad un altro (questo lo si riesce a fare partendo dall'arrivo, fino ad arrivare al punto di inizio salendo come un salmone e verificando la presenza degli archi).

In questo esempio, invece si può vedere la visita BFS in azione in un labirinto.



Visita DFS (Depth – First Search) – Visita in Profondità

Il procedimento di visita del grafo è praticamente identico, ovvero viene passato un grafo, e si visita il grafo vertice per vertice (creando un albero di visita) che ha per nodi i vertici esplorati.

L'unica differenza di questa visita è che si parte da un nodo "sorgente" e si cerca di andare il più lontano possibile dal nodo sorgente, per provare a scoprire i più vertici possibili. Quindi, a differenza della visita BFS (dove si aprivano i vertici e li si esplorava uno ad uno), qui si apre un nodo, (il sorgente) e si inizia a scappare dal nodo sorgente per provare a raggiungere (senza mai passare dal percorso già fatto) nuovi vertici. Quando sono già stati raggiunti tutti i vertici, si inizia ad andare a ritroso per tornare indietro an nodo sorgente. Finora non sembra tutto di nuovo, ma la particolarità è che si riesce a scoprire l'istante di visita di quel nodo, e l'istante di chiusura di quel nodo, dove per chiusura si intende l'istante temporale in cui l'avanzamento porterebbe a compiere nuovamente l'arco già seguito in precedenza.

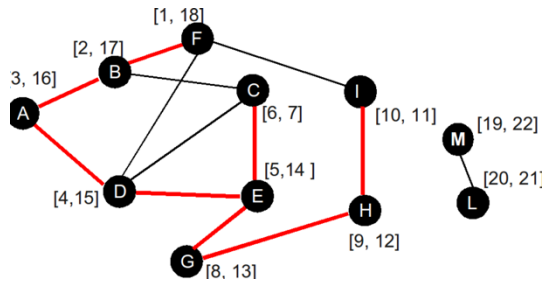
Un'altra particolarità è che questa funzione è ricorsiva ed inoltre permette di ricavare delle visualizzazioni del grafico che possono essere particolarmente utili ed interessanti. Inoltre, a differenza della BFS, questa visualizzazione permette di visualizzare tutti i vertici (anche quelli scollegati dal grafo principale).

```
global time := 0; //var.globale

algoritmo DFS (Grafo G)
  for each u in V do
    u.mark := white; Inizializzazione
    u.parent := nil;
  endfor
  for each u in V do
    if (u.mark == white) then
      DFS-visit(u);
    endif
  endfor

algoritmo DFS-visit(vertex u)
  u.mark := gray;
  time := time+1;
  u.dt := time;
  for each v adiacente a u do
    if (v.mark = white) then
      v.parent := u;
      DFS-visit(v); Ricorsione
    endif
  endfor
  "visita il vertex u"
  time := time+1;
  u.ft := time;
  u.mark := black;
  Passaggio finale, terminale. Da quel vertex non ci toro più
```

Come si può vedere c'è una funzione di facciata e una di "lavoro", dove questa funzione di "lavoro" è anche la funzione ricorsiva, che effettivamente scopre, marca e segna i vertici.

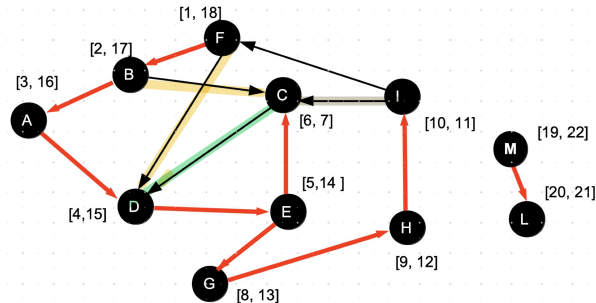


Come si può vedere ogni vertice viene marcato all'istante di apertura e all'istante di chiusura, e come se si aprissero e si chiudessero le parentesi in un'espressione i nodi che vengono scoperti per "più tardi" vengono marcato come chiusi per primi, questo perché quando la funzione nota che non ci sono più strade "nuove" l'ultimo nodo viene marcato come "chiuso" (:=nero). Di conseguenza il nodo

iniziale sarà anche l'ultimo nodo che verrà chiuso.

Costo: $\theta(n^2)$ con matrice di adiacenza || $\theta(n + m)$ con liste di adiacenza

Grafi Orientati



Come si può vedere in questo grafo orientato scopriamo che la visita DFS ci dà parecchie informazioni sul tipo di grafo.

Prendiamo per esempio l'arco verde. Come si può vedere se la scoperta di "C" è maggiore della scoperta di "D", ma la chiusura di "C" è minore della scoperta di "D" allora si ha un grafo ciclico, e questo tipo di arco viene chiamato arco all'indietro.

Se invece prendiamo l'arco giallo "B-C" vediamo che "B" viene scoperto prima di "C", ma "B" viene chiuso dopo "C" allora siamo davanti ad un classico arco all'avanti.

Se prendiamo l'arco marrone, e vediamo che la chiusura di "C" è minore dell'apertura di "I", allora siamo davanti ad un arco di attraversamento a sinistra.

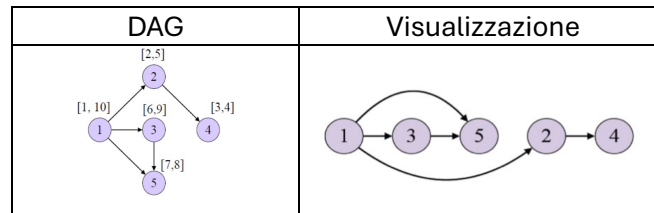
Concettualmente, l'arco più particolare è l'arco marrone ovvero gli archi che hanno un intervallo di visita tra vertici già chiusi.

- Consideriamo un grafo orientato e un arco (u,v) non incluso nella foresta DF. Identificano un ciclo
- Se $v.dt < u.dt$ e $u.ft < v.ft$ l'arco (u,v) è **all'indietro**
- Se $u.dt < v.dt$ e $v.ft < u.ft$ l'arco (u,v) è **in avanti**
- Se $v.ft < u.dt$ l'arco (u,v) è **di attraversamento a sx**
- NOTA: non possono esistere altri casi! (ovvero $u.ft < v.dt$)

Tramite questa rappresentazione è facile comprendere quando un grafo orientato è un grafo ciclico o un DAG, questo perché appena trovo un arco verde posso immediatamente classificare il grafo come un DAG, senza fare ulteriori analisi.

DAG

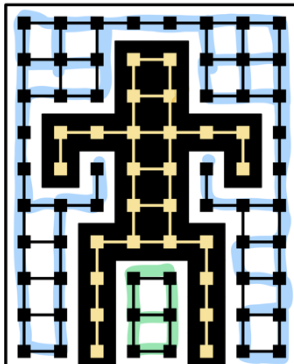
Dato un DAG è interessante creare il suo ordinamento topologico, ovvero creare una visualizzazione del grafo dove i vertici con il tempo di chiusura più alto compare per primo. In sistemi si parte dal più grande finish-time per arrivare al più piccolo finish-time, l'idea è quella di non aprire un nuovo vertice se il nuovo vertice è stato chiuso prima del successivo, per esempio, non faccio arrivare la merce in azienda quando l'azienda è chiusa, oppure non monto le ruote della macchina senza avere le sospensioni.



L'algoritmo è molto semplice, si usa una lista, e appena si chiude un vertice lo si inserisce in testa alla lista, in questo modo, l'ultimo vertice chiuso sarà l'ultimo inserito (in ordine temporale) in lista, ma il primo visualizzato.

Componenti Connesse – grafo non orientato

Si parla di componenti connesse per quei vertici di un grafo vicini tra di loro e che permettono un ciclo tra di loro e che hanno lo stesso indice di componente connesso.



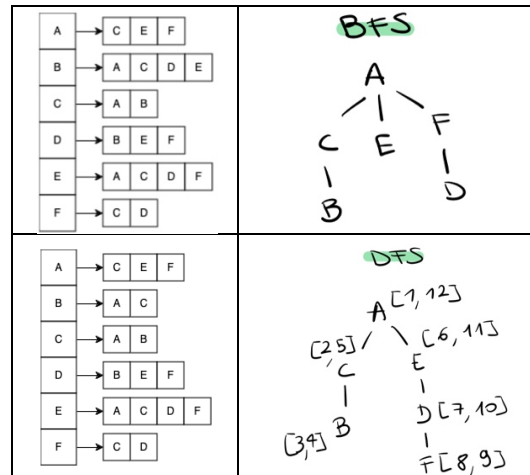
Colori uguali hanno lo stessi indice di componente connesso. A livello algoritmico si inizia a visitare un grafo, quando si entra in una parte di grafo ciclica si aggiorna il contatore e si assegna a tutti quei nodi del ciclo l'indice corrispondente a quel ciclo. Quando i vertici ciclici terminano, si inizia un altro insieme di nodi ciclici, ovvero un altro insieme di componenti connesse.

Componenti Fortemente Connesse – grafo orientato

Nei grafici orientati si parla di componenti fortemente connesse per tutti quei nodi che partendo da un vertice "x" ci portano ad "n" vertici distinti, e da quei "n" vertici distinti è possibile ritornare a quel vertice "x" preciso. Per trovare questi vertici fortemente connessi è necessario far partire una visita in ampiezza o profondità da un vertice e segnare in un insieme quali vertici sono stati raggiunti. Successivamente, è necessario invertire tutti gli archi, e vedere da quali "n" nodi si ritorna esattamente a "x", risegnare i nodi ottenuti e fare l'intersezione tra i due "insiemi" segnati.

BFS	DFS
Qui uso una coda per visualizzare in modo "ordinato i punti"	Qui visito un vertice e subito le adiacenze del vertice appena scoperto
Aggiunge distanza da vertice sorgente	Aggiunge istante di scoperta e arrivo
Conserva il padre del vertice	Conserva il padre del vertice

Esempio / Esercizi



14^ Lezione

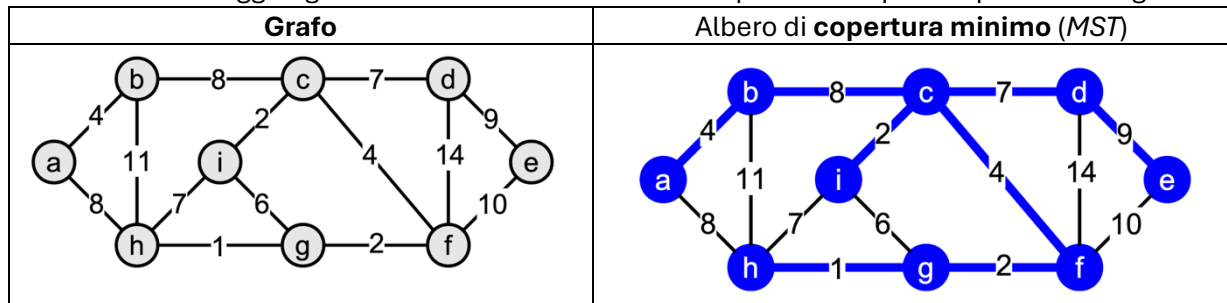
Minimum Spanning Tree (MST)

Definizione

Dato un grafico non orientato, completamente connesso e pesato (ovvero un costo per il passaggio da un vertice ad un altro percorrendo un nodo, dove tale costo può essere anche pari ad infinito) è richiesto di trovare un albero di copertura minimo.

Dove per un arco di copertura si intende un sottoinsieme di archi (un sotto-grafo) aciclico che raggiunge tutti gli archi. Dove per alberi di copertura si intende l'albero di copertura che ha la sommatoria dei pesi degli archi minore possibile.

In modo iterativo si aggiunge un arco fino a restituire il MST – problema: qual è il prossimo migliore?

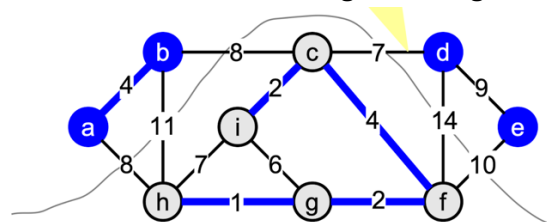


Nota! L'MST non è unico.

Proprietà

Regola del taglio: Avendo già una serie di archi blu, quale è il miglior prossimo arco da collegare?

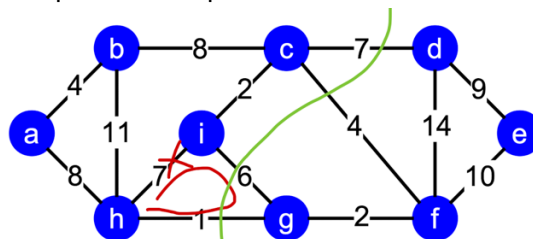
Questa regola si pone di tagliare a metà un grafo, separando dei vertici sopra e sotto a una linea, e **non esiste nessun'altro arco "blu" capace di tagliare la linea** (nessuno è ancora stato scelto come migliore). Quindi ho un taglio tra vertici e almeno uno tra gli archi tagliati deve essere scelto.



Un approccio greedy si presuppone di rendere blu l'arco tagliato con peso minore e questo è esattamente quello che esegue la regola del taglio. A mano a mano si taglia il grafo, si sceglie l'arco con il peso minore e si continua.

Questo arco quindi per noi non esiste più, bisognerà trovare un'altra strada.

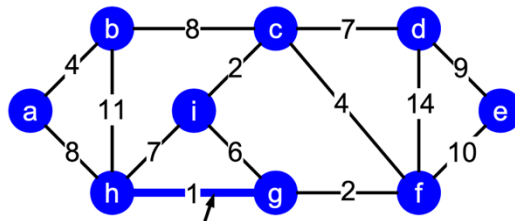
I vertici vengono collegati in blu, e tutti questi vertici prima o poi dovranno essere collegati tramite archi. Si inizia a visualizzare gli archi in ordine non decrescente di peso e si decide se usare la regola del ciclo o del taglio partendo da quell'arco di peso minore.



Sintesi: evito gli archi che vanno a ciclare ed invalidare la proprietà MST

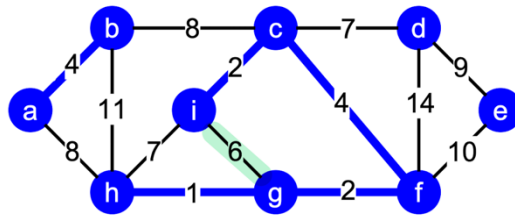
costo: $\theta(m \log m)$

Esempio



In questa situazione, si prende in considerazione l'arco di peso minimo, ovvero l'arco che va da "g" ad "f", successivamente si vedono i rappresentanti del vertice "g" e del vertice "f". Se i rappresentanti sono diversi (come in questo caso), si collegano i vertici tramite quell'arco, e si crea un unico sottoinsieme che contiene i vertici "h, g, f".

Dopo diverse iterazioni ci troveremo a confrontare nuovamente l'arco attuale di peso minore, che, come viene raffigurato dall'immagine sottostante, è l'arco di peso 6, che collega i vertici "i", "g".

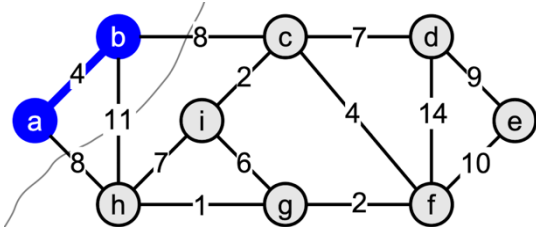


Ora si vede che i vertici "i", "g" hanno lo stesso rappresentante, infatti fanno parte dello stesso sottoinsieme, di conseguenza, quell'arco verrà scartato.

Algoritmo di Prim

Invece di costruire prima una foresta di alberi con un solo nodo, si preferisce costruire un unico albero, con una radice, e successivamente aggiungere rami a quell'albero. Qui l'idea è quindi quella di accrescere un albero che di partenza ha una sola radice.

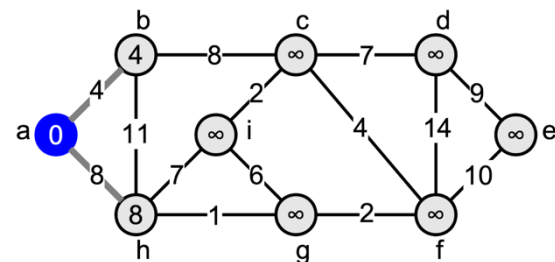
L'algoritmo inizia prendendo un vertice qualsiasi (ipoteticamente il primo del grafo); con la regola del taglio prendo l'arco con peso minore che parte da quel vertice.



Ora dalla situazione riportata a sinistra scelgo nuovamente l'arco che parte da due dei vertici selezionati (a, b) che ha peso minore, in questo caso verrà scelto l'arco di peso 8 (uno dei due indifferentemente). Quindi, in sintesi, l'algoritmo di Prim usa sempre la regola del taglio fino a che non ci sono " $n-1$ " archi creati.

Il problema principale di questo algoritmo è quello di effettuare n confronti per trovare ogni volta l'arco con peso minimo; quindi, si può usare una struttura dati del tipo "coda con priorità" di supporto.

$$Q = \{ (b,4), (h,8) \}$$



"Q" è quindi la coda con priorità e il numero all'interno del vertice "b" è il peso dell'arco con peso minore che mi permette di arrivare a quel determinato vertice. Infatti, quando entrerà il vertice "b" a far parte dell'MST gli archi adiacenti saranno due, dove uno fra quei due di peso 11 porta al vertice "h". Ma questo peso è maggiore del numero contenuto all'interno del vertice "h", di conseguenza, quell'arco che parte da "b" di peso 11 non verrà scelto.

Situazione particolare

$Q = \{ (g,1), (i,7), (c,8) \}$	$Q = \{ (f,2), (i,6), (c,8) \}$
In questo istante so che il peso minimo per arrivare a "i" è di 7	All'istante $k + 1$ mi accorgo che grazie al nuovo vertice "g" appena collegato posso raggiungere "i" con un arco di peso minore, di conseguenza aggiorno la chiave (il valore) relativo al vertice "i" con il nuovo peso minore
$Q = \{ (e,9) \}$	

Pseudocodice

```

integer[] Prim-MST(Grafo G=(V,E,w), nodo s)
  for each v ∈ V do
    v.dist ← ∞; inizializzazione
    v.parent ← null; inizializzazione
    v.bool ← false; Dice se il nodo è già presente all'interno dell'MST
  endfor
  s.dist ← 0; coda con priorità per permettere costo computazionale basso
  CodaPriorita<integer, double> Q; Q.insert(s, s.dist);
  while (not Q.isEmpty()) do
    u ← Q.find(); Q.deleteMin(); u.bool ← true;
    for each v ∈ u.adj do iterazione sulle adiacenze non segnate true dal bool
      if (not v.bool) then
        if (v.dist == ∞) then se è la prima volta che vedo quel vertice
          Q.insert(v, w(u,v));
          v.dist ← w(u,v);
          v.parent ← u;
        elseif (w(u,v) < v.dist) then ho già scoperto, ma aggiorno
          Q.decreaseKey(v, w(u,v));
          v.dist ← w(u,v);
          v.parent ← u;
        endif
      endif
    endfor
  endwhile

```

Costo: per n volte si elimina dalla coda il minimo (per forza in quanto si vogliono collegare tutti i vertici). Per $n - 1$ volte si inserisce nella coda un vertice, e al massimo n volte modificherò il contenuto all'interno della coda. L'eliminazione costa $n \log n$ l'inserimento $n \log n$, la modifica $m \log n$ quindi il costo finale è di: $\Theta(m \log n)$.

15^ Lezione

Cammini Minimi

Problema

Dato un grafo orientato e pesato voglio trovare il cammino di peso minimo che parte da punto “a” e arriva a punto “b”. Da notare che il cammino minimo è completamente diverso dall’MST, in quanto il problema è locale, si ha uno specifico punto di partenza e uno specifico punto di arrivo; quindi, mi interessa come arrivare nel modo più efficiente partendo da un vertice.

Ci sono diversi varianti per il cammino minimo, nel caso sopra descritto (variante tipica) ha un grafo, un vertice sorgente e un vertice “b”. Tuttavia, ci sono anche altre versioni, una di queste prende in input solo un vertice sorgente e restituisce un albero dei cammini minimi, che contiene tutti i cammini minimi partendo dal vertice sorgente e andando in ogni vertice di arrivo. L’ultima variante cerca il cammino minimo per ogni altra coppia di possibili vertici; quindi, cerca tutti i cammini minimi presenti in un grafo, in questo caso l’unico input è il grafo stesso.

Condizioni di Esistenza

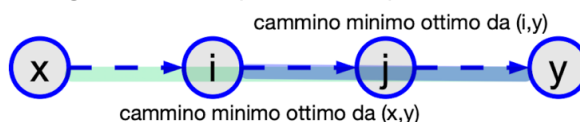
Il cammino minimo non esiste se il punto di arrivo non è raggiungibile, oppure se sono presenti cicli che comprendono archi di costo negativo. Se queste due condizioni non sono verificate, allora esiste il cammino minimo.

Esistenza

Nei grafi orientati e pesati, privi di cicli negativi, è sempre possibile individuare un cammino semplice di costo minimo. Dove per cammino semplice si intende un percorso che non attraversa lo stesso vertice più di una volta.

Proprietà:

- Posso sempre trasformare un cammino rimuovendo i cicli
- La rimozione di eventuali cicli diminuisce il costo, o lo lascia invariato
- Il numero di cammini semplici è sempre finito, pertanto esiste sempre un minimo
- Dato un cammino ottimale, ogni sua sotto-parte è sempre un cammino minimo ottimo



Sia “s” un vertice prefissato in un grafo orientato pesato, allora esiste un albero “T” radicato in “s” che contiene i vertici raggiungibili da “s” tale che ogni cammino in “T” sia un cammino di costo minimo. In sintesi, un albero può essere utilizzato per rappresentare tutti i cammini minimi.

Distanza tra vertici in un Grafo

La distanza d_{xy} è il costo di un cammino minimo che connette “x” ed “y”. Concettualmente, nella visita BFS contavamo quanti archi sono stati attraversati, qui sostituiamo gli archi attraversati con il peso di tali archi. Il cammino minimo, infatti è la sommatoria minore di archi attraversati.

$$d_{xy} = \begin{cases} w(\pi_{xy}^*) & \text{se esiste un cammino di costo minimo } \pi_{xy}^* \\ +\infty & \text{altrimenti} \end{cases}$$

Dove $d_{vv} = 0$ e in un grafo/sotto-grafo di forma pseudotriangolare (con tre vertici) si ha:

$d_{xz} \leq d_{xy} + d_{yz}$, dove so che il cammino d_{xz} è il costo minimo da pagare per andare da “x” a “z”.

L’algoritmo di Bellman-Ford si basa su iterazione ed è capace di rilassare (:= diminuire) il costo definito dalla disequazione triangolare. In pratica, si vanno a costruire diverse disequazioni triangolari (inizializzate con un costo pari a infinito), successivamente si vanno ad approssimare, quindi ridurre il costo definito dalla triangolazione passo dopo passo. Al passo i-esimo si avrà il cammino minimo.

$$d_{sv} \leq d_{su} + w(u, v)$$

non ne conosco ancora il peso

conosco il loro peso, posso effettuare questi dati conosciuti come approssimazione

Da notare che si deduce che l'arco (u, v) fa parte di un cammino di costo minimo se e solo se $d_{sv} = d_{su} + w(u, v)$ [dove $w := \text{weight}$]

- Supponiamo di mantenere una stima $D_{sv} \geq d_{sv}$ della lunghezza del cammino di costo minimo tra s e v

Allora posso effettuare dei passi di rilassamento finché $D_{sv} = d_{sv}$ dove "D" è l'approssimazione per eccesso e "d" è il peso ottimale. Allora posso definire la tecnica di rilassamento come segue:

```
if (  $D_{su} + w(u, v) < D_{sv}$  ) then  $D_{sv} \leftarrow D_{su} + w(u, v)$ 
```

Pseudocodice – Algoritmo Bellman-Ford

```
double[1..n] BellmanFord(Grafo G=(V,E,w), int s)
int n ← G.numNodi();
int pred[1..n], v, u;
double D[1..n];
for v ← 1 to n do
    D[v] ← +∞;
    pred[v] ← -1;
endfor
D[s] ← 0;
for int i ← 1 to n - 1 do per n-1 iterazioni cerco di fare dei rilassamenti
    for each (u,v) in E do guardo tutti gli archi, e cerco dei rilassamenti
        if (  $D[u] + w(u, v) < D[v]$  ) then trovo rilassamenti
             $D[v] \leftarrow D[u] + w(u, v)$ ; se ci riesco, aggiorno.
             $pred[v] \leftarrow u$ ; quando rilassiamo la distanza, devo anche ricordare
            il padre (l'arco) che mi ha permesso di fare il
            rilassamento, in quanto ipotizziamo che quell'arco
            sia l'ultimo arco del cammino minimo
        endif
    endfor
endfor
// eventuale controllo per cicli negativi (vedi seguito)
return D;
```

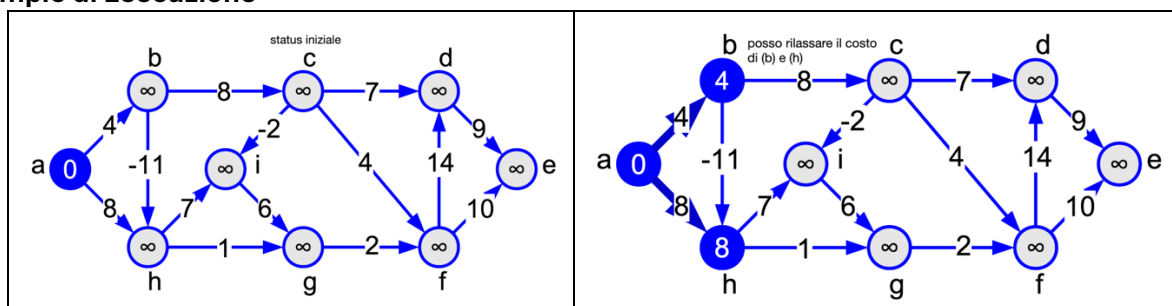
I nodi del grafo sono identificati dagli interi 1, ... n

$D[v]$ = (stima della) distanza del nodo v dalla sorgente s

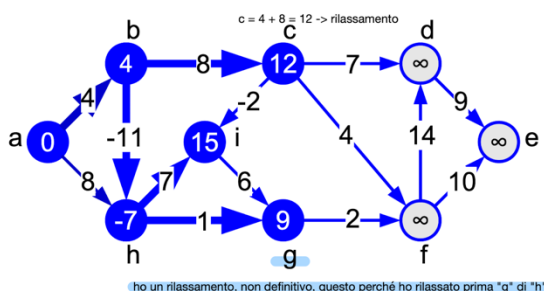
$pred[v]$ = predecessore del nodo v sul cammino di costo minimo che collega s con v

Costo: $O(n * m)$

Esempio di Esecuzione

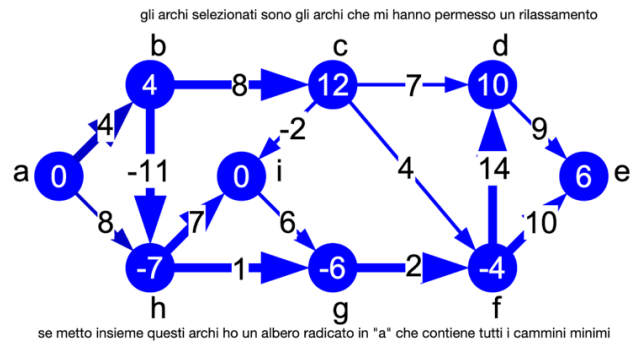


L'algoritmo punta a rivisitare per "n-1" volte tutti i cammini fino ad ora inizializzati. All'ultima iterazione si è sicuri ogni cammino è effettivamente il minimo.



Ad esempio, alla terza iterazione scopro i nuovi cammini e riguardo i vecchi. Qui a lato, infatti si può vedere che alla terza iterazione si aggiorna il valore dei nodi "c", "g" "i" da infinito a un valore ragionevolmente più basso (approssimazione che magari non è definitiva). Mentre il nodo "h", già noto assume il valore di -7, questo perché l'algoritmo ha trovato un'approssimazione migliore.

La ricerca di queste approssimazioni (rilassamenti) continua per $n - 1$ volte, fino a trovare un cammino



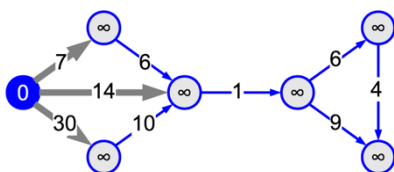
Da notare che l'algoritmo di Bellman – Ford funziona anche con archi di peso negativo, ma lo pseudocodice precedente funziona anche se sono presenti dei cicli di peso negativo, quindi tecnicamente lo pseudocodice sarebbe sbagliato. Per ovviare a questo problema basta aggiungere una riga di codice prima del *return*. Questa riga di codice è un ciclo che permette di rilevare ulteriori rilassamenti, e se tali sono presenti, allora si ha la sicurezza di avere un ciclo di costo negativo. Di conseguenza, se un ciclo di costo negativo viene rilevato, l'algoritmo si interrompe.

```
// eventuale controllo per cicli negativi
for each (u,v) in E do
    if ( D[u] + w(u,v) < D[v] ) then
        error "Il grafo contiene cicli negativi"
    endif
endfor
```

Algoritmo di Dijkstra

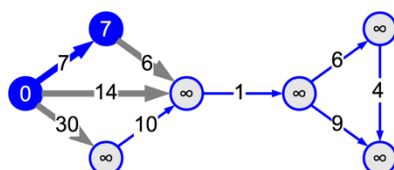
Se poniamo tutti **gli archi di un grafo non negativi**, allora l'algoritmo più efficiente è questo.

L'algoritmo è un algoritmo greedy, dove si va a dare un peso ad ogni vertice, pari al peso dell'arco che lo raggiunge + peso del vertice da cui parte l'arco di arrivo. L'algoritmo, quindi, va a prendere ed inserire in una coda con priorità ogni arco utile ($:=$ arco che non ha un costo maggiore di quello già individuato e scelto) e sceglie sempre l'arco di peso minore.



Partendo dal vertice blu si individuano tre archi possibili, il primo di peso 7, l'altro di peso 14 e l'altro di peso 30. Allora viene scelto l'arco di peso minimo, e si scopre il nuovo vertice, e si assegna a quel vertice il costo di $0 + 7 = 7$.

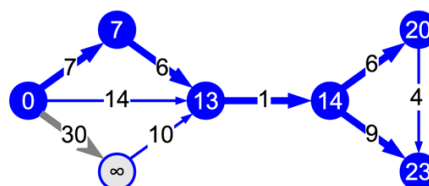
All'istante due, invece si ha una situazione del tipo seguente. Si individuano tre archi, due di quelli



(archi superiori) vanno allo stesso nodo. Con l'arco centrale arrivo al nodo centrale con un costo pari a 14, mentre con l'arco superiori arrivo al nodo centrale con un costo pari a:

$$7 (\text{vertice arco di partenza}) + 6 (\text{peso arco}) = 13.$$

Di conseguenza, verrà scelto l'arco superiore, il costo associato al vertice sarà di 13, ma soprattutto, alla prossima iterazione l'arco centrale di peso 14 non verrà più utilizzato, in quanto non utile. Da notare, che l'ultimo arco, il più pesante, quindi l'ultimo scelto sarà proprio l'arco di peso 30.



Pseudocodice

```
double[1..n] Dijkstra(Grafo G=(V,E,w), int s)
int n ← G.numNodi();
int pred[1..n], v, u;
double D[1..n];
for v ← 1 to n do
    D[v] ← +∞;
    pred[v] ← -1;
endfor
D[s] ← 0;
CodaPriorita<int, double> Q; Q.insert(s, D[s]);
while (not Q.isEmpty()) do
    u ← Q.find(); Q.deleteMin();
    for each v adiacente a u do
        if (D[v] == +∞) then
            D[v] ← D[u] + w(u,v);
            Q.insert(v, D[v]);
            pred[v] ← u;
        elseif (D[u] + w(u,v) < D[v]) then
            Q.decreaseKey(v, D[u] + w(u,v));
            D[v] ← D[u] + w(u,v);
            pred[v] ← u;
        endif
    endfor
endwhile
return D;
```

Trova e rimuovi il nodo con distanza minima

Somiglia all'algoritmo di Prim (MST), ma usa una priorit  diversa

Rendi $D[u]+w(u,v)$ la nuova distanza di v da s

L'inizializzazione ha costo n , le operazioni *find* e *deletemin* hanno costo pari a $\log n$, e queste operazioni sui vertici le si fanno per un massimo di n volte. Le operazioni *insert* e *decreaseKey* hanno costo pari a $\log n$ e vengono fatte per un massimo di m volte. Di conseguenza, il costo complessivo finale   di $O(m \log n)$.

Lemma di Dijkstra

Questo lemma dimostra il perch  dell'algoritmo, infatti, supponendo un percorso gi  completo, terminato e finito, ma con un arco mancante. L'arco mancante viene scelto tra gli archi di peso minore; quindi, se tale scelta viene ripetuta ricorsivamente si ha la sicurezza di avere il cammino di peso minimo. In sintesi, creando porzioni di cammini minimi si ha la certezza di creare un unico cammino di peso minimo.

Algoritmo di Floyd – Warshall

  un algoritmo che si basa sulla programmazione dinamica, che si pu  utilizzare con grafi orientati (anche con archi negativi), ma come al solito senza cicli di costo negativo.

I vertici vengono inizializzati come segue: $\{1, 2, \dots, n\}$, in quanto useremo i vertici come indici di una matrice. Sia inoltre D_{xy}^k la distanza minima dal nodo x al nodo y , con il parametro k visto come parametro ausiliario per semplificare il problema, nell'ipotesi in cui i nodi intermedi possano appartenere esclusivamente all'insieme $\{1, \dots, k\}$. Quando k sar  uguale a n , il nostro problema sar  risolto.

Soluzioni Algebriche

Sia D_{xy}^0 la distanza minima tra x e y nell'ipotesi si non poter passare per alcun nodo intermedio (parametro $k = 0 \rightarrow$ no nodi intermedi utilizzati) posso calcolare D_{xy}^0 come:

$$D_{xy}^0 = \begin{cases} 0 & \text{se } x = y \\ w(x, y) & \text{se } (x, y) \in E \\ \infty & \text{se } (x, y) \notin E \end{cases}$$

Dove i cappi da x a x non esistono, in quanto sarebbero considerati/visti come cicli di costo negativo. Nel caso generale, per andare da x ad y usando solo nodi intermedi in $\{1, \dots, k\}$ ho due possibilit :

1. Non passare mai per il nodo k , dove la distanza   D_{xy}^{k-1}
2. Passo per il nodo k , per la propriet  di sottostruttura ottima la distanza in tal caso   di:

$$D_{xk}^{k-1} + D_{ky}^{k-1}$$

Di conseguenza, $D_{xy}^k = \min\{D_{xy}^{k-1}; D_{xk}^{k-1} + D_{ky}^{k-1}\}$.

Da notare che per calcolare questo algoritmo serve una matrice tridimensionale, dove k è l'indice di profondità. Di conseguenza, il costo è pari a $O(n^3)$. Ovviamente, senza miglioramento è difficile immaginare l'esecuzione del codice a mente, perché è difficile scrivere e mappare una matrice tridimensionale.

```
double[1..n,1..n] FloydWarshall( G=(V,E,w) )
int n ← G.numNodi();
double D[1..n, 1..n, 0..n]; int x, y, k; inizializzazione
for x ← 1 to n do casi base e predisposizione della matrice
  for y ← 1 to n do
    if (x == y) then D[x,y,0] ← 0;
    elseif ((x,y) ∈ E) then D[x,y,0] ← w(x,y);
    else D[x,y,0] ← +∞;
  endif
endfor
for k ← 1 to n do considerazione di tutte le coppie possibili x,y
  for x ← 1 to n do
    for y ← 1 to n do
      D[x,y,k] ← D[x,y,k-1];
      if (D[x,k,k-1] + D[k,y,k-1] < D[x,y,k]) then
        D[x,y,k] ← D[x,k,k-1] + D[k,y,k-1];
      endif
    endfor
  endfor
endfor
// eventuale controllo per cicli negativi (vedi seguito)
return D[1..n, 1..n, n];
```

$D_{xy}^k = \min \{ D_{xy}^{k-1}, D_{xk}^{k-1} + D_{ky}^{k-1} \}$

Per visualizzare l'esecuzione dell'algoritmo, si "affetta" la matrice 3x3x3 in fette di k , in questo modo si riesce a lavorare con matrici bidimensionali. Questo algoritmo, per ora, non definisce il cammino di costo minimo, ma stabilisce solo i costi.

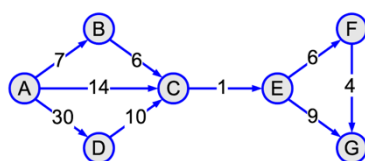
Ottimizzazione

Guardando lo pseudocodice si capisce che da un punto in poi parte della matrice non serve più; quindi, si può lavorare in una matrice bidimensionale tutto il tempo, utilizzando uno spazio in memoria di $O(n^2)$. È un po' difficile da capire, ma con un esempio (eseguito a mano) su una matrice bidimensionale si può capire bene l'ottimizzazione.

```
double[1..n,1..n] FloydWarshall2( G=(V,E,w) )
int n ← G.numNodi();
double D[1..n, 1..n];
int x, y, k, next[1..n, 1..n];
for x ← 1 to n do
  for y ← 1 to n do
    if (x == y) then D[x,y] ← 0;
    elseif ((x,y) ∈ E) then D[x,y] ← w(x,y);
    else D[x,y] ← +∞;
  endif
endfor
for k ← 1 to n do serve solo per dire che l'aggiornamento deve essere eseguito k volte
  for x ← 1 to n do ora il vecchio valore non viene più incollato ed eventualmente aggiornato.
    for y ← 1 to n do Adesso lavoro sempre sulla stessa matrice, e se ne ho bisogno, aggiorno.
      if (D[x,k] + D[k,y] < D[x,y]) then
        D[x,y] ← D[x,k] + D[k,y];
      endif
    endfor
  endfor
endfor
return D;
```

Costo di spazio $O(n^2)$, costo di tempo $O(n^3)$.

Esempio



D =	A ← sorgente	B	C	D	E	F	G
A	0	7	14	30	Inf	Inf	Inf
B	Inf	0	6	Inf	Inf	Inf	Inf
C	Inf	Inf	0	Inf	1	Inf	Inf
D	Inf	Inf	10	0	Inf	Inf	Inf
E	Inf	Inf	Inf	Inf	0	6	9
F	Inf	Inf	Inf	Inf	Inf	0	4
G	Inf	Inf	Inf	Inf	Inf	Inf	0

Eseguiamo il codice sullo stesso grafo dello scorso algoritmo, dove vediamo lo status iniziale composto da 0 su tutta la diagonale; infatti, pago 0 per andare dal vertice A al vertice A. Inoltre, se non ho ancora trovato una strada per arrivare ai vertici inserisco infinito. Adesso inizio ad usare il vertice sorgente come vertice intermedio, provando a migliorare quindi le distanze stimate. Come si può vedere, da A partono solo tre archi, quelli già scoperti; quindi, non posso utilizzare il vertice A come miglioramento. Ora passo al vertice B, cercandolo di scoprire eventuali miglioramenti.

Il punto di passaggio k adesso è uguale a B ; infatti, da B posso ottenere dei miglioramenti. In questo momento, devo capire la riduzione del costo di questi miglioramenti.

D =

uso come punto di passaggio K

	A	B	C	D	E	F	G
A	0	7	13	30	Inf	Inf	Inf
B	Inf	0	6	Inf	Inf	Inf	Inf
C	Inf	Inf	0	Inf	1	Inf	Inf
D	Inf	Inf	10	0	Inf	Inf	Inf
E	Inf	Inf	Inf	Inf	0	6	9
F	Inf	Inf	Inf	Inf	Inf	0	4
G	Inf	Inf	Inf	Inf	Inf	Inf	0

Vedo che da A riesco ad arrivare a B pagando 7, ma vedo che da B riesco arrivare a C , pagando 6, quindi $7 + 6 = 13$, che è minore del peso dell'arco che va da A e arriva a C .

All'iterazione successiva si passa a C e si ottengono altri miglioramenti, che è possibile vedere nell'immagine sottostante. Combinando quindi le

istanze colonne – righe è possibile ottenere dei miglioramenti. Ad ogni iterazione si riguardano quindi tutti i vertici intermedi già prima scoperti e si riguardano i possibili miglioramenti. È facile capire il motivo di un costo così alto.

D =

	A	B	C	D	E	F	G
A	0	7	13	30	14	Inf	Inf
B	Inf	0	6	Inf	7	Inf	Inf
C	Inf	Inf	0	Inf	1	Inf	Inf
D	Inf	Inf	10	0	11	Inf	Inf
E	Inf	Inf	Inf	Inf	0	6	9
F	Inf	Inf	Inf	Inf	Inf	0	4
G	Inf	Inf	Inf	Inf	Inf	Inf	0

Individuazione di cicli negativi

```
// eventuale controllo per cicli negativi
for x ← 1 to n do
  if ( D[x,x,n] < 0 ) then
    error "Il grafo contiene cicli negativi"
  endif
endfor
```

Stampa cammini

Per andare a stampare i cammini è necessaria un'altra struttura dati che viene chiamata **next**

```
double[1..n,1..n] FloydWarshall2( G=(V,E,w) )
int n ← G.numNodi();
double D[1..n, 1..n];
int x, y, k, next[1..n, 1..n];
for x ← 1 to n do
  for y ← 1 to n do
    if (x == y) then
      D[x,y] ← 0;
      next[x,y] ← -1; sto fermo, non esiste un vertici per andare dallo stesso punto allo stesso punto
    elseif ((x,y) ∈ E) then
      D[x,y] ← w(x,y);
      next[x,y] ← y; contiene tutti i cammini minimi
    else
      D[x,y] ← +∞;
      next[x,y] ← -1; sosotutisce a infinito -1 per leggibilità
    endif
  endfor
endfor
for k ← 1 to n do
  for x ← 1 to n do
    for y ← 1 to n do
      for y ← 1 to n do
        if (D[x,k] + D[k,y] < D[x,y]) then
          D[x,y] ← D[x,k] + D[k,y];
          next[x,y] ← next[x,k]; aggiorna per avere la tabella next completa
                                rappresenta il caso in cui è utile usare un nuovo vertice
                                "k" di passaggio
        endif
      endfor
    endfor
  endfor
endfor
return D;
```

Questo algoritmo è molto utile perché dice esplicitamente tutti passi intermedi da fare per andare da un vertice ad un altro. *Guardare le slides relative al capitolo per vedere l'ultimo passo di esecuzione.*